

计算机组成原理

讲师：老汤

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



第一章：计算机系统概述

第二章：总线系统

第三章：主存储器

第四章：数据的表示与运算

第五章：指令系统

第六章：中央处理器 (CPU)

第七章：输入输出 (I/O) 系统



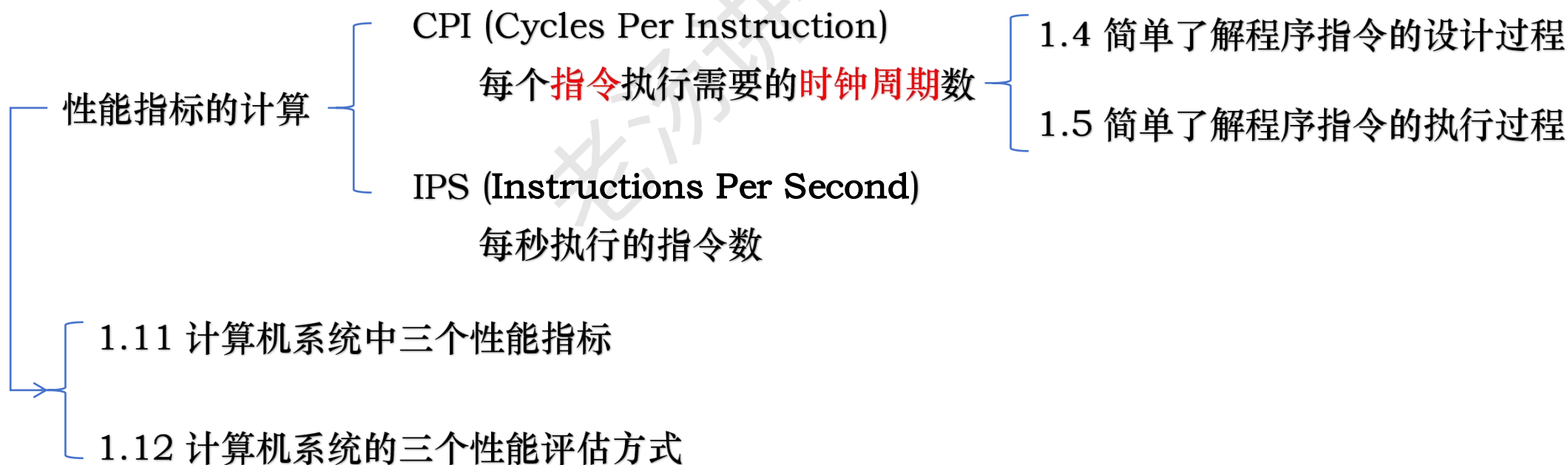
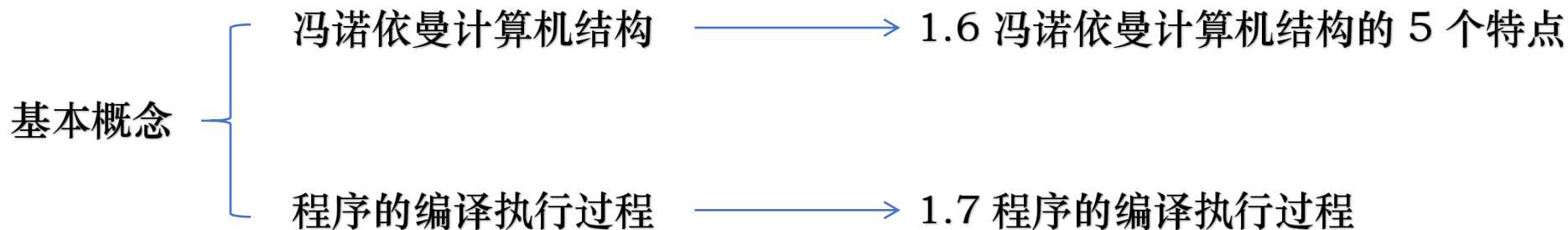
老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

408 如何考本章的内容?



第一章：计算机系统概述

用简单例子来看组成原理到底学什么内容

二进制、十进制和十六进制之间的转换

程序指令

简单了解程序指令的设计过程

简单了解程序指令的执行过程

冯诺依曼计算机结构的 5 个特点

程序的编译执行过程

什么是机器语言、汇编语言、高级语言？

什么是汇编器、解释器以及编译器？

编译执行 vs 解释执行

程序员最重要的能力：抽象能力

器件和逻辑电路（补补数字电路的基础）

计算机系统的层次结构

理解抽象分层思想

计算机系统中三个性能指标

时钟周期

主频

CPI

计算机系统的三个性能评估方式

指令执行速度

基准程序

阿姆达尔 (Amdahl) 定律

计算机体系结构中的 7 个伟大思想：

① 使用抽象分层简化设计



② 加速经常性事件

③ 通过并行提高性能

④ 通过流水线提高性能

⑤ 通过预测提高性能

⑥ 存储层次

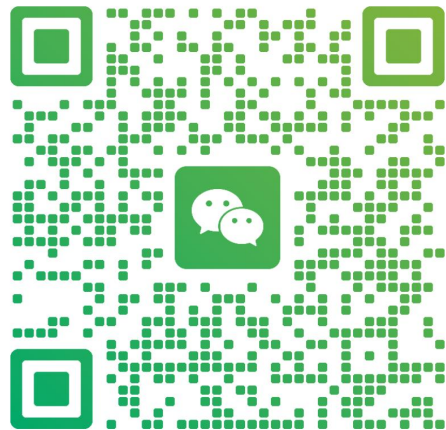
⑦ 通过冗余提高可靠性

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

第一章：计算机系统概述

用简单例子来看组成原理到底学什么内容

二进制、十进制和十六进制之间的转换

程序指令

简单了解程序指令的设计过程

简单了解程序指令的执行过程

冯诺依曼计算机结构的 5 个特点

程序的编译执行过程

什么是机器语言、汇编语言、高级语言？

什么是汇编器、解释器以及编译器？

编译执行 vs 解释执行

程序员最重要的能力：抽象能力

器件和逻辑电路（补补数字电路的基础）

计算机系统的层次结构

理解抽象分层思想

计算机系统中三个性能指标

时钟周期

主频

CPI

计算机系统的三个性能评估方式

指令执行速度

基准程序

阿姆达尔 (Amdahl) 定律

如何正确使用 408 考研真题？

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

尽早开始做真题！！！！

第一遍分门别类的去做！！！！

留最近两年的真题不要做，用于最后的摸底估计



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

```
#include <stdio.h>
```

```
int main() {  
    printf("Hello World\n");  
    return 0;  
}
```

老汤讲408



机械硬盘



固态硬盘



hello.c

硬盘

计算机组成原理

讲师：老汤

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

第一章：计算机系统概述

第二章：总线系统

第三章：主存储器

第四章：数据的表示与运算

第五章：指令系统

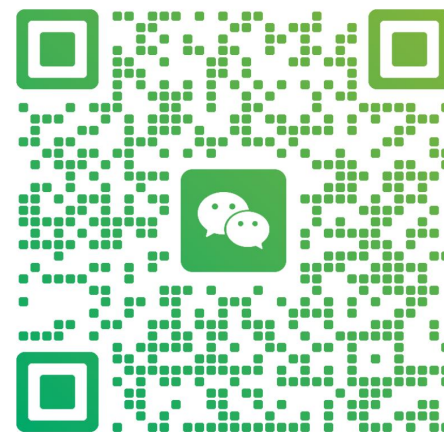
第六章：中央处理器 (CPU)



第七章：输入输出 (I/O) 系统



老汤
浙江 杭州

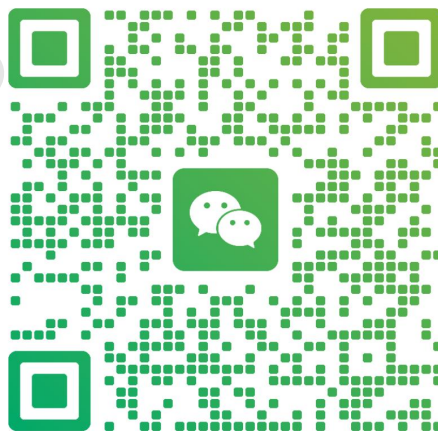


扫一扫上面的二维码图案，加我为朋友。

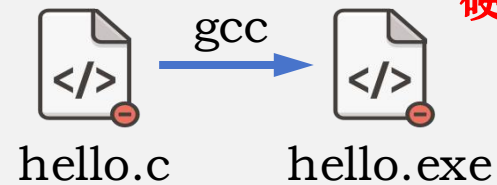
加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

```
#include <stdio.h>
```

```
int main() {  
    printf("Hello World\n");  
    return 0;  
}
```



扫一扫上面的二维码图案，加我为朋友。



```
#include <stdio.h>
```

```
int main() {  
    printf("Hello World\n");  
    return 0;  
}
```



```
00401460 <_main>:  
401460:      55                push    %ebp  
401461:      89 e5             mov     %esp,%ebp  
401463:      83 e4 f0          and     $0xfffffffff0,%esp  
401466:      83 ec 10          sub     $0x10,%esp  
401469:      e8 52 05 00 00    call    4019c0 <__main>  
40146e:      c7 04 24 64 50 40 00 movl    $0x405064,(%esp)  
401475:      e8 e6 25 00 00    call    403a60 <_puts>  
40147a:      b8 00 00 00 00    mov     $0x0,%eax  
40147f:      c9               leave  
401480:      c3               ret
```



hello.c

gcc



hello.exe

硬盘

计算机组成原理

讲师：老汤

第一章：计算机系统概述

第二章：总线系统

第三章：主存储器

第四章：数据的表示与运算

 **第五章：指令系统**

第六章：中央处理器 (CPU)

第七章：输入输出 (I/O) 系统

```
#include <stdio.h>
```

```
int main() {  
    printf("Hello World\n");  
    return 0;  
}
```



```
00401460 <_main>:
```

401460:	55	push	%ebp
401461:	89 e5	mov	%esp,%ebp
401463:	83 e4 f0	and	\$0xfffffffff0,%esp
401466:	83 ec 10	sub	\$0x10,%esp
401469:	e8 52 05 00 00	call	4019c0 <__main>
40146e:	c7 04 24 64 50 40 00	movl	\$0x405064, (%esp)
401475:	e8 e6 25 00 00	call	403a60 <_puts>
40147a:	b8 00 00 00 00	mov	\$0x0,%eax
40147f:	c9	leave	
401480:	c3	ret	



hello.c

gcc



hello.exe

硬盘

```
#include <stdio.h>

int main() {
    printf("Hello World\n");
    return 0;
}
```



```
push    %ebp
mov     %esp,%ebp
and     $0xfffffffff0,%esp
sub     $0x10,%esp
call    4019c0 <__main>
movl    $0x405064, (%esp)
call    403a60 <_puts>
mov     $0x0,%eax
leave
ret
```

指令序列

+

数据



hello.c

gcc



hello.exe

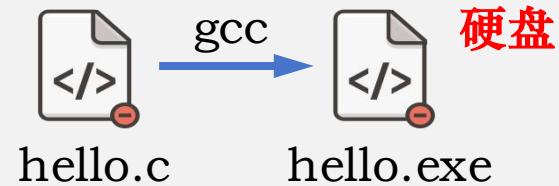
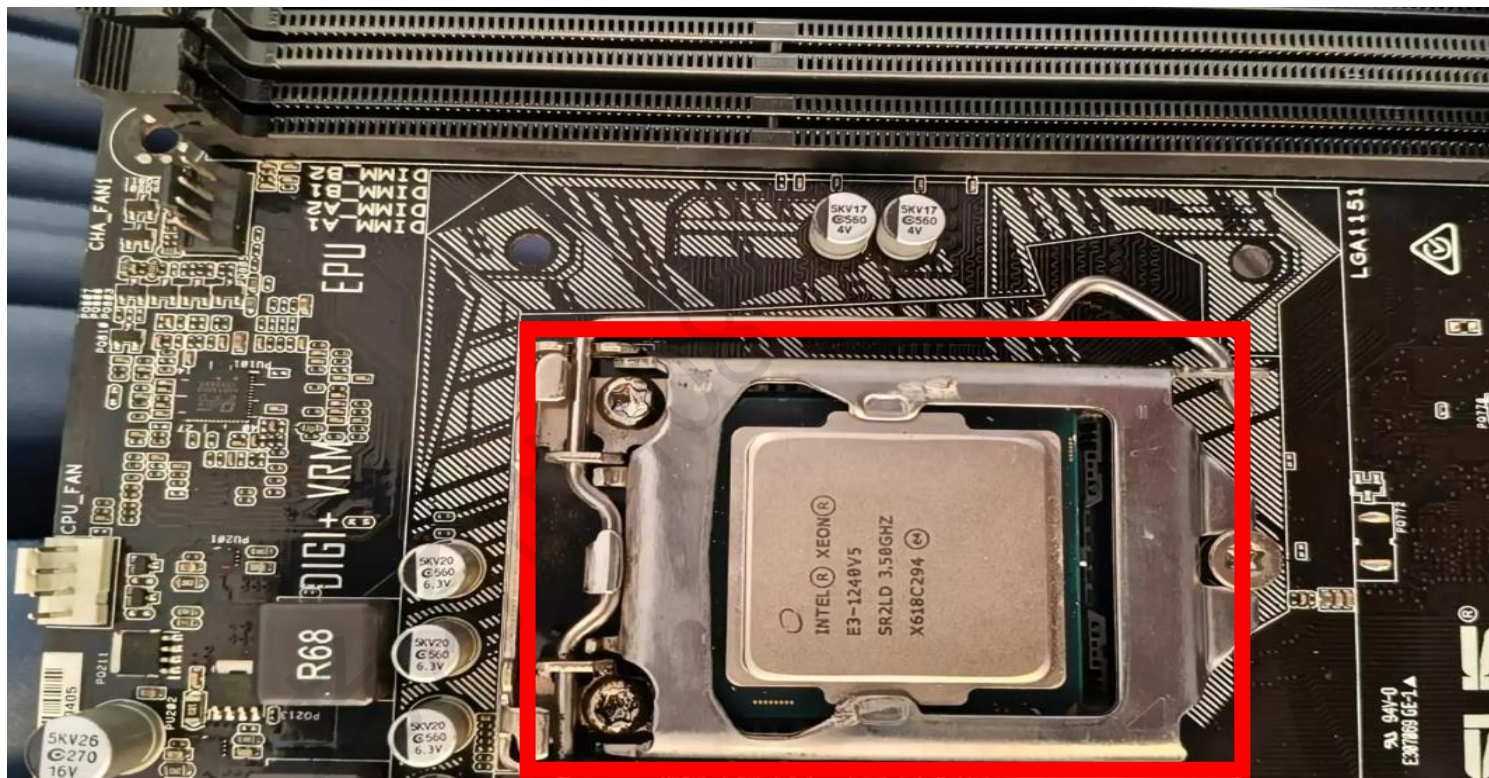
硬盘

CPU 通用寄存器组

0	
1	
2	
3	

执行指令

中央处理器 (Central Processing Unit, 简称 CPU)



计算机组成原理

讲师：老汤

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

第一章：计算机系统概述

第二章：总线系统

✓ 第三章：主存储器

第四章：数据的表示与运算

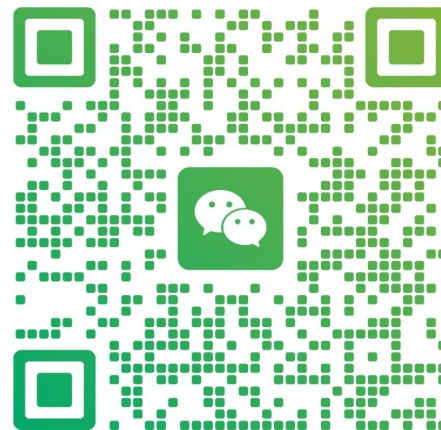
第五章：指令系统

第六章：中央处理器（CPU）

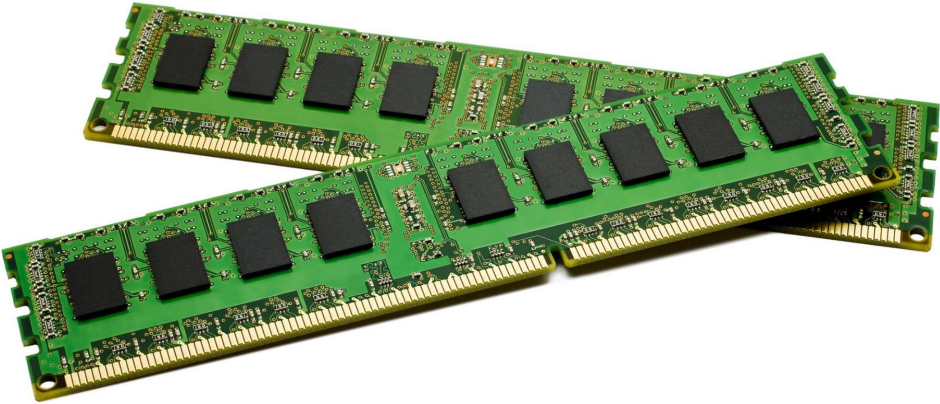
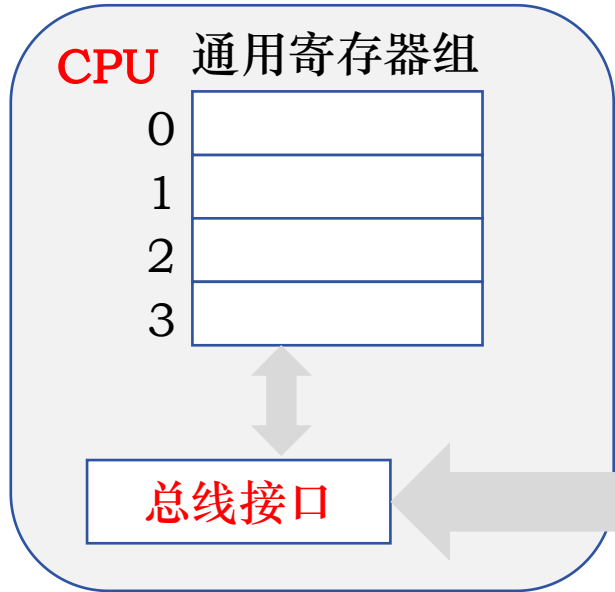
第七章：输入输出（I/O）系统



老汤
浙江 杭州

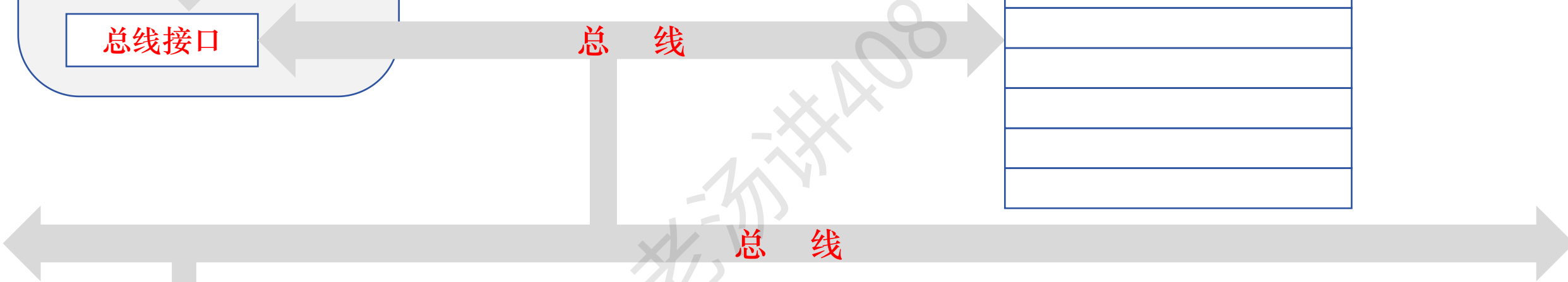
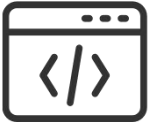


扫一扫上面的二维码图案，加我为朋友。



主存储器 (主存、内存)

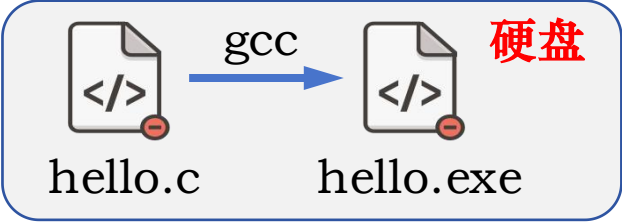
命令解释程序 (CMD)

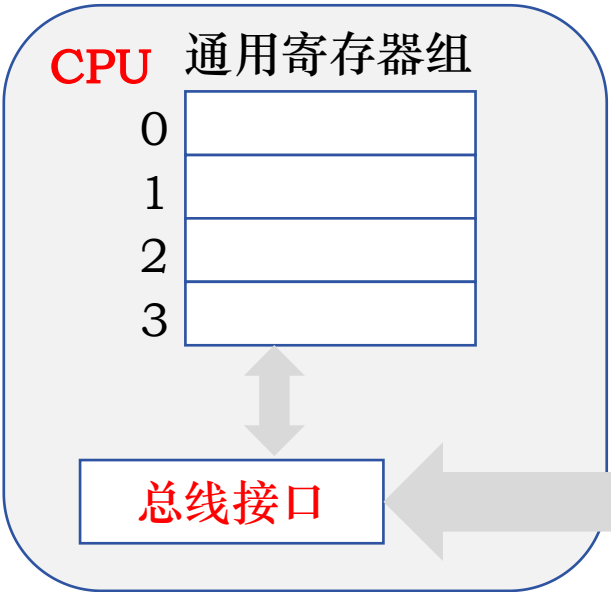


键盘控制器



hello.exe





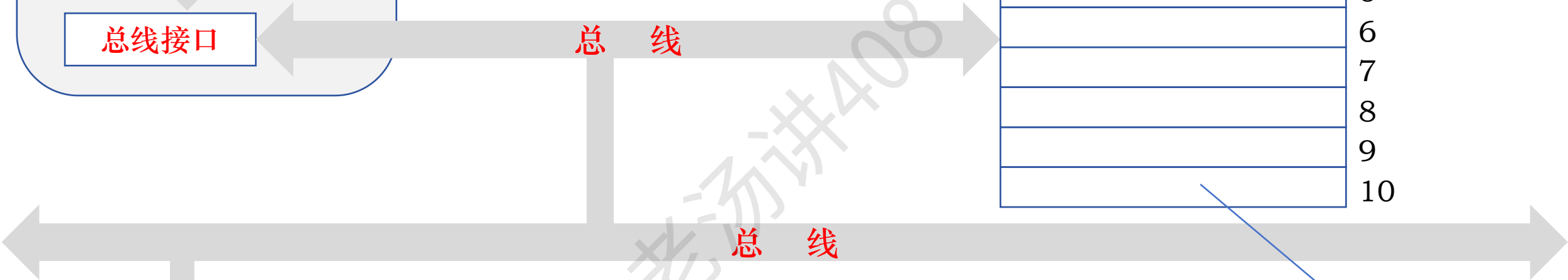
数 组

4	8	7	8	1	2	9
0	1	2	3	4	5	6

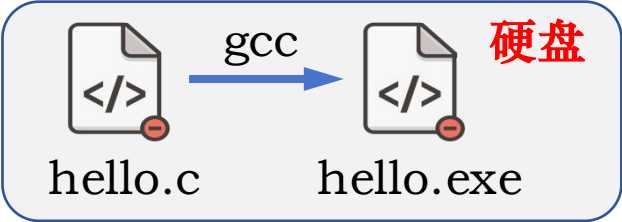
主存储器（主存、内存）

hello.exe	0
	1
	2
	3
	4
	5
	6
	7
	8
	9
	10

存储单元



键盘控制器



计算机组成原理

讲师：老汤

第一章：计算机系统概述

第二章：总线系统

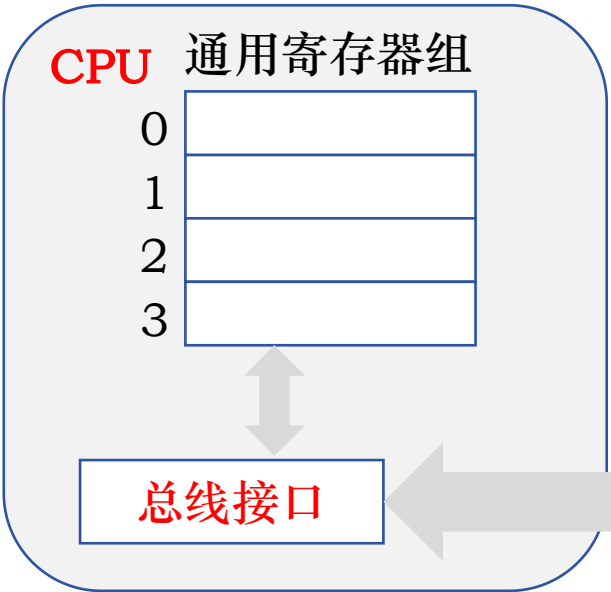
 第三章：主存储器

第四章：数据的表示与运算

第五章：指令系统

第六章：中央处理器 (CPU)

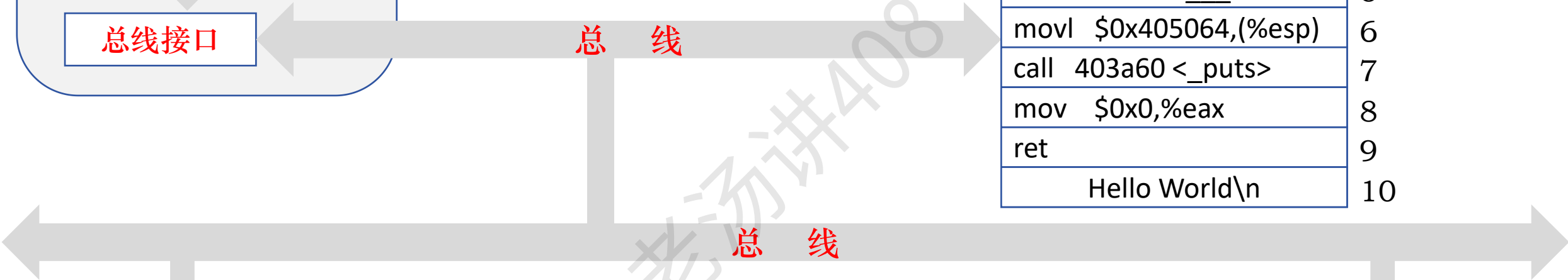
第七章：输入输出 (I/O) 系统



主存储器（主存、内存）

hello.exe	0
push %ebp	1
mov %esp,%ebp	2
and \$0xffffffff0,%esp	3
sub \$0x10,%esp	4
call 4019c0 <__main>	5
movl \$0x405064,(%esp)	6
call 403a60 <_puts>	7
mov \$0x0,%eax	8
ret	9
Hello World\n	10

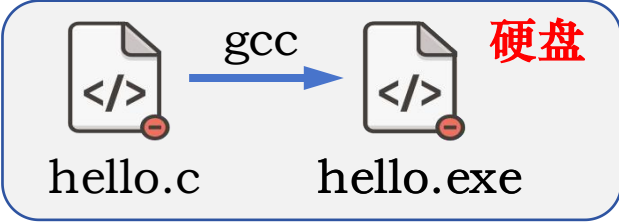
命令解释程序（CMD）

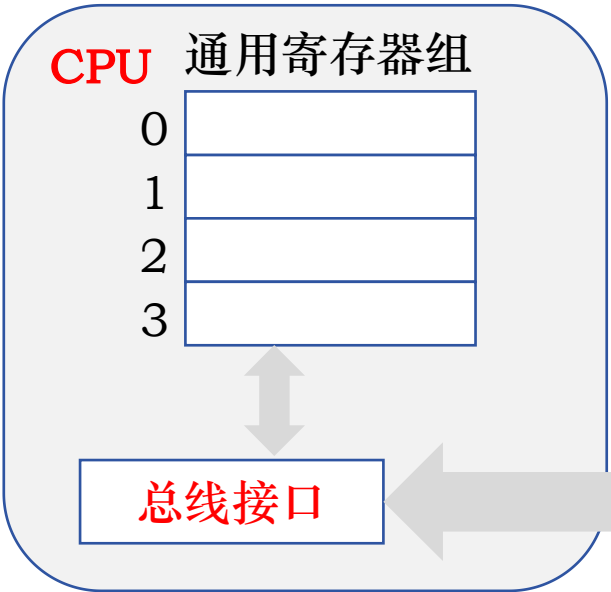


键盘控制器



硬盘控制器

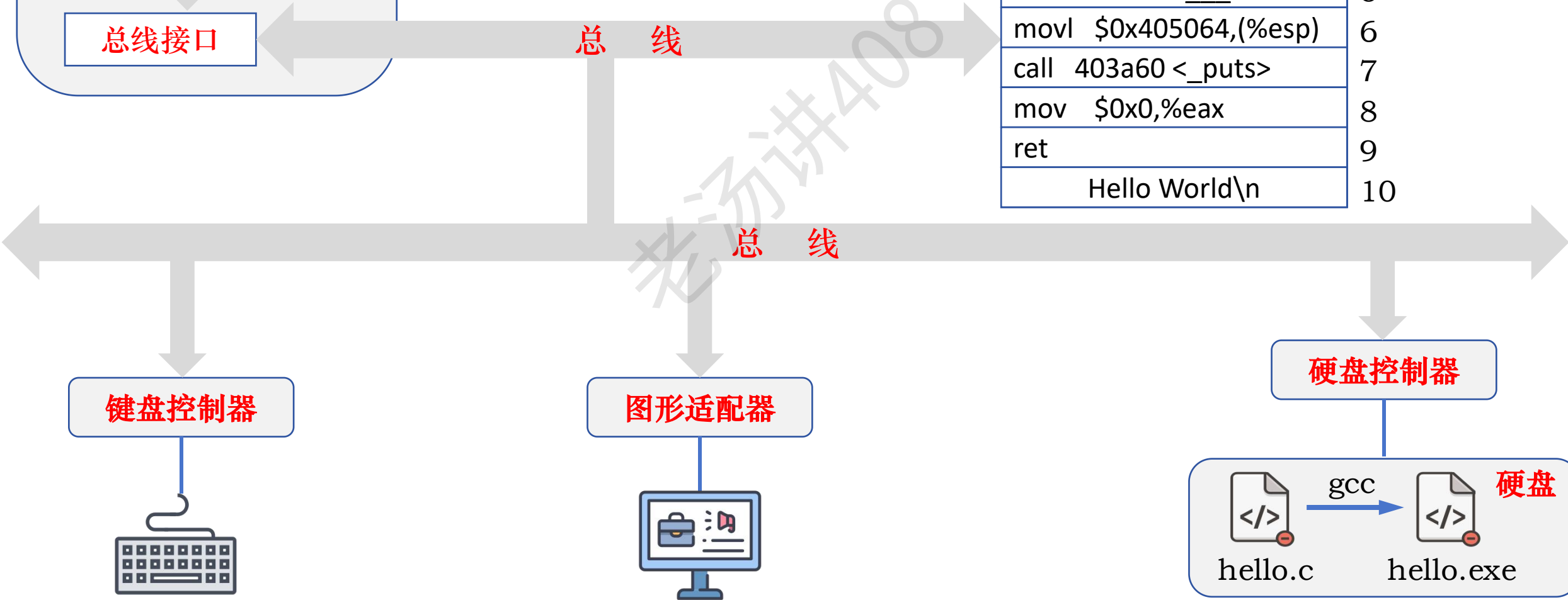


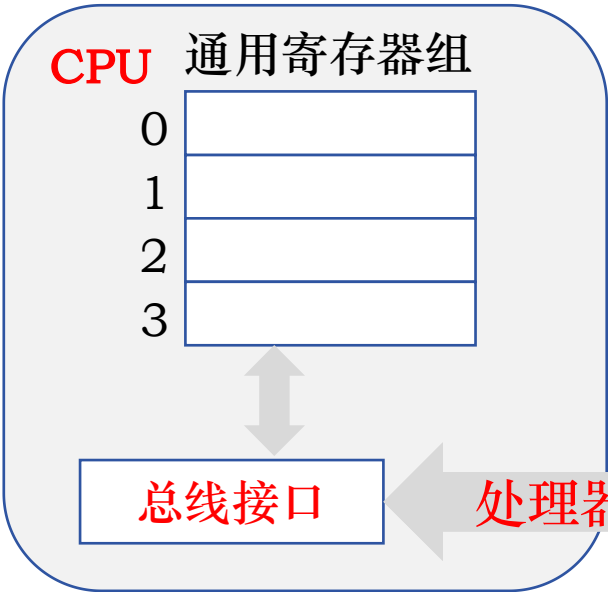


主存储器（主存、内存）

hello.exe	0
push %ebp	1
mov %esp,%ebp	2
and \$0xffffffff0,%esp	3
sub \$0x10,%esp	4
call 4019c0 <__main>	5
movl \$0x405064,(%esp)	6
call 403a60 <_puts>	7
mov \$0x0,%eax	8
ret	9
Hello World\n	10

命令解释程序 (CMD)





处理器总线



内存总线

主存储器 (主存、内存)

hello.exe	0
push %ebp	1
mov %esp,%ebp	2
and \$0xffffffff0,%esp	3
sub \$0x10,%esp	4
call 4019c0 <__main>	5
movl \$0x405064,(%esp)	6
call 403a60 <_puts>	7
mov \$0x0,%eax	8
ret	9
Hello World\n	10

命令解释程序 (CMD)

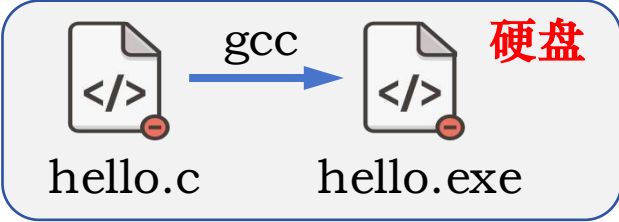


输入 (Input) 设备

输出 (Onput) 设备



I/O 设备



计算机组成原理

讲师：老汤

第一章：计算机系统概述



第二章：总线系统

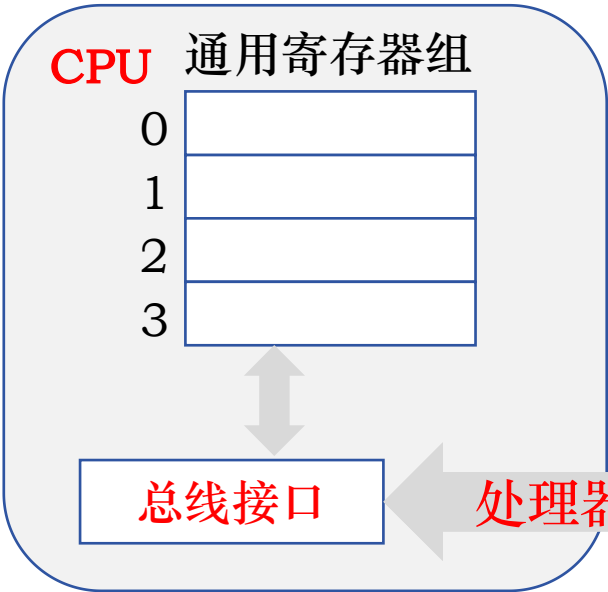
第三章：主存储器

第四章：数据的表示与运算

第五章：指令系统

第六章：中央处理器 (CPU)

第七章：输入输出 (I/O) 系统



处理器总线



内存总线

主存储器 (主存、内存)

hello.exe	0
push %ebp	1
mov %esp,%ebp	2
and \$0xffffffff0,%esp	3
sub \$0x10,%esp	4
call 4019c0 <__main>	5
movl \$0x405064,(%esp)	6
call 403a60 <_puts>	7
mov \$0x0,%eax	8
ret	9
Hello World\n	10

命令解释程序 (CMD)

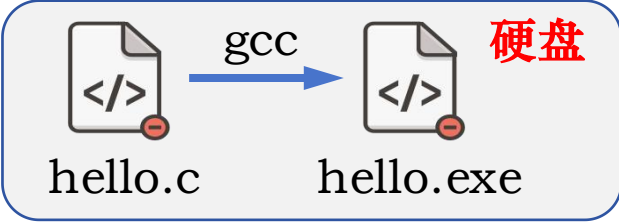


输入 (Input) 设备

输出 (Onput) 设备



I/O 设备



计算机组成原理

讲师：老汤

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

第一章：计算机系统概述

第二章：总线系统

第三章：主存储器

第四章：数据的表示与运算

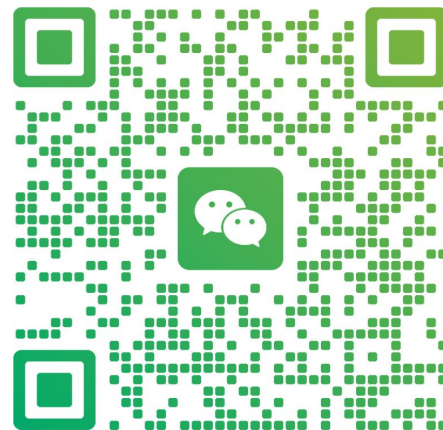
第五章：指令系统

第六章：中央处理器 (CPU)

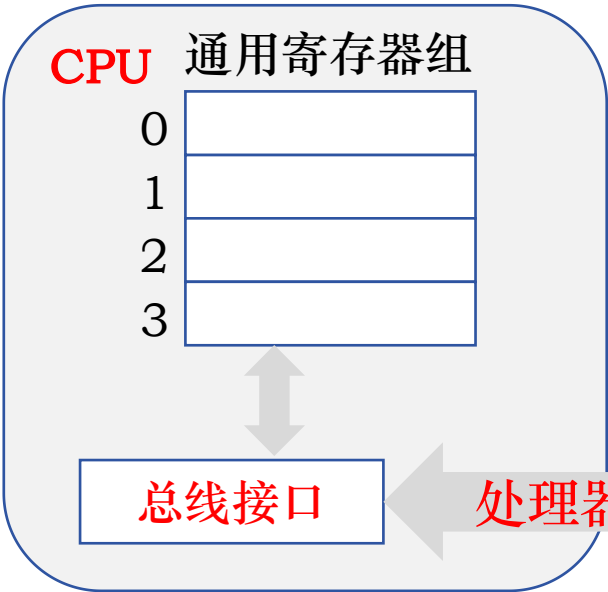
✓第七章：输入输出 (I/O) 系统



老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。



处理器总线



内存总线

主存储器 (主存、内存)

hello.exe	0
push %ebp	1
mov %esp,%ebp	2
and \$0xffffffff0,%esp	3
sub \$0x10,%esp	4
call 4019c0 <__main>	5
movl \$0x405064,(%esp)	6
call 403a60 <_puts>	7
mov \$0x0,%eax	8
ret	9
Hello World\n	10

命令解释程序 (CMD)

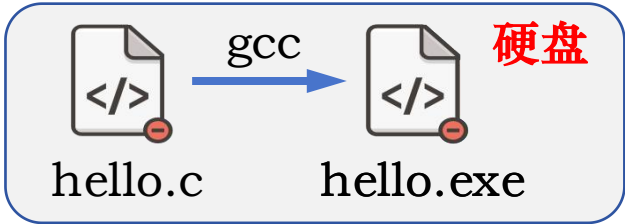


I/O 方式
(中断方式、DMA 方式)

I/O 设备



hello



二进制、十进制和十六进制之间转换

第一章：计算机系统概述

第二章：总线系统

第三章：主存储器



第四章：数据的表示与运算

第五章：指令系统

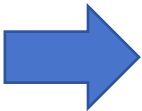
第六章：中央处理器 (CPU)

第七章：输入输出 (I/O) 系统

二进制、十进制和十六进制之间转换

主存储器（主存、内存）

hello
push %ebp
mov %esp,%ebp
and \$0xffffffff0,%esp
sub \$0x10,%esp
call 4019c0 <__main>
movl \$0x405064,(%esp)
call 403a60 <_puts>
mov \$0x0,%eax
ret
Hello World\n



主存储器（主存、内存）

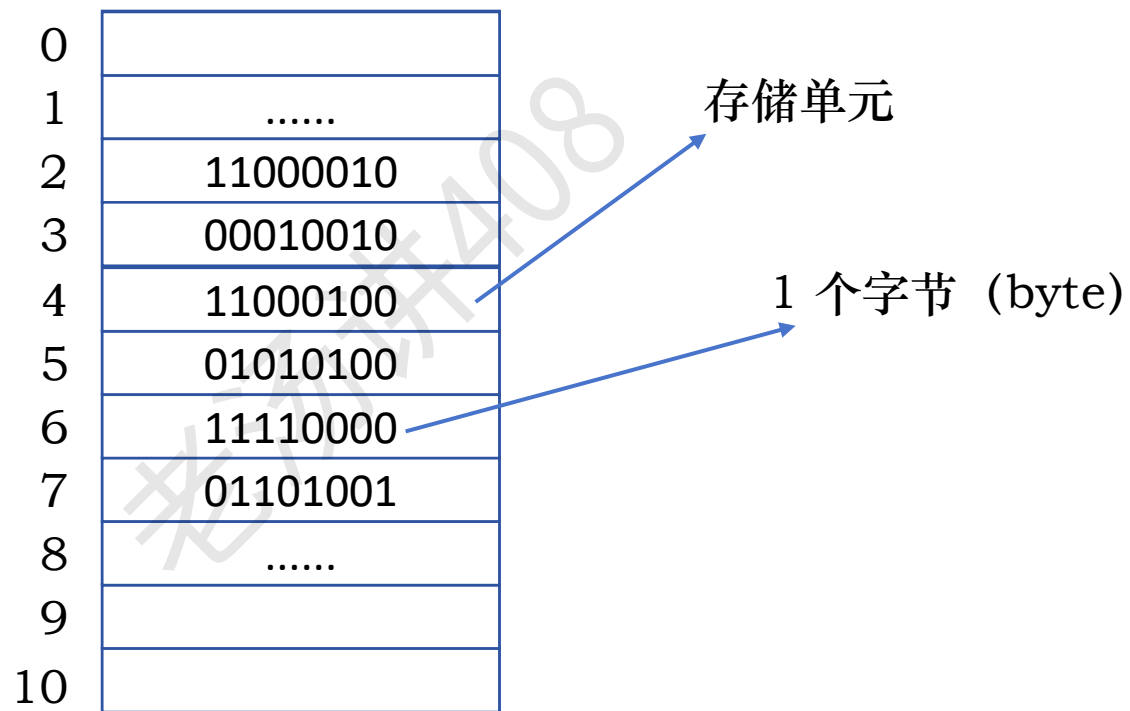
01010101
10001001 11100101
10000011 11100100 11110000
10000011 11101100 00010000
11101000 01010010 00000101 00000000 00000000
11000111 00000100 00100100 01100100 01010000 01000000 00000000
11101000 11100110 00100101 00000000 00000000
10111000 00000000 00000000 00000000 00000000
11000011

二进制： 0 1 位 (bit)

当把多个位组合起来，再加上某种解释，即赋予不同位组合以不同的含义

二进制、十进制和十六进制之间转换

主存储器（主存、内存）



字节数组

二进制、十进制和十六进制之间转换

主存储器（主存、内存）

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	

0110 1000 0110 1001

`char str[2] = { 'h' , 'i' }`

`short a = 26729`

ASCII (American Standard Code for Information Interchange): 美国信息交换标准代码

0110 0001	0141	97	0x61	a	小写字母a
0110 0010	0142	98	0x62	b	小写字母b
0110 0011	0143	99	0x63	c	小写字母c
0110 0100	0144	100	0x64	d	小写字母d
0110 0101	0145	101	0x65	e	小写字母e
0110 0110	0146	102	0x66	f	小写字母f
0110 0111	0147	103	0x67	g	小写字母g
0110 1000	0150	104	0x68	h	小写字母h
0110 1001	0151	105	0x69	i	小写字母i
0110 1010	0152	106	0x6A	j	小写字母j
0110 1011	0153	107	0x6B	k	小写字母k
0110 1100	0154	108	0x6C	l	小写字母l
0110 1101	0155	109	0x6D	m	小写字母m

二进制、十进制和十六进制之间转换

主存储器（主存、内存）

0	
1	
2	
3	
4	0110 1000
5	0110 1001
6	
7	
8	
9	
10	

‘h’
‘i’

0110 1000 0110 1001

`char str[2] = { ‘h’ , ‘i’ }`

二进制、十进制和十六进制之间转换

高位 低位
←-----→
0110 1000 0110 1001
 ↑

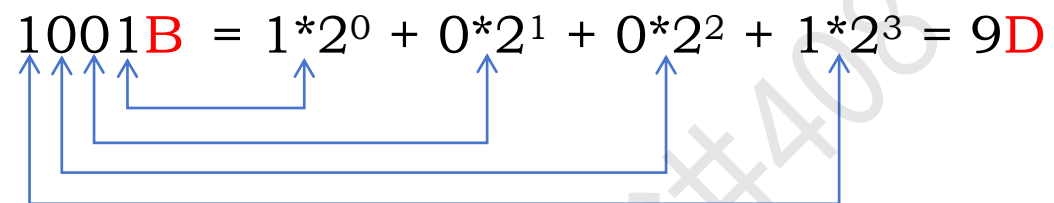
主存储器（主存、内存）

0	
1	
2	
3	
4	0110 1000
5	0110 1001
6	
7	
8	
9	
10	

short a = 26729

按权展开： $1*2^0 + 0*2^1 + 0*2^2 + 1*2^3 + 0*2^4 + 0*2^5 + \dots + 1*2^{14} + 0*2^{15} = 26729$

二进制转十进制

$$1001_{\text{B}} = 1 \cdot 2^0 + 0 \cdot 2^1 + 0 \cdot 2^2 + 1 \cdot 2^3 = 9_{\text{D}}$$


$$10101_{\text{B}} = 1 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 + 0 \cdot 2^3 + 1 \cdot 2^4 = 21$$

十进制转二进制

方法：除基取余，上右下左

基数：2

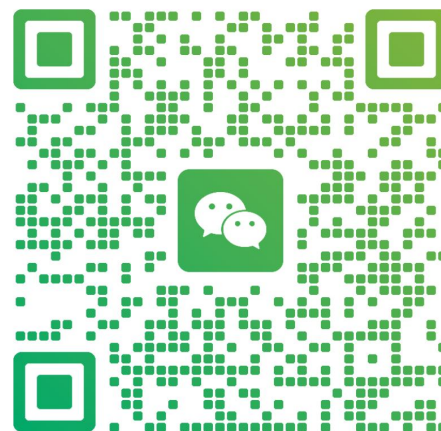
		余数	
2	135	1	↓ 低位 ↑ 高位
2	67	1	
2	33	1	
2	16	0	
2	8	0	
2	4	0	
2	2	0	
2	1	1	
	0		

$$135 = 10000111\text{B}$$

十进制、二进制之间转换很繁琐

加老汤微信，帮你 408 冲 120+

2 | 245685179
.....



扫一扫上面的二维码图案，加我为朋友。

245685179 = 1110 1010 0100 1101 1011 1011 1011**B**

二进制表示法太冗长了!!!!

十六进制

二进制：0、1

十进制：0、1、2、3、4、5、6、7、8、9

十六进制：0、1、2、3、4、5、6、7、8、9、A、B、C、D、E、F

$1010\text{B} = 0*2^0 + 1*2^1 + 0*2^2 + 1*2^3 = 10$

$1110\text{B} = 0*2^0 + 1*2^1 + 1*2^2 + 1*2^3 = 14 = \text{EH}$

十六进制	十进制	二进制
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

十六进制

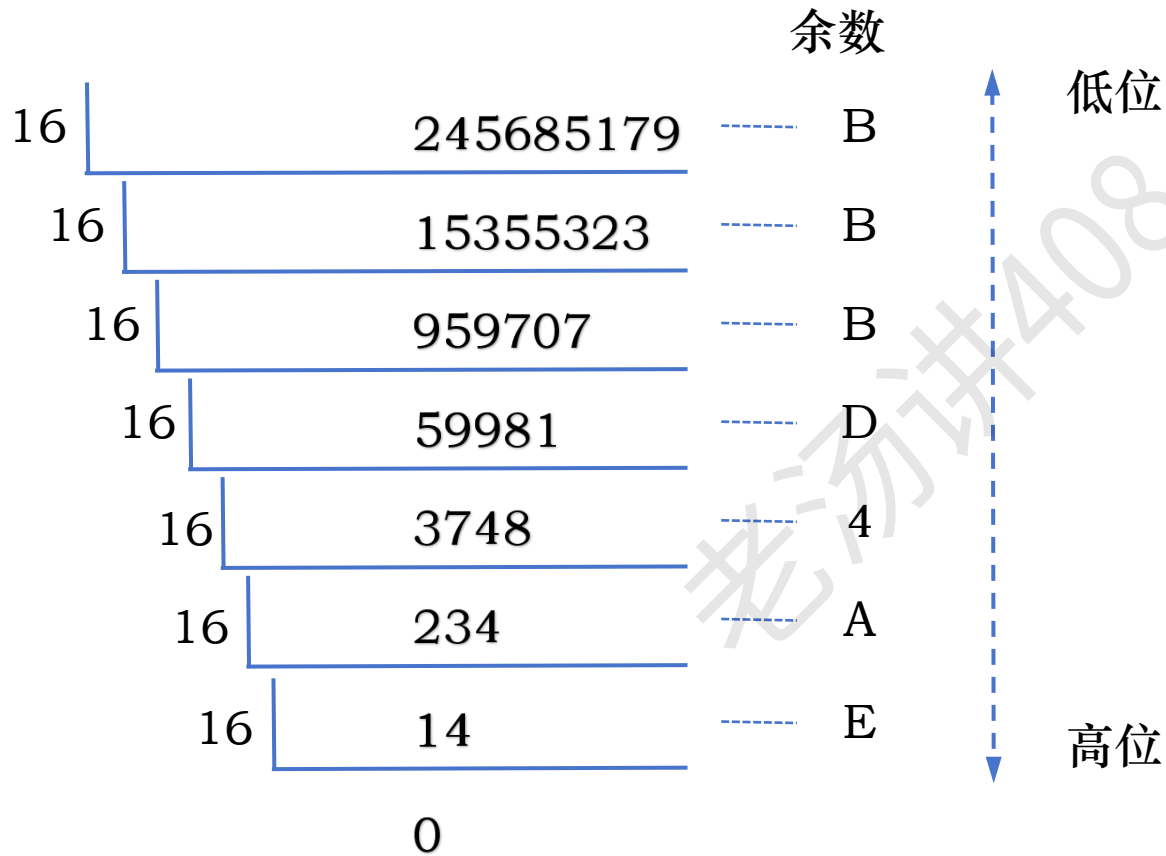
按权展开，权值是 16

$13D_{\text{H}} = D \times 16^0 + 3 \times 16^1 + 1 \times 16^2 = 13 \times 16^0 + 3 \times 16^1 + 1 \times 16^2 = 317$

十六进制	十进制	二进制
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

十六进制

方法：除基取余，上右下左

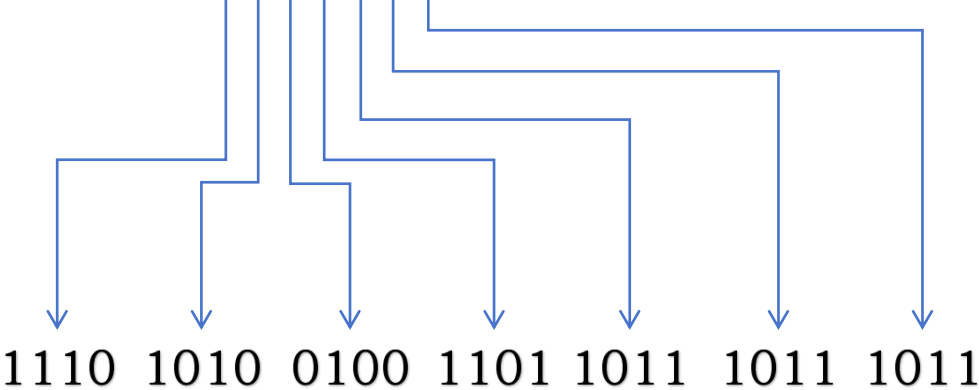


245685179 = EA4DBBBH

十六进制	十进制	二进制
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

十六进制

245685179 = EA4DBBBH

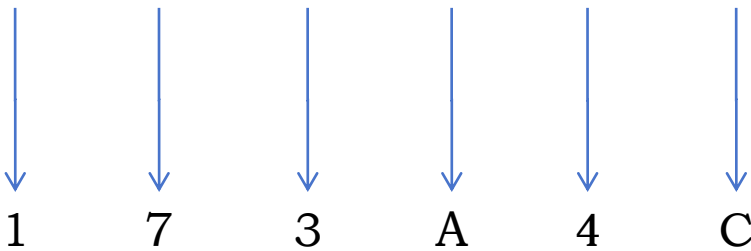


245685179 = EA4DBBBH = 1110 1010 0100 1101 1011 1011 1011B

十六进制	十进制	二进制
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

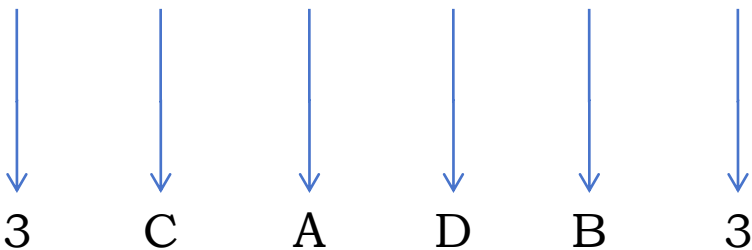
十六进制

0001 0111 0011 1010 0100 1100



0001 0111 0011 1010 0100 1100B = 173A4CH

0011 1100 1010 1101 1011 0011



11 1100 1010 1101 1011 0011B = 3CADB3H = 3cadb3H

十六进制	十进制	二进制
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

二进制、十进制、十六进制

考研

第一章：计算机系统概述

第二章：总线系统

第三章：主存储器



第四章：数据的表示与运算

第五章：指令系统

第六章：中央处理器 (CPU)

第七章：输入输出 (I/O) 系统

十六进制	十进制	二进制
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

指令的设计和執行过程

第一章：计算机系统概述

第二章：总线系统

第三章：主存储器

第四章：数据的表示与运算



第五章：指令系统

第六章：中央处理器 (CPU)

第七章：输入输出 (I/O) 系统

主要还是为了教学，本节课达到了了解计算机指令的执行过程的目的就可以了

指令的设计和执行过程

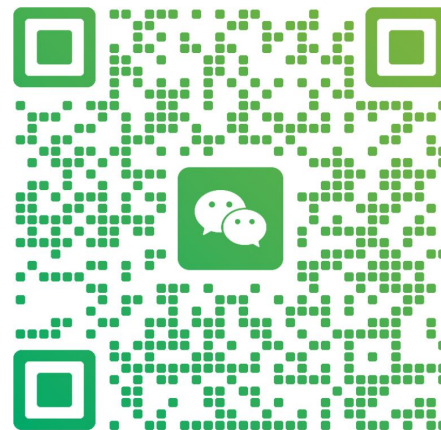
加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

表达式: $ax + b$



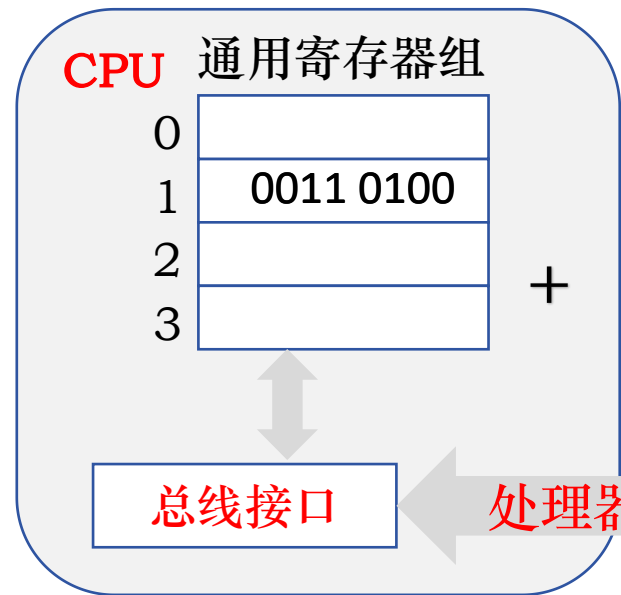
老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

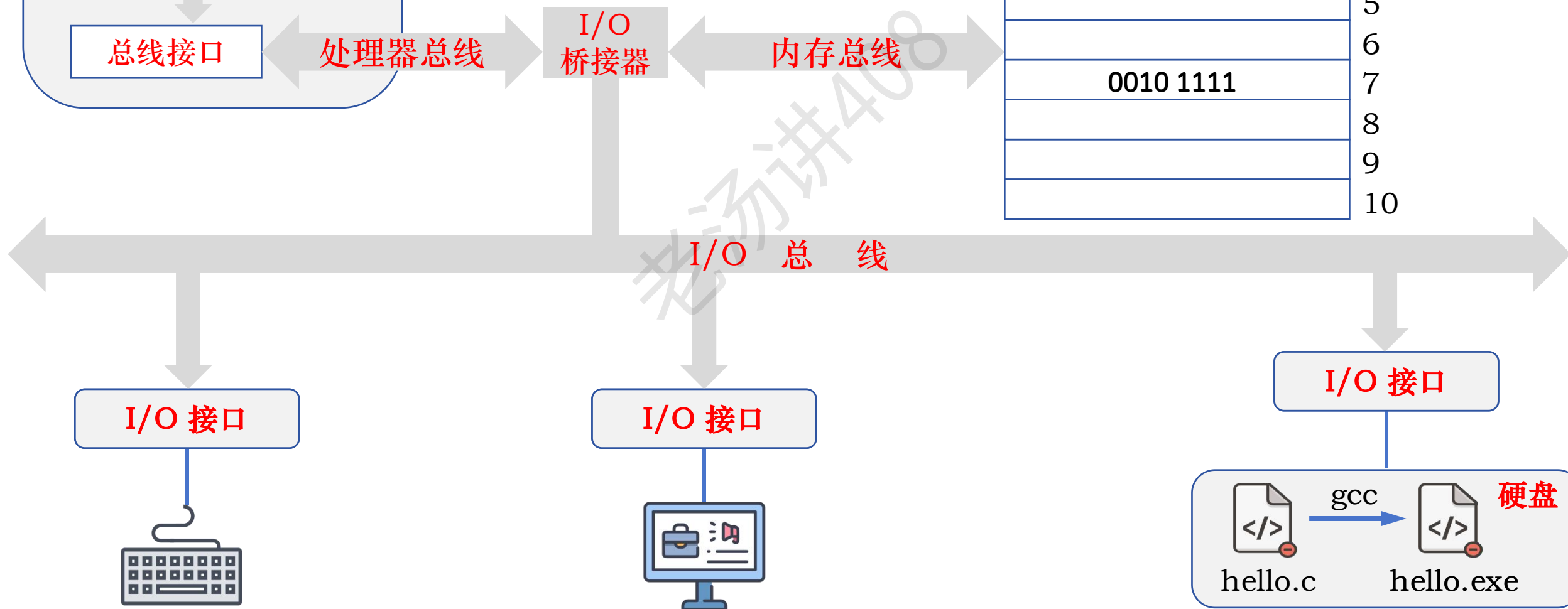
指令的设计和执行过程



- ① 取数操作
- ② 运算操作：加、减、乘、除
- ③ 存数操作
- ④ 打印操作

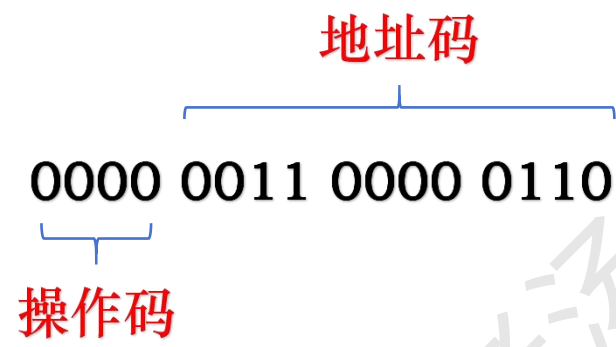
主存储器（主存、内存）

	0
	1
0000 1100	2
0010 1000	3
	4
	5
	6
0010 1111	7
	8
	9
	10



指令的设计和執行过程

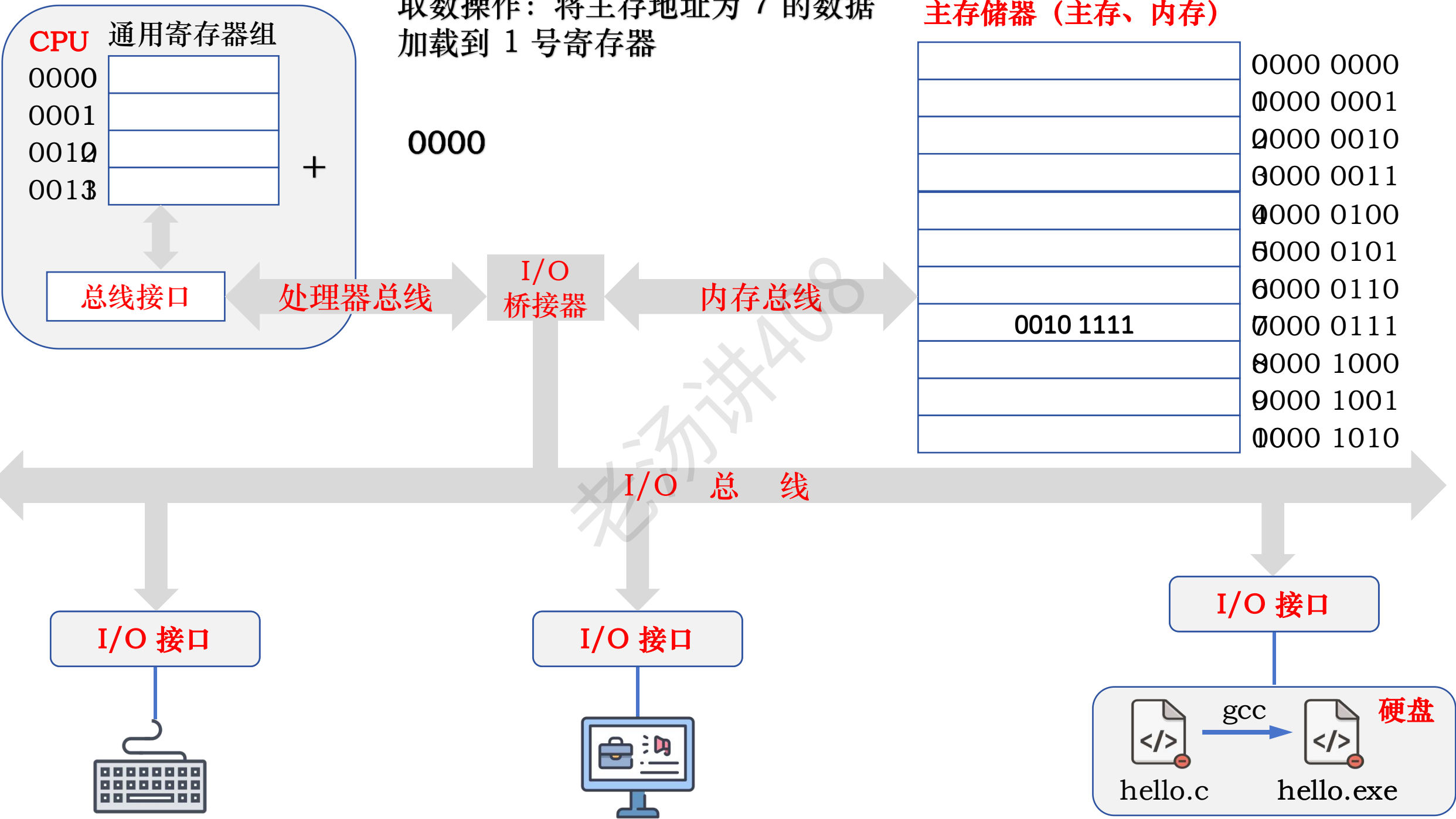
一条指令：



操作码	操作类型
0000	取数操作
0001	存数操作
0010	加法操作
0011	乘法操作
0100	打印操作
0101	停机操作
.....	

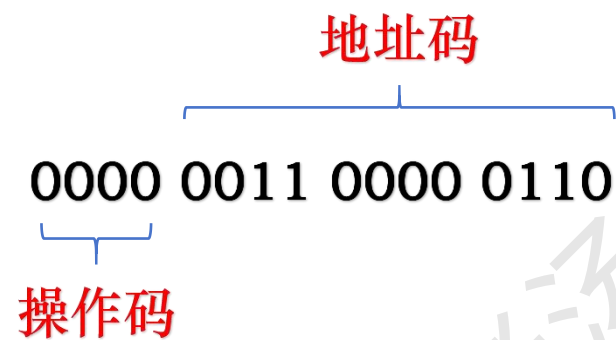
取数操作：将主存地址为 7 的数据
加载到 1 号寄存器

主存储器（主存、内存）



指令的设计和執行过程

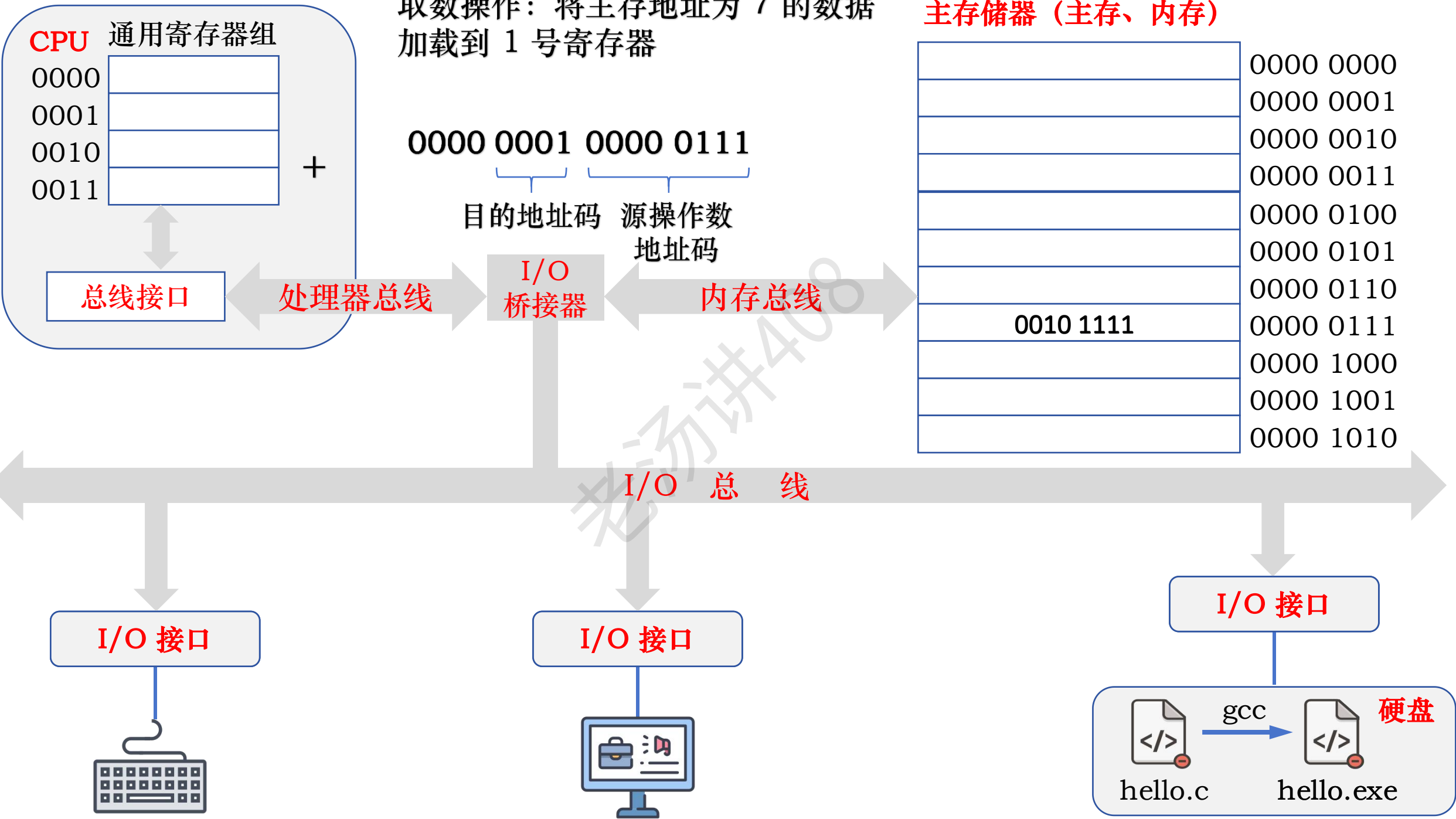
一条指令：



操作码	操作类型
0000	取数操作
0001	存数操作
0010	加法操作
0011	乘法操作
0100	打印操作
0101	停机操作
.....	

取数操作：将主存地址为 7 的数据
加载到 1 号寄存器

主存储器（主存、内存）



指令的设计和执行过程

表达式: $ax + b$

主存储器 (主存、内存)

	0000 0000	0
	0000 0010	2
	0000 0100	4
	0000 0110	6
	0000 1000	8
0010 1111 0110 1001	0000 1010	10
	0000 1100	12
	0000 1110	14
	0001 0000	16
	0001 0010	18
	0001 0100	20

指令的设计和执行过程

表达式: $ax + b$

$a = 5 = 0000\ 0101\text{B}$ $x = 6 = 0000\ 0110\text{B}$ $b = 12 = 0000\ 1100\text{B}$

主存储器（主存、内存）

	0000 0000
	0000 0010
	0000 0100
	0000 0110
	0000 1000
	0000 1010
	0000 1100
	0000 1110
	0001 0000
0000 0101 0000 0110	0001 0010
0000 1100	0001 0100

变量	值	内存地址
a	5	0001 0010
x	6	0001 0011
b	12	0001 0100

表达式: $ax + b$

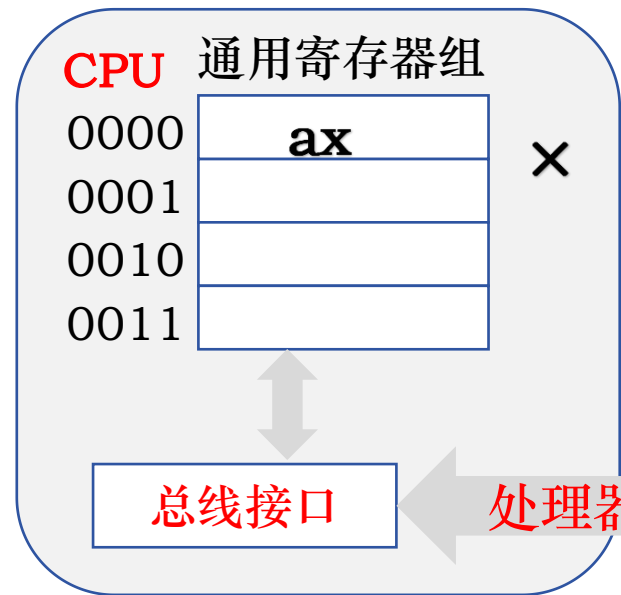
① 取数: 将 a 加载到 0 号寄存器 (R0)

② 取数: 将 x 加载到 1 号寄存器 (R1)

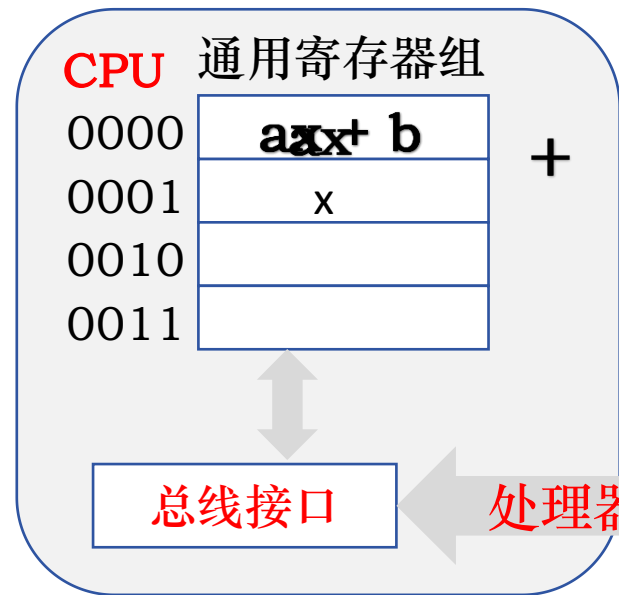
③ 乘法: 将 0 号寄存器的值和 1 号寄存器的值做乘法, 结果送到 0 号寄存器

主存储器 (主存、内存)

	0000 0000
	0000 0010
	0000 0100
	0000 0110
	0000 1000
	0000 1010
	0000 1100
	0000 1110
	0001 0000
a	x
	0001 0010
b	0001 0100



表达式: $ax + b$



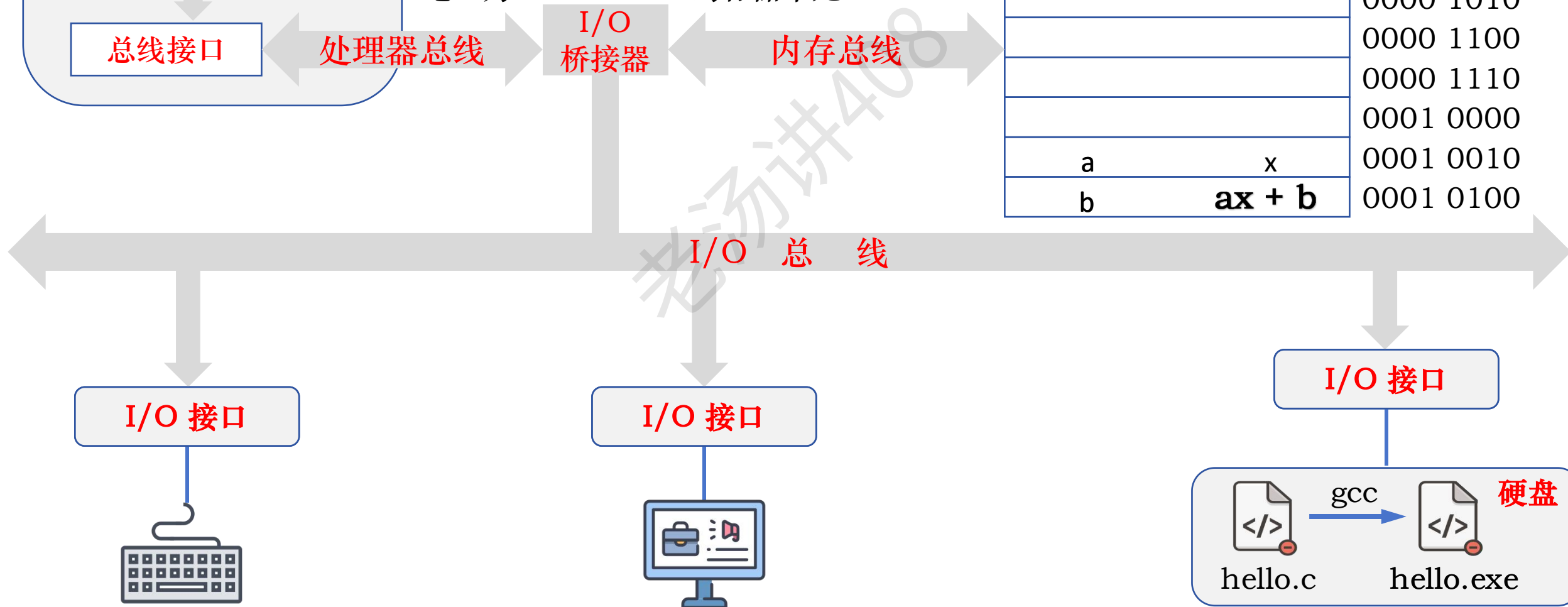
④ 取数: 将 b 加载到 1 号寄存器

⑤ 加法: 将 0 号寄存器的值和 1 号寄存器的值做加法, 结果送到 0 号寄存器

⑥ 存数: 将 0 号寄存器的值存放到内存地址为 0001 0101 的存储单元

主存储器 (主存、内存)

	0000 0000
	0000 0010
	0000 0100
	0000 0110
	0000 1000
	0000 1010
	0000 1100
	0000 1110
	0001 0000
a	x
b	$ax + b$



指令的设计和执行情况

表达式: $ax + b$

① 取数: 将 a 加载到 0 号寄存器

指令: 0000 0000 0001 0010

② 取数: 将 x 加载到 1 号寄存器

指令: 0000 0001 0001 0011

③ 乘法: 将 0 号寄存器的值和 1 号寄存器的值做乘法, 结果送到 0 号寄存器

指令: 0011 0000 0000 0001

④ 取数: 将 b 加载到 1 号寄存器

指令: 0000 0001 0001 0100

⑤ 加法: 将 R0 的值和 R1 的值做加法, 结果送到 R0

指令: 0010 0000 0000 0001

⑥ 存数: 将 0 号寄存器的值存放到内存地址为 0001 0101 的存储单元

指令: 0001 0001 0101 0000

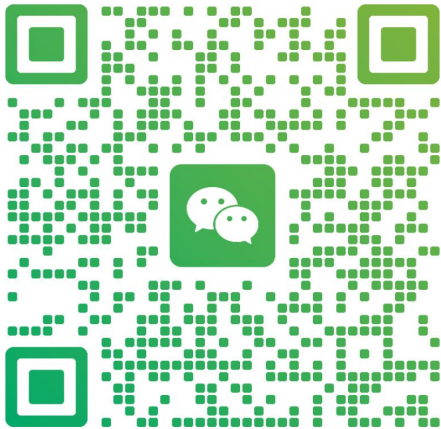
变量	值	内存地址
a	5	0001 0010
x	6	0001 0011
b	12	0001 0100

通用寄存器	
0000	
0001	
0010	
0011	

操作码	操作类型
0000	取数操作
0001	存数操作
0010	加法操作
0011	乘法操作
0100	打印操作
0101	停机操作
.....	

指令的设计和執行过程

加老汤微信，帮你 408 冲 120+

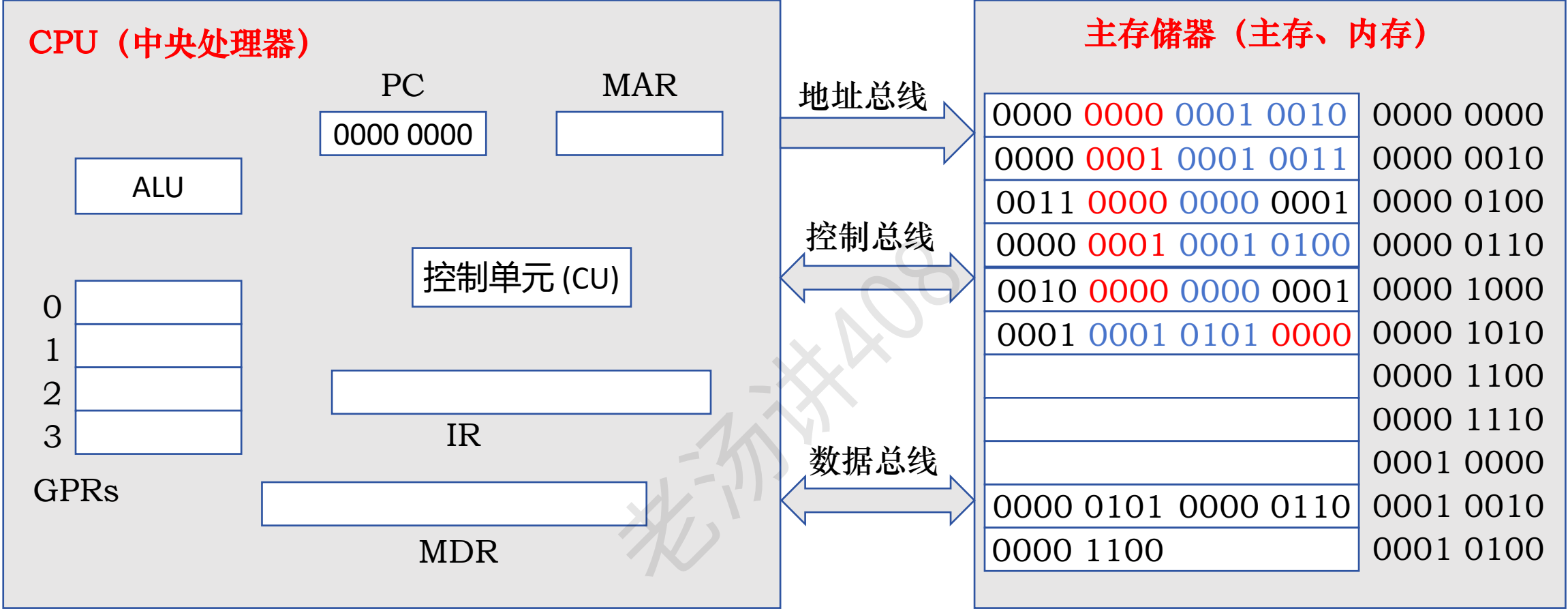


扫一扫上面的二维码图案，加我为朋友。

主存储器（主存、内存）

指令 ①:	0000 0000 0001 0010	0000 0000
指令 ②:	0000 0001 0001 0011	0000 0010
指令 ③:	0011 0000 0000 0001	0000 0100
指令 ④:	0000 0001 0001 0100	0000 0110
指令 ⑤:	0010 0000 0000 0001	0000 1000
指令 ⑥:	0001 0001 0101 0000	0000 1010
		0000 1100
		0000 1110
		0001 0000
	0000 0101 0000 0110	0001 0010
	0000 1100	0001 0100

指令的~~设计~~和执行过程



通用寄存器组 (General Purpose Register set, GPRs)

程序计数器 (Program Counter, PC)

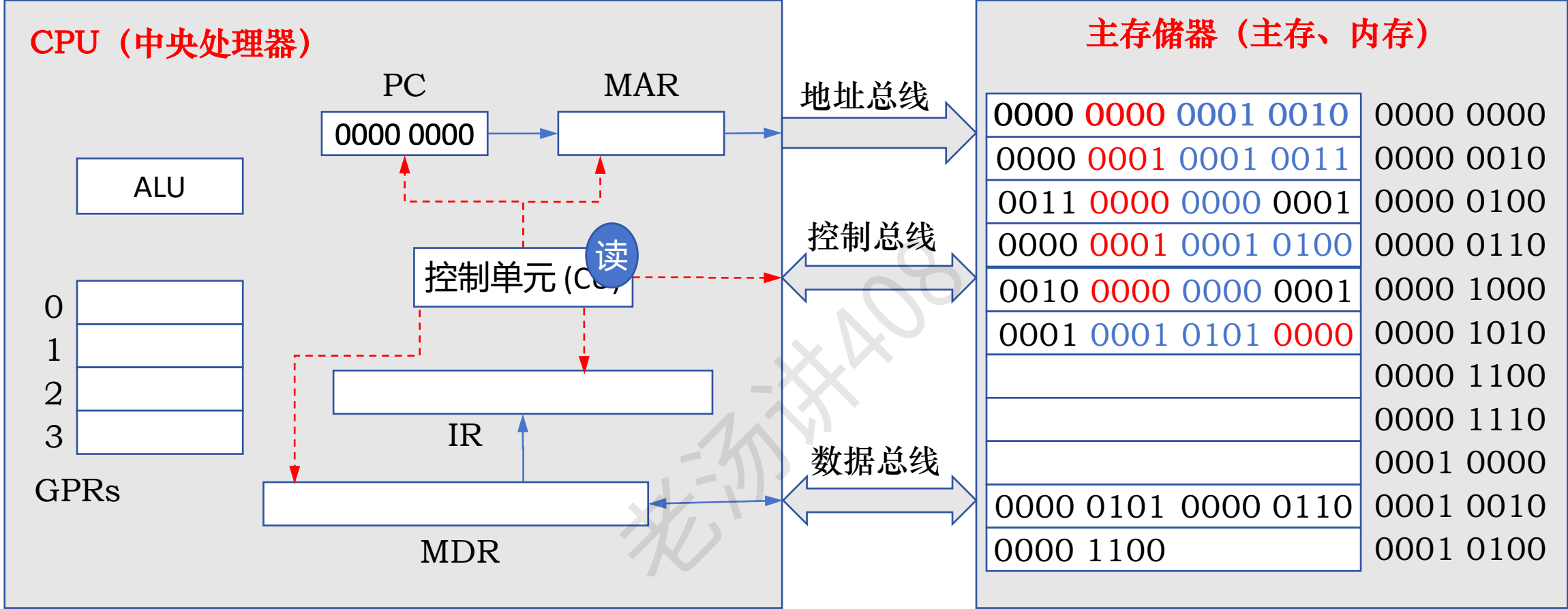
指令寄存器 (Instruction Register, IR)

内存地址寄存器 (Memory Address Register, MAR) 内存数据寄存器 (Memory Data Register, MDR)

算术逻辑单元 (Arithmetic and Logic Unit, ALU)

控制单元 (Control Unit, CU)

指令的设计和执行过程



通用寄存器组 (General Purpose Register set, GPRs)

程序计数器 (Program Counter, PC)

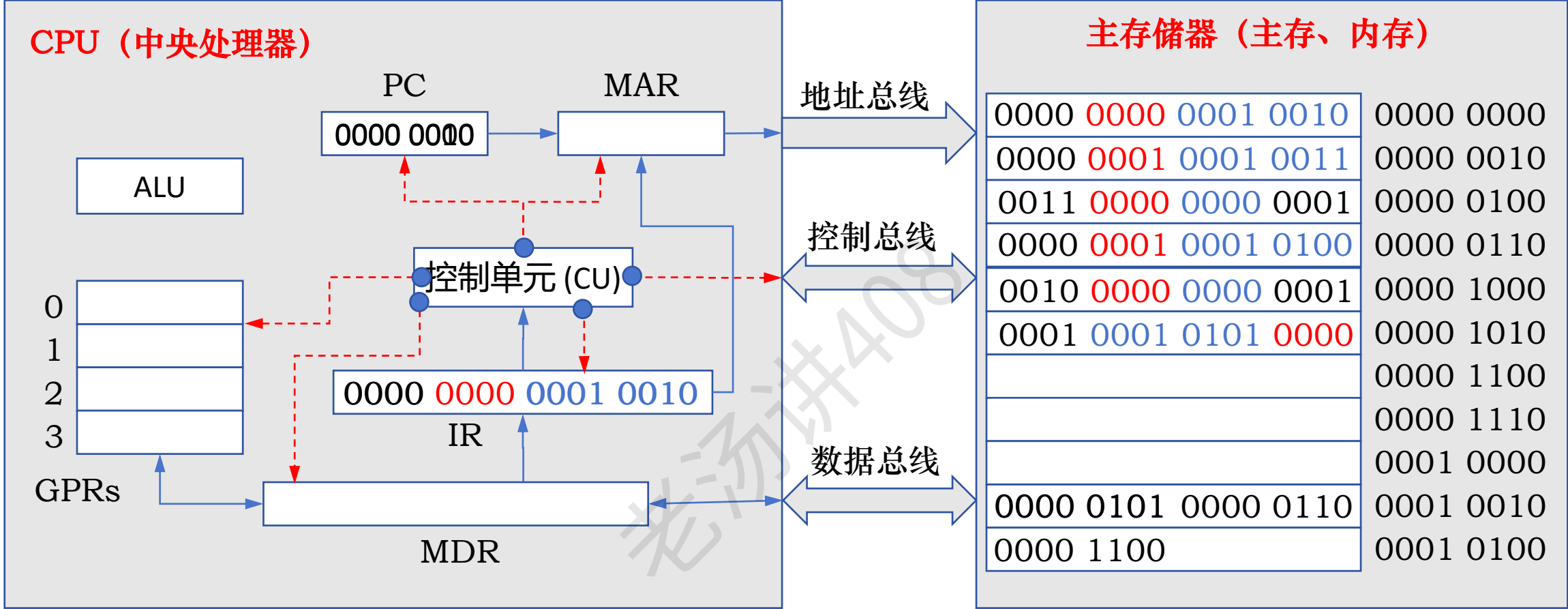
指令寄存器 (Instruction Register, IR)

内存地址寄存器 (Memory Address Register, MAR) 内存数据寄存器 (Memory Data Register, MDR)

算术逻辑单元 (Arithmetic and Logic Unit, ALU)

控制单元 (Control Unit, CU)

指令的设计和執行过程



通用寄存器组 (General Purpose Register set, GPRs)

程序计数器 (Program Counter, PC)

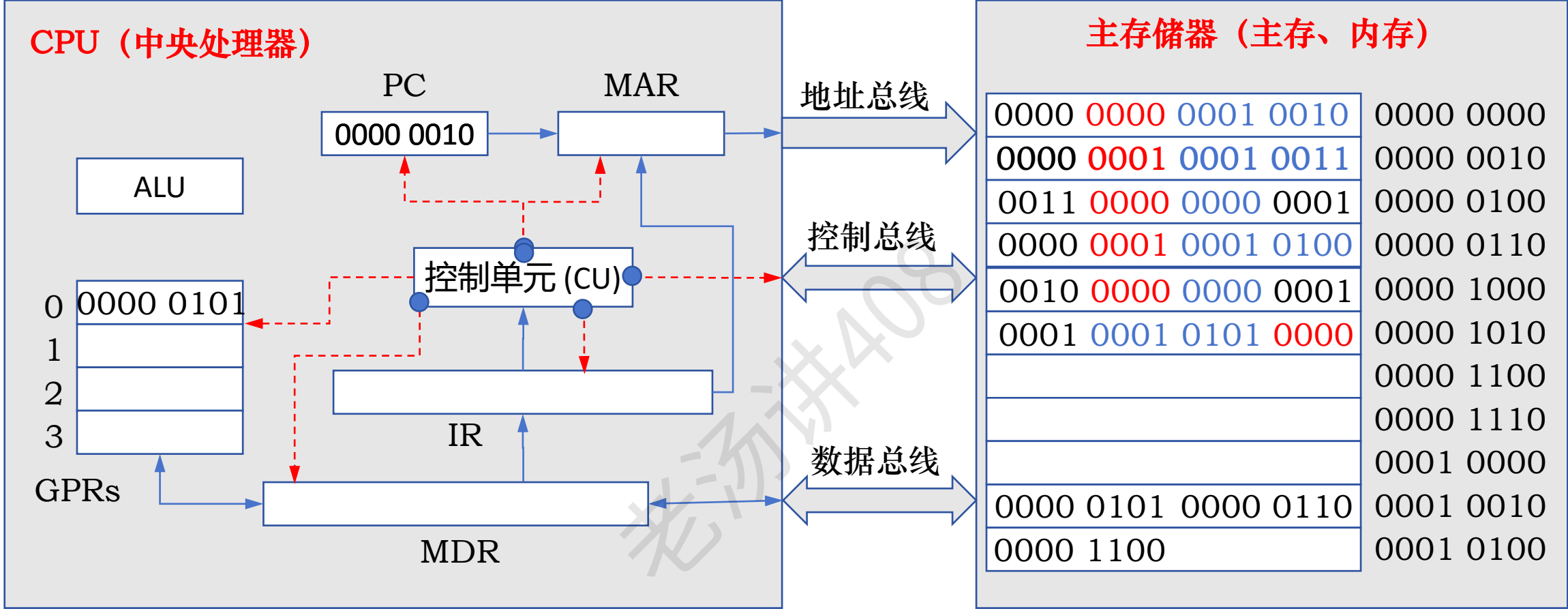
指令寄存器 (Instruction Register, IR)

内存地址寄存器 (Memory Address Register, MAR) 内存数据寄存器 (Memory Data Register, MDR)

算术逻辑单元 (Arithmetic and Logic Unit, ALU)

控制单元 (Control Unit, CU)

指令的~~设计~~和执行过程



通用寄存器组 (General Purpose Register set, GPRs)

程序计数器 (Program Counter, PC)

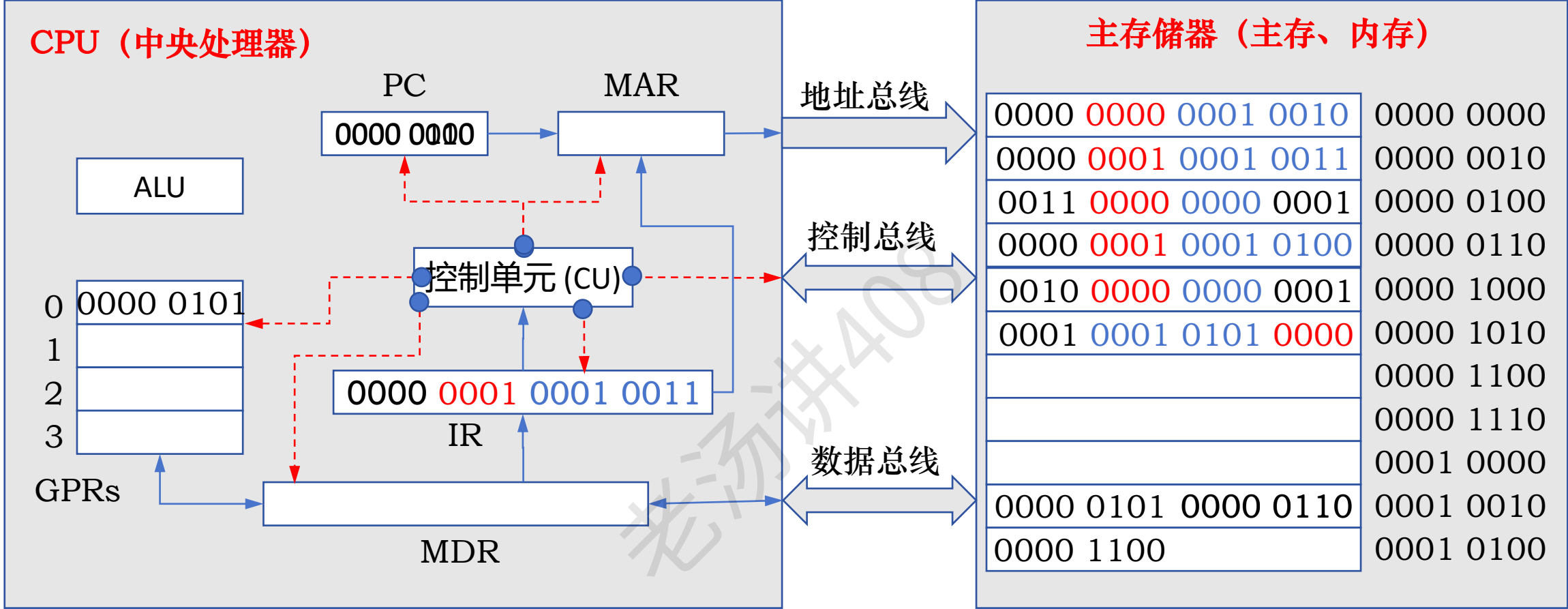
指令寄存器 (Instruction Register, IR)

内存地址寄存器 (Memory Address Register, MAR) 内存数据寄存器 (Memory Data Register, MDR)

算术逻辑单元 (Arithmetic and Logic Unit, ALU)

控制单元 (Control Unit, CU)

指令的设计和执行过程



通用寄存器组 (General Purpose Register set, GPRs)

程序计数器 (Program Counter, PC)

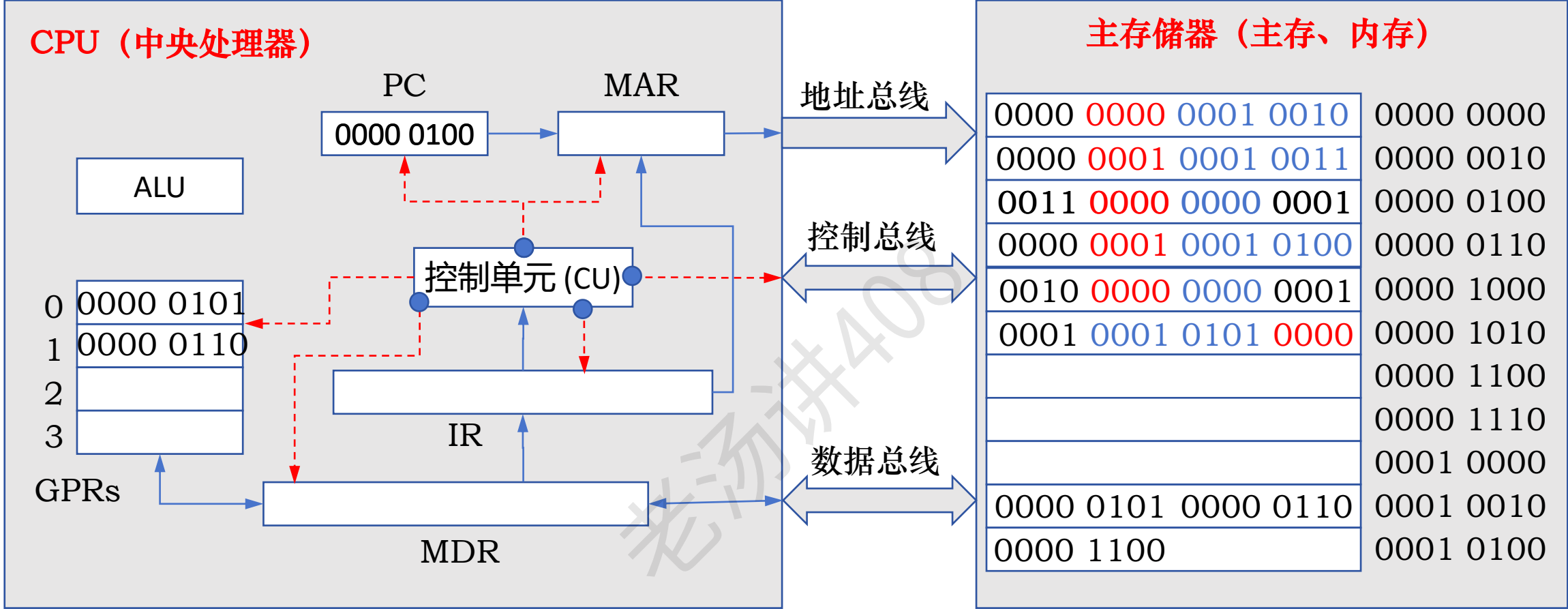
指令寄存器 (Instruction Register, IR)

内存地址寄存器 (Memory Address Register, MAR) 内存数据寄存器 (Memory Data Register, MDR)

算术逻辑单元 (Arithmetic and Logic Unit, ALU)

控制单元 (Control Unit, CU)

指令的设计和执行过程



通用寄存器组 (General Purpose Register set, GPRs)

程序计数器 (Program Counter, PC)

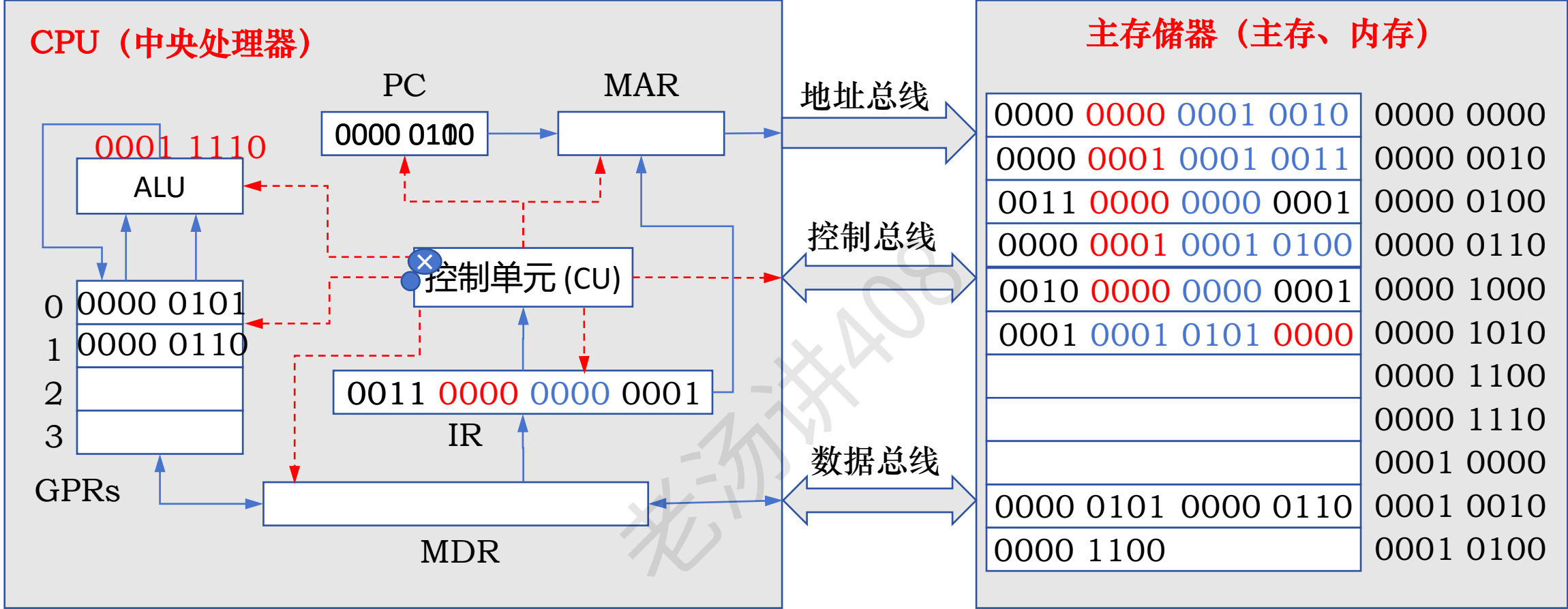
指令寄存器 (Instruction Register, IR)

内存地址寄存器 (Memory Address Register, MAR) 内存数据寄存器 (Memory Data Register, MDR)

算术逻辑单元 (Arithmetic and Logic Unit, ALU)

控制单元 (Control Unit, CU)

指令的设计和执行过程



通用寄存器组 (General Purpose Register set, GPRs)

程序计数器 (Program Counter, PC)

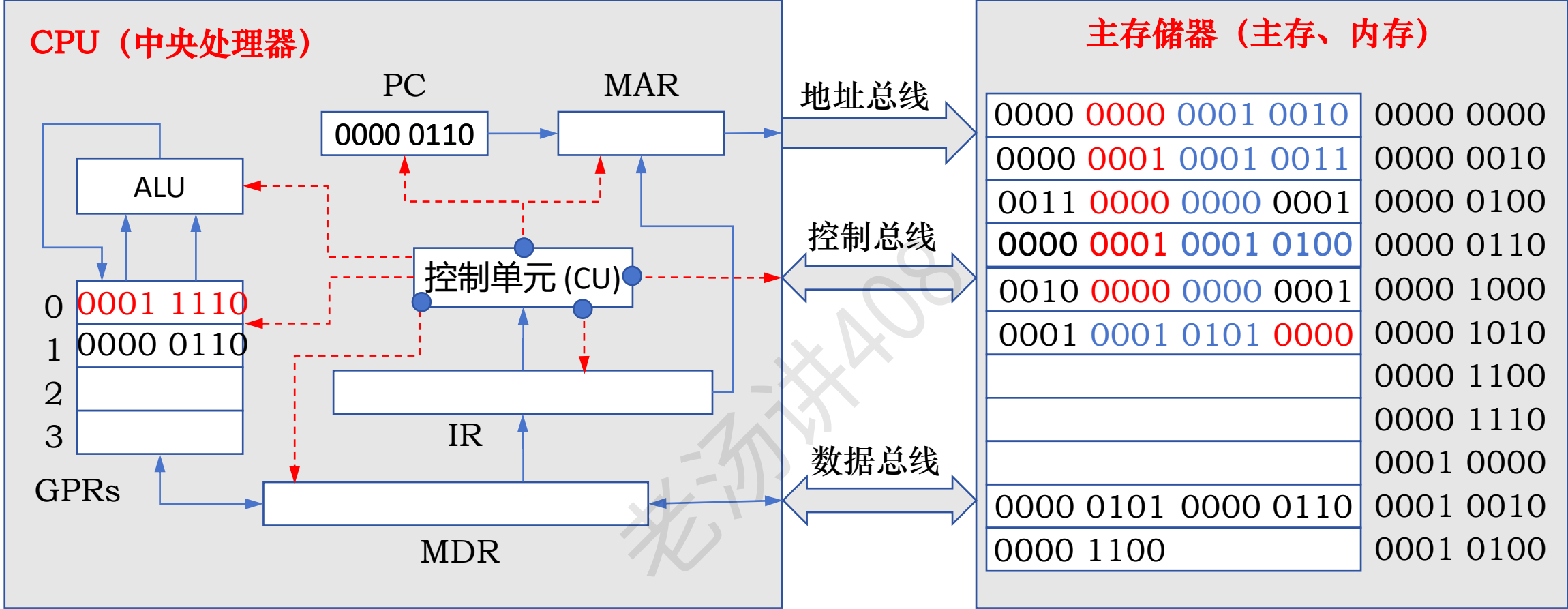
指令寄存器 (Instruction Register, IR)

内存地址寄存器 (Memory Address Register, MAR) 内存数据寄存器 (Memory Data Register, MDR)

算术逻辑单元 (Arithmetic and Logic Unit, ALU)

控制单元 (Control Unit, CU)

指令的设计和执行过程



通用寄存器组 (General Purpose Register set, GPRs)

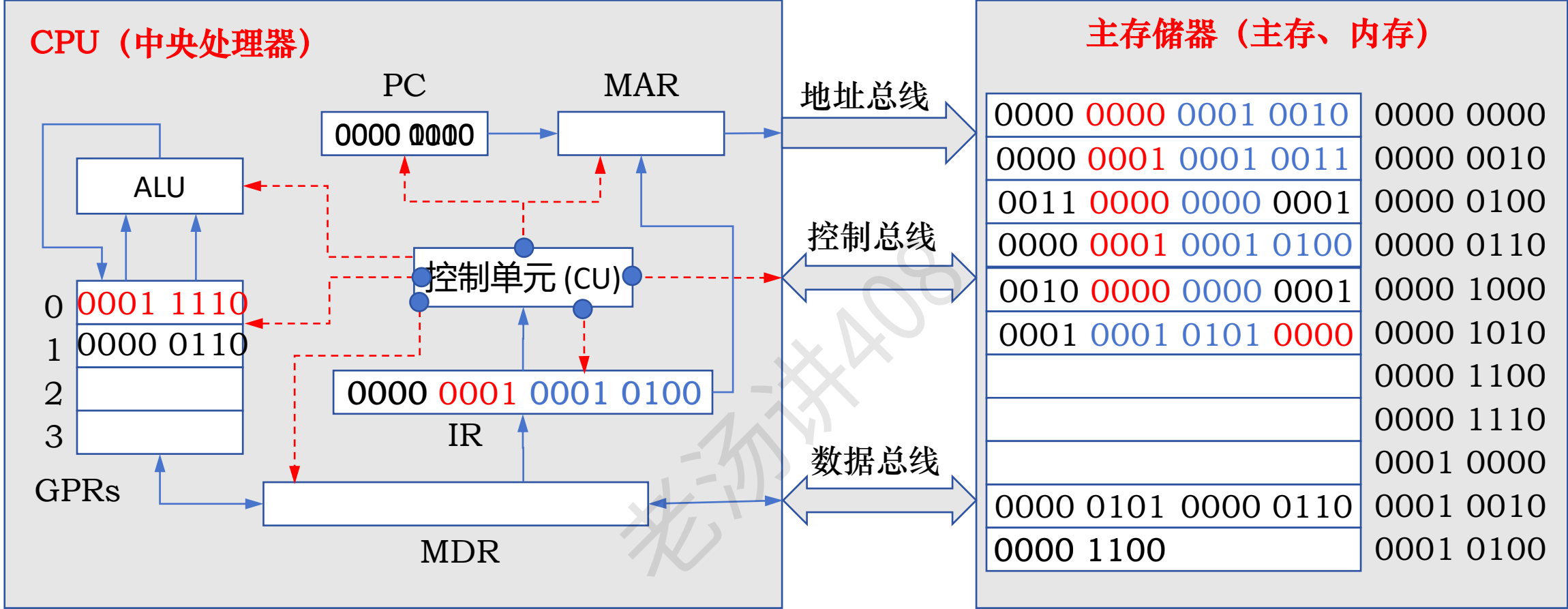
程序计数器 (Program Counter, PC)

指令寄存器 (Instruction Register, IR)

内存地址寄存器 (Memory Address Register, MAR) 内存数据寄存器 (Memory Data Register, MDR)

算术逻辑单元 (Arithmetic and Logic Unit, ALU) 控制单元 (Control Unit, CU)

指令的设计和執行过程



通用寄存器组 (General Purpose Register set, GPRs)

程序计数器 (Program Counter, PC)

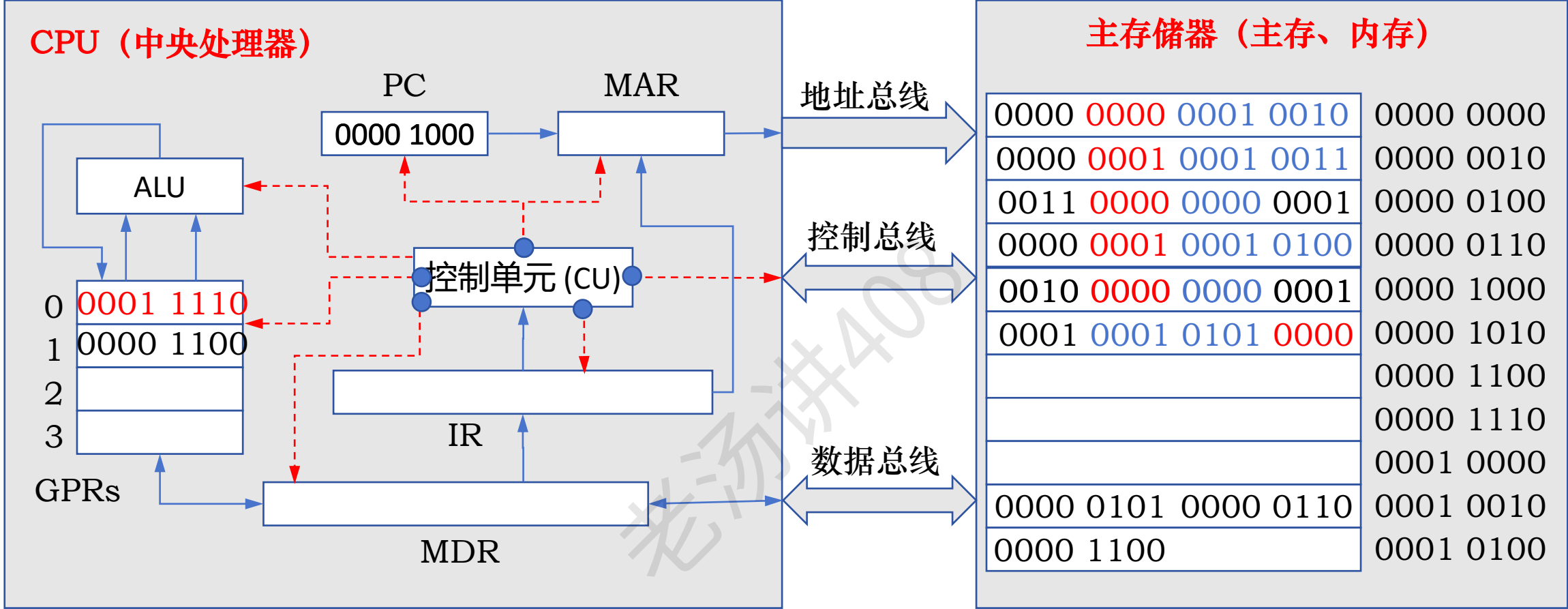
指令寄存器 (Instruction Register, IR)

内存地址寄存器 (Memory Address Register, MAR) 内存数据寄存器 (Memory Data Register, MDR)

算术逻辑单元 (Arithmetic and Logic Unit, ALU)

控制单元 (Control Unit, CU)

指令的指令和执行过程



通用寄存器组 (General Purpose Register set, GPRs)

程序计数器 (Program Counter, PC)

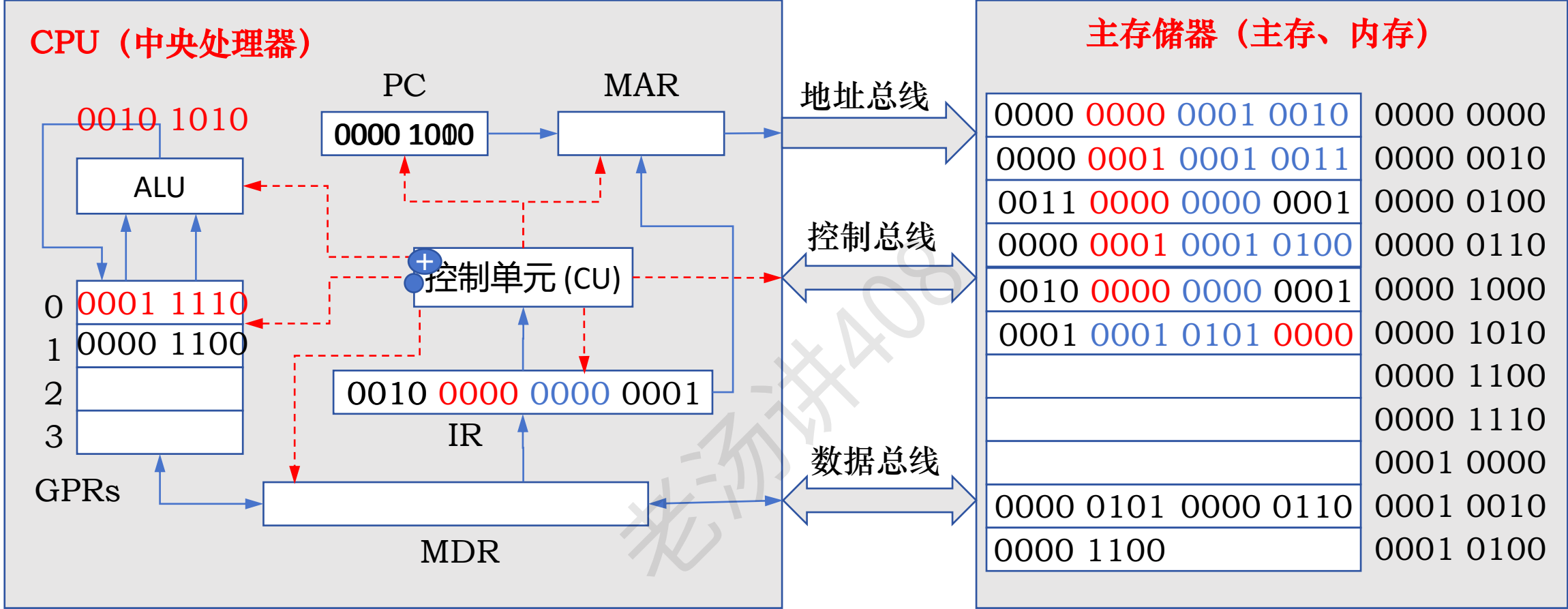
指令寄存器 (Instruction Register, IR)

内存地址寄存器 (Memory Address Register, MAR) 内存数据寄存器 (Memory Data Register, MDR)

算术逻辑单元 (Arithmetic and Logic Unit, ALU)

控制单元 (Control Unit, CU)

指令的设计和執行过程



通用寄存器组 (General Purpose Register set, GPRs)

程序计数器 (Program Counter, PC)

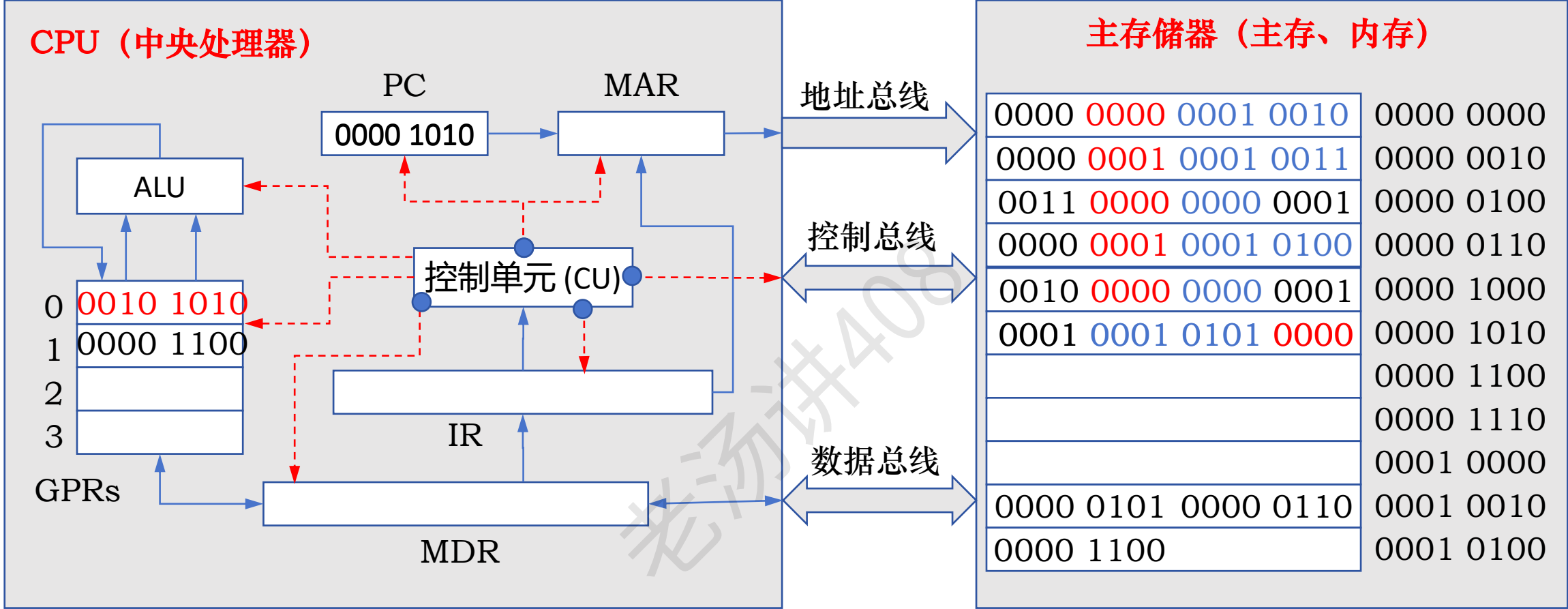
指令寄存器 (Instruction Register, IR)

内存地址寄存器 (Memory Address Register, MAR) 内存数据寄存器 (Memory Data Register, MDR)

算术逻辑单元 (Arithmetic and Logic Unit, ALU)

控制单元 (Control Unit, CU)

指令的设计和执行过程



通用寄存器组 (General Purpose Register set, GPRs)

程序计数器 (Program Counter, PC)

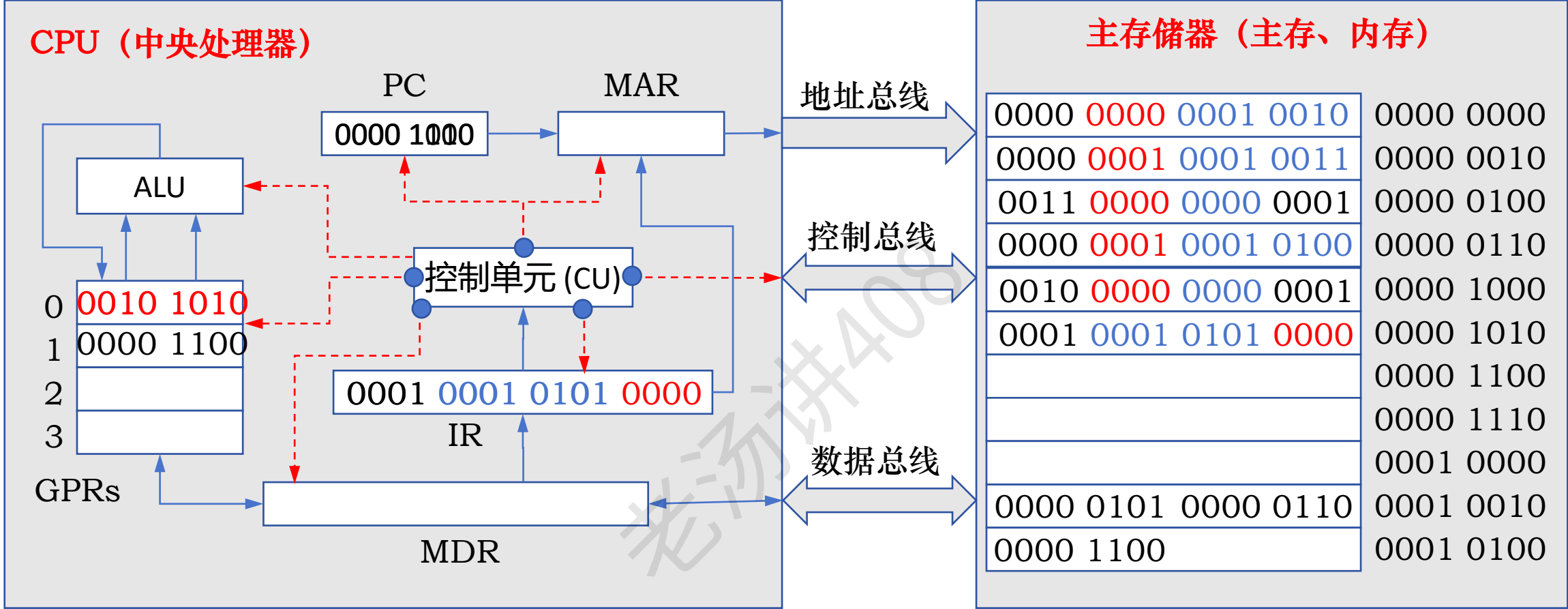
指令寄存器 (Instruction Register, IR)

内存地址寄存器 (Memory Address Register, MAR) 内存数据寄存器 (Memory Data Register, MDR)

算术逻辑单元 (Arithmetic and Logic Unit, ALU)

控制单元 (Control Unit, CU)

指令的设计和执行过程



通用寄存器组 (General Purpose Register set, GPRs)

程序计数器 (Program Counter, PC)

指令寄存器 (Instruction Register, IR)

内存地址寄存器 (Memory Address Register, MAR) 内存数据寄存器 (Memory Data Register, MDR)

算术逻辑单元 (Arithmetic and Logic Unit, ALU)

控制单元 (Control Unit, CU)

注意！！！！

第一章：计算机系统概述

第二章：总线系统

第三章：主存储器

第四章：数据的表示与运算

第五章：指令系统



第六章：中央处理器 (CPU)

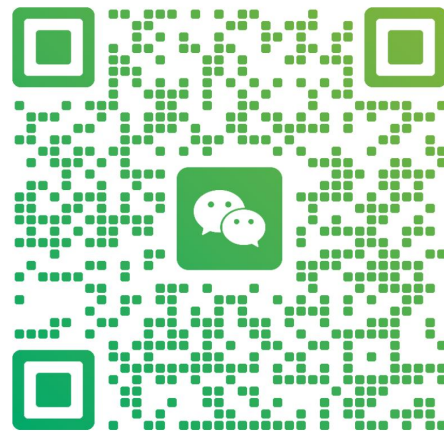
第七章：输入输出 (I/O) 系统

加老汤微信，帮你 408 冲 120+



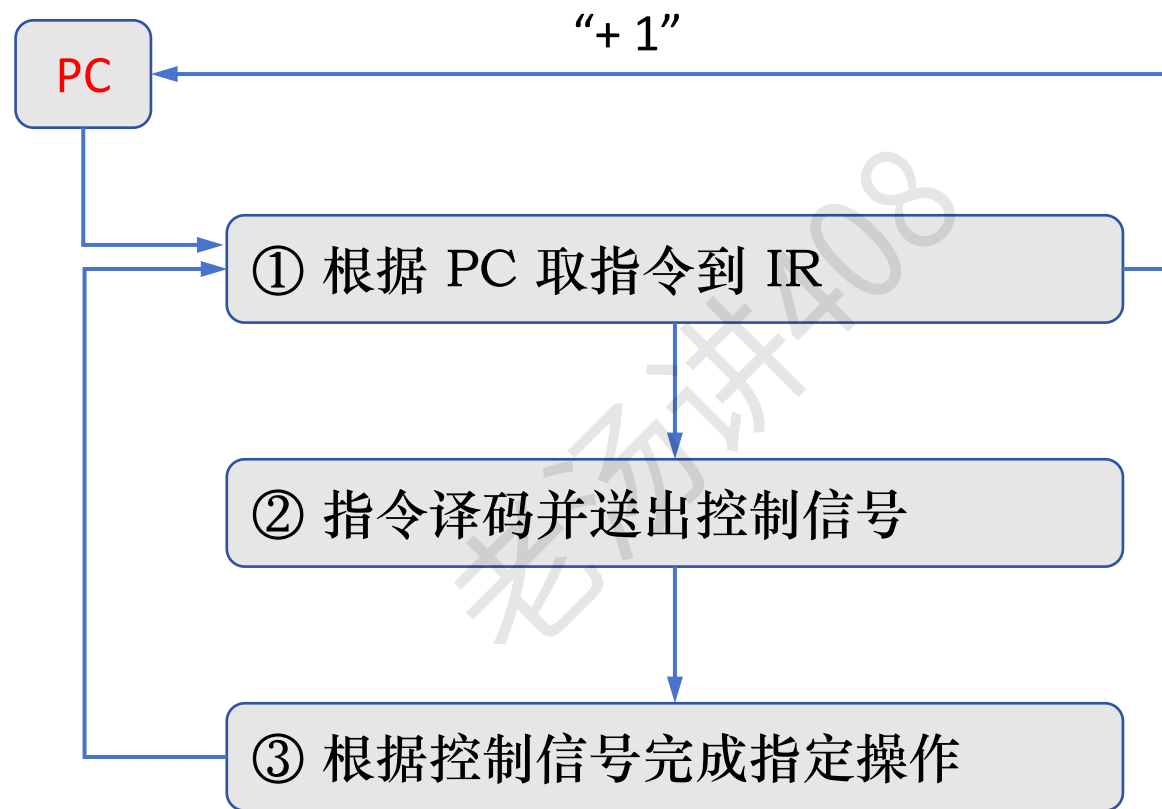
老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

总结下指令的执行过程

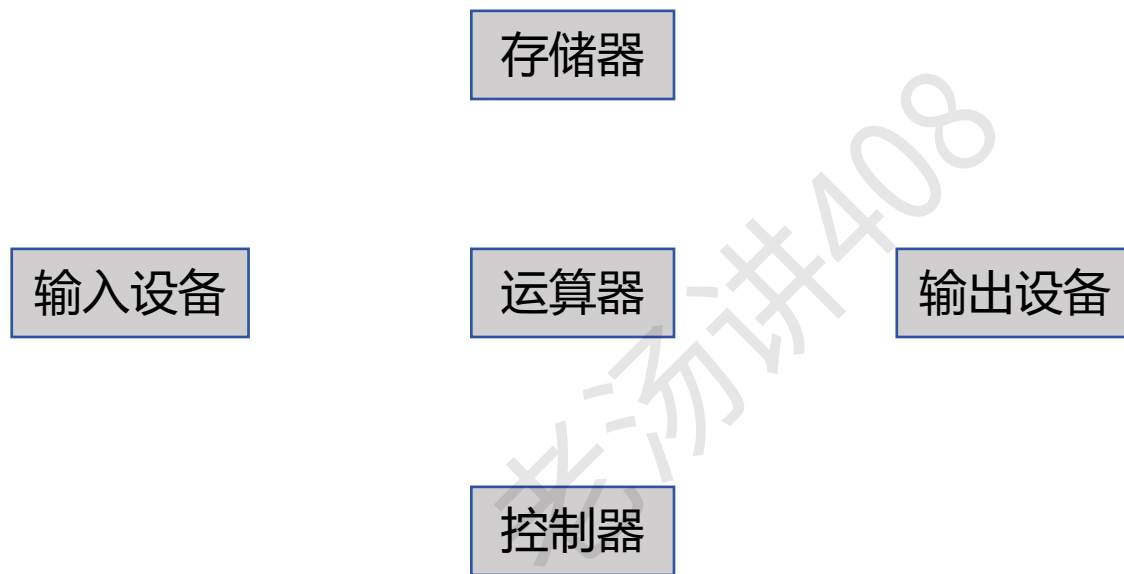


冯·诺依曼计算机结构的 5 个特点

- ①：计算机由运算器、控制器、存储器、输入设备和输出设备 5 个基本部件组成
- ②：指令和数据以同等地位存储在存储器中，形式上没有区别，但计算机应能区分它们
- ③：指令和数据均用二进制代码表示
- ④：指令由操作码和操作数组成两部分组成，操作数可以是多个
- ⑤：采用“存储程序”工作方式

冯·诺依曼计算机结构的 5 个特点

①：计算机由运算器、控制器、存储器、输入设备和输出设备 5 个基本部件组成



运算器是计算机的执行部件 {

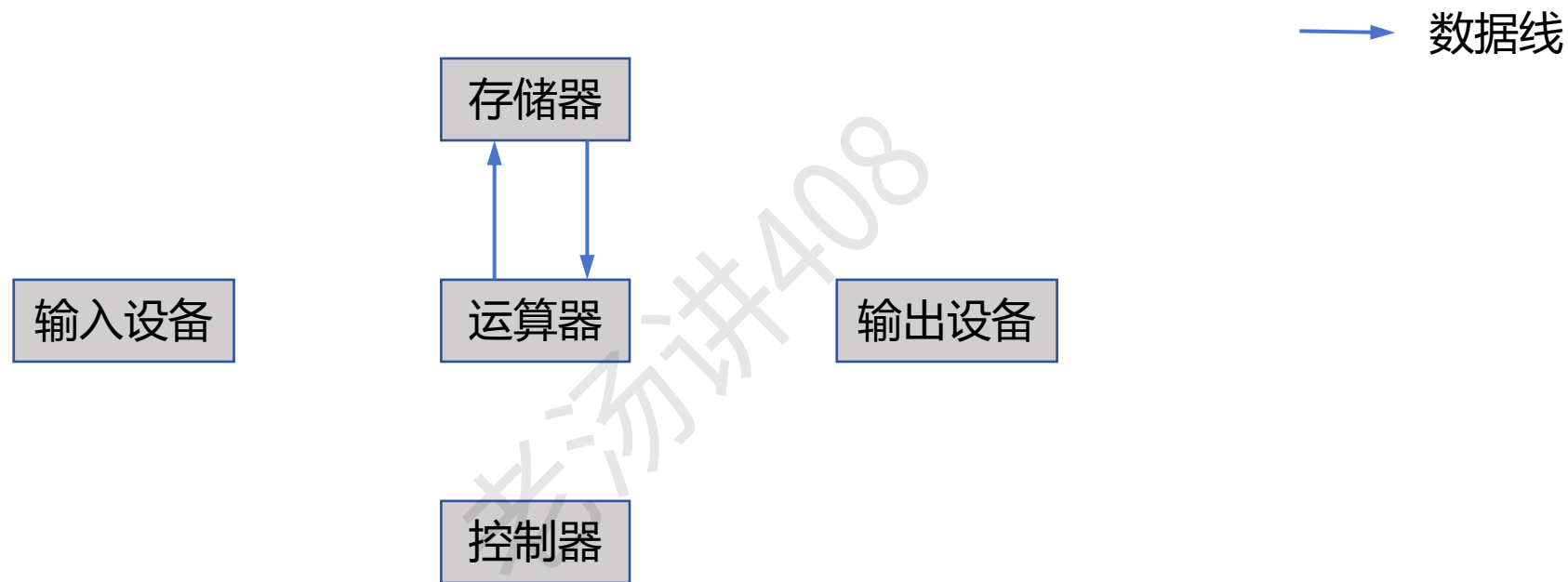
- ① 算术运算：加、减、乘、除等
- ② 逻辑运算：与、或、非、异或、移位等

核心部件：ALU

通用寄存器组 (GRPs)

冯·诺依曼计算机结构的 5 个特点

①：计算机由运算器、控制器、存储器、输入设备和输出设备 5 个基本部件组成

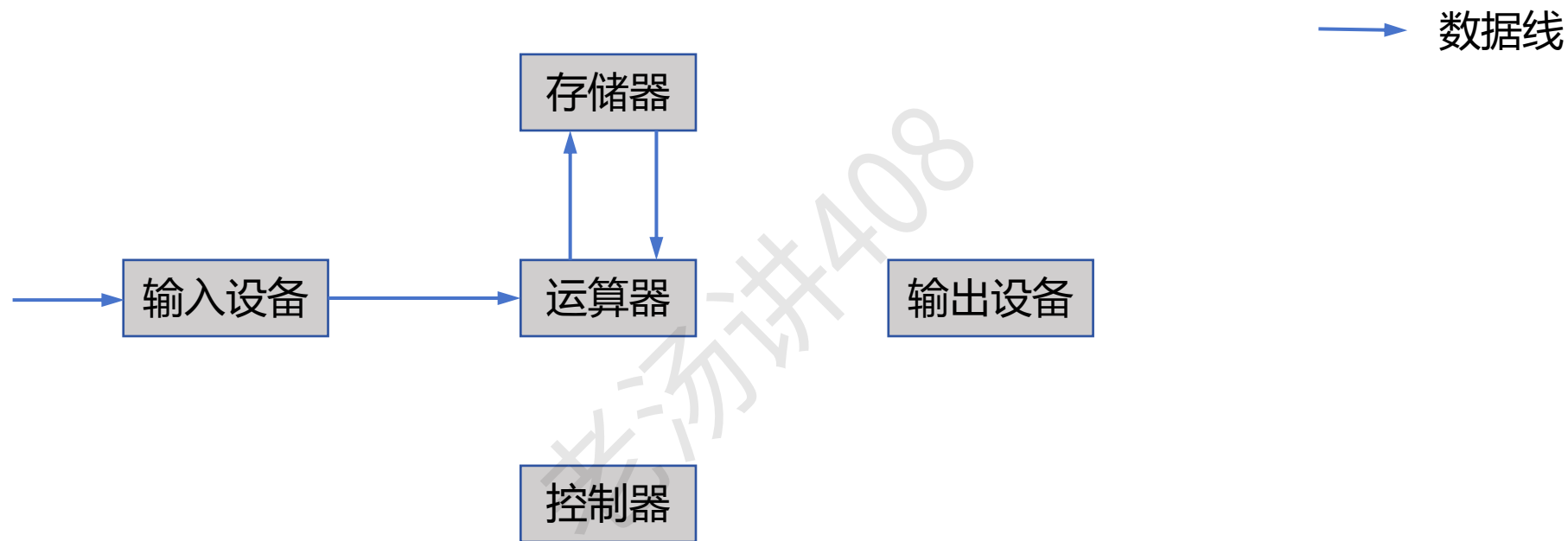


存储器用于存放数据和指令 {

- ① 主存储器（内存）
- ② 辅助存储器（外存）：比如硬盘等

冯·诺依曼计算机结构的 5 个特点

①：计算机由运算器、控制器、存储器、输入设备和输出设备 5 个基本部件组成

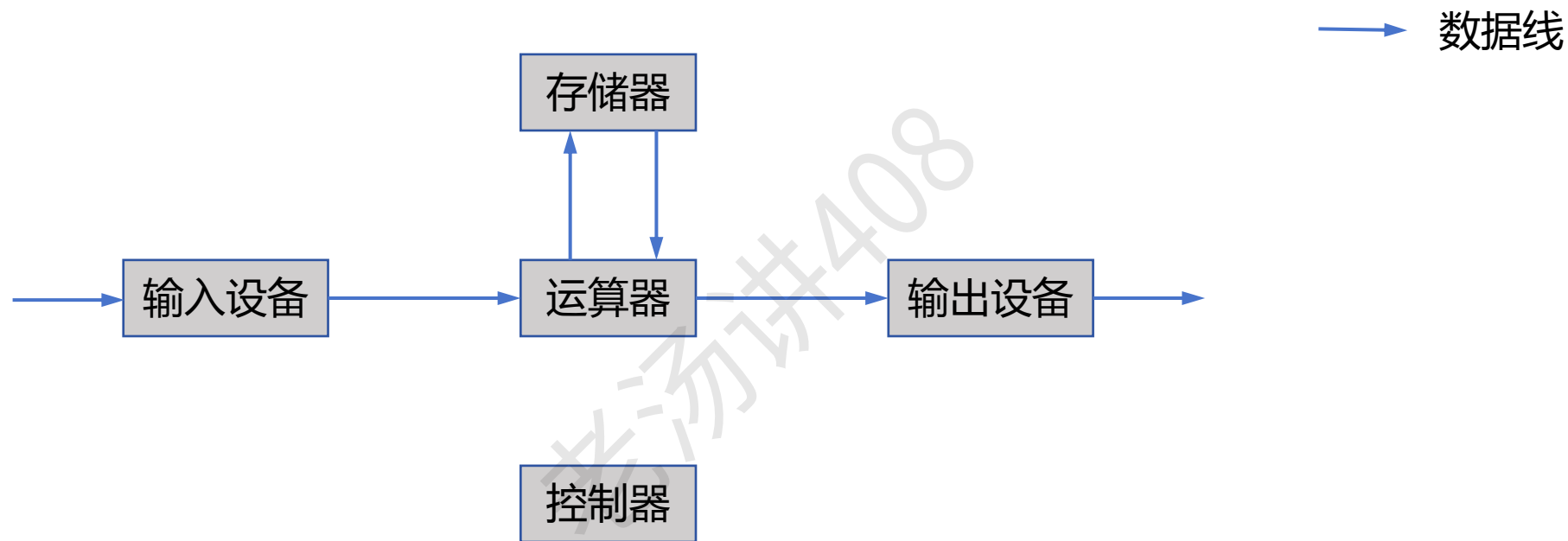


输入设备的主要功能是将程序和数据以机器所能识别和接受的信息形式输入计算机

最常用也最基本的输入设备是键盘，此外还有鼠标、扫描仪、摄像机等

冯·诺依曼计算机结构的 5 个特点

①：计算机由运算器、控制器、存储器、输入设备和输出设备 5 个基本部件组成

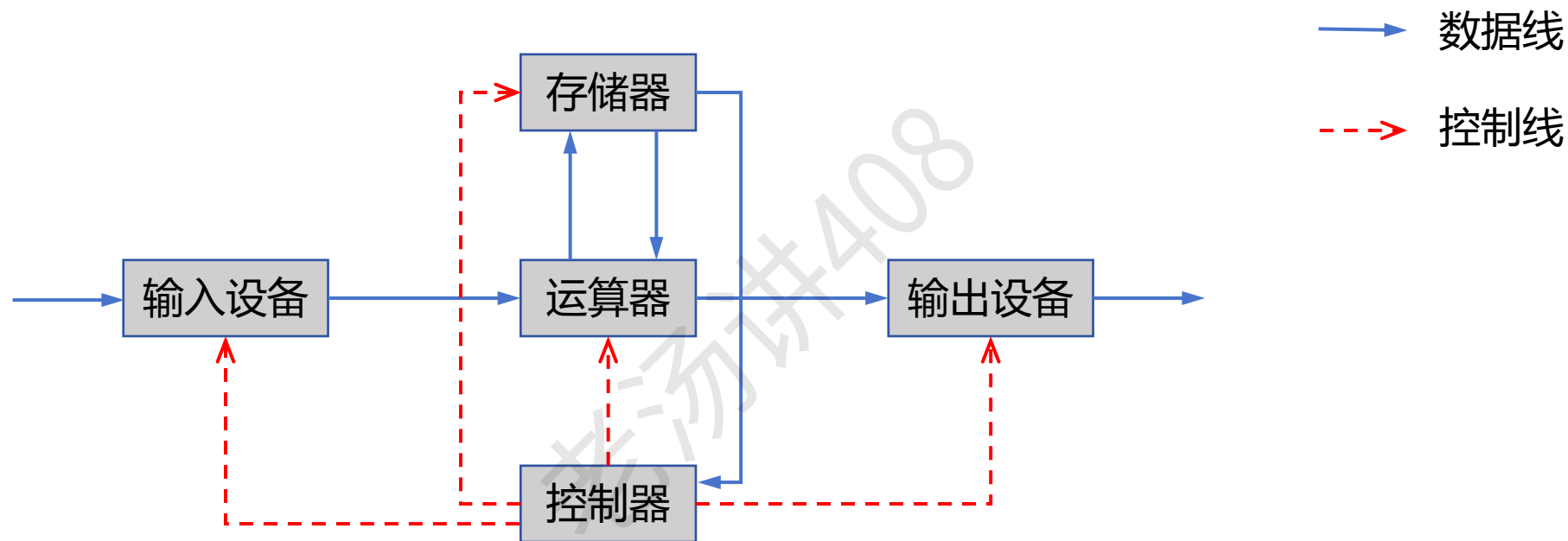


输出设备的任务是将计算机处理的结果以人们所能接受的形式或其他系统所要求的信息形式输出

最常用、最基本的输出设备是显示器、打印机

冯·诺依曼计算机结构的 5 个特点

①：计算机由运算器、控制器、存储器、输入设备和输出设备 5 个基本部件组成



控制器是计算机的指挥中心，由其“指挥”各部件自动协调地进行工作

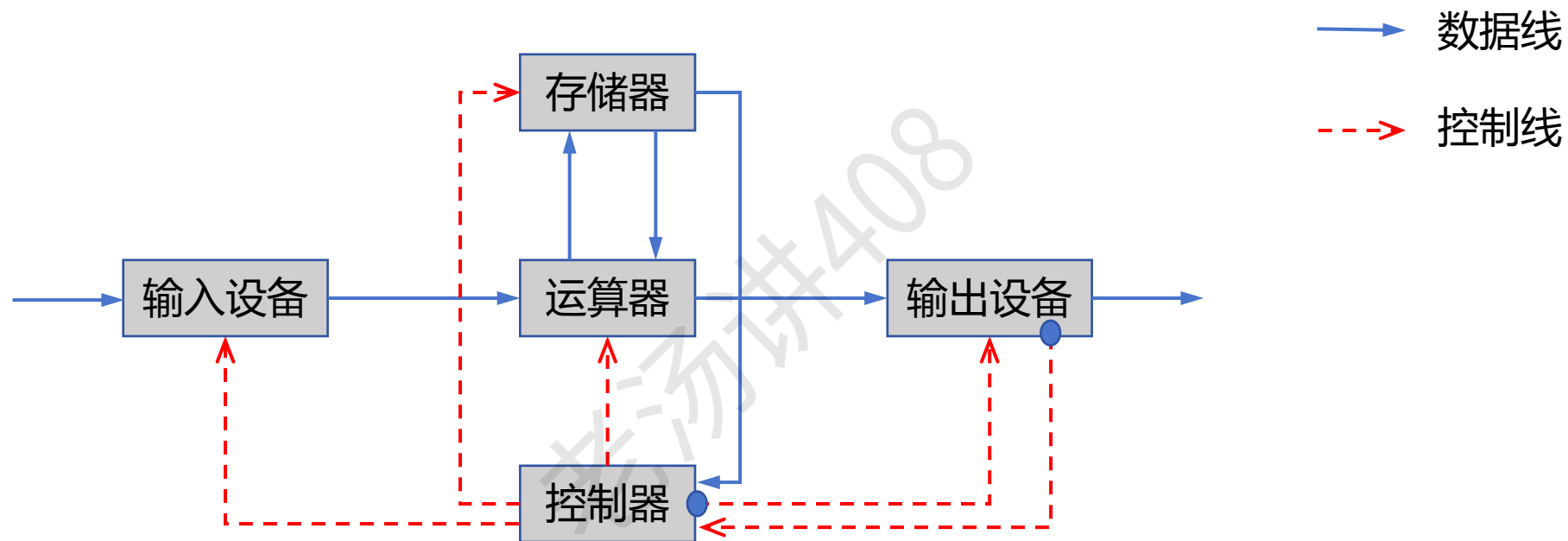
程序计数器 (PC)

指令寄存器 (IR)

控制单元 (CU)

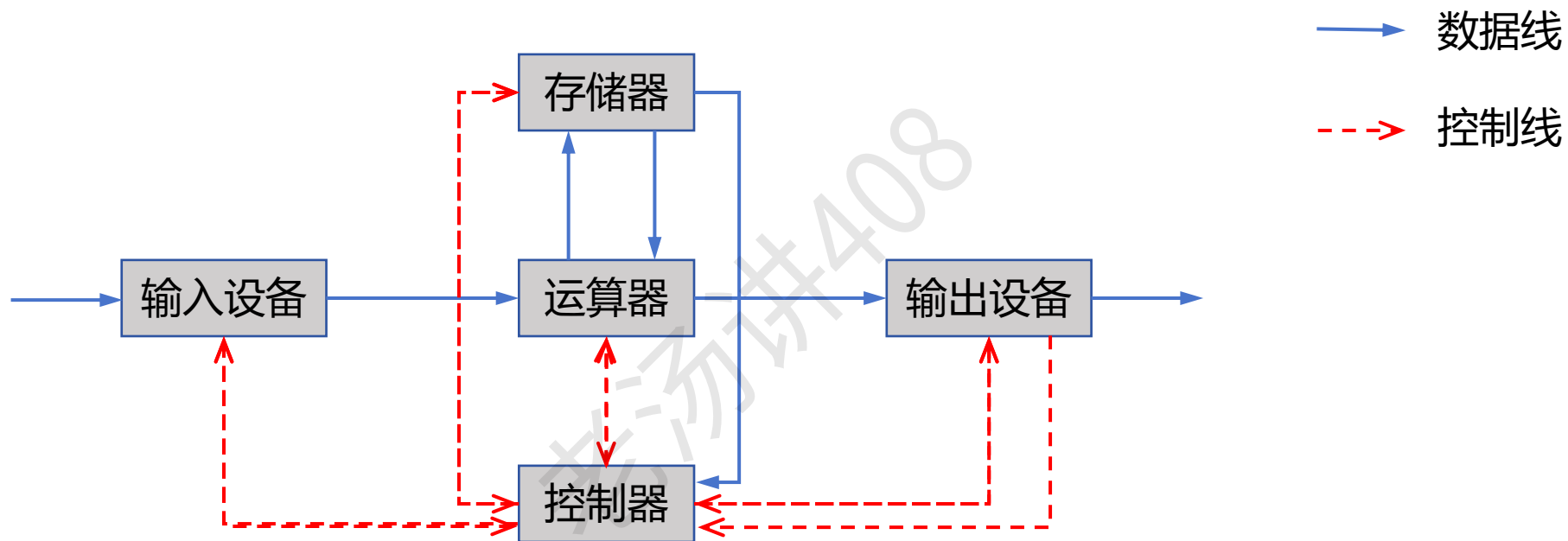
冯·诺依曼计算机结构的 5 个特点

①：计算机由运算器、控制器、存储器、输入设备和输出设备 5 个基本部件组成



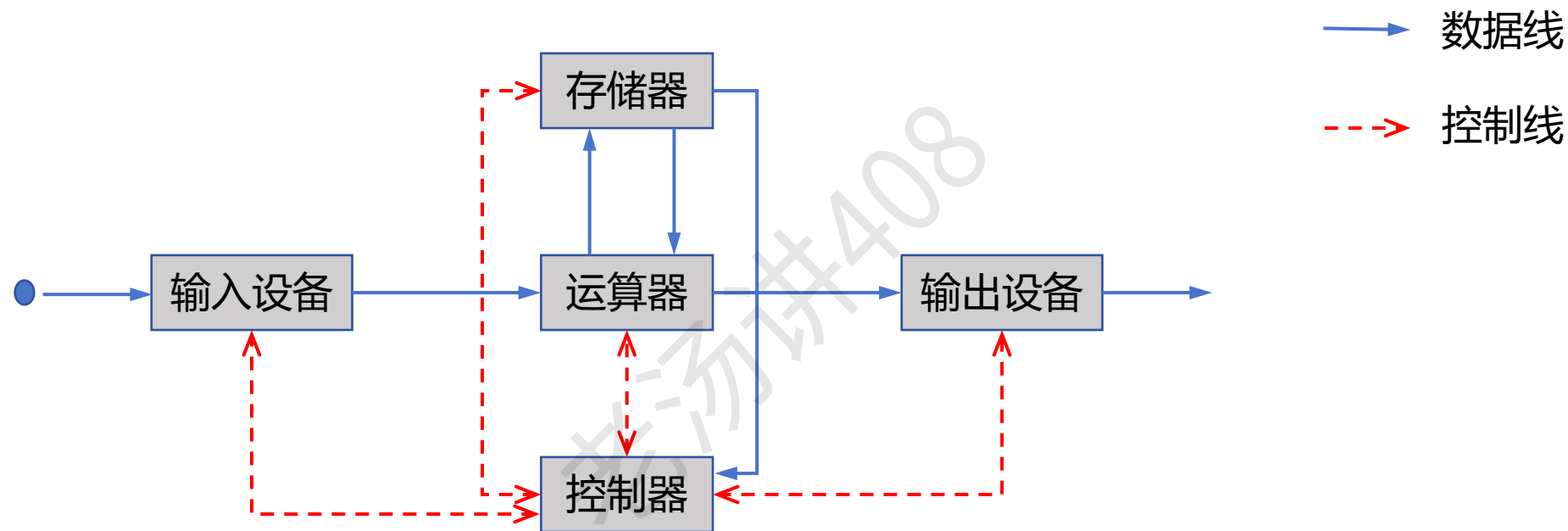
冯·诺依曼计算机结构的 5 个特点

①：计算机由运算器、控制器、存储器、输入设备和输出设备 5 个基本部件组成



冯·诺依曼计算机结构的 5 个特点

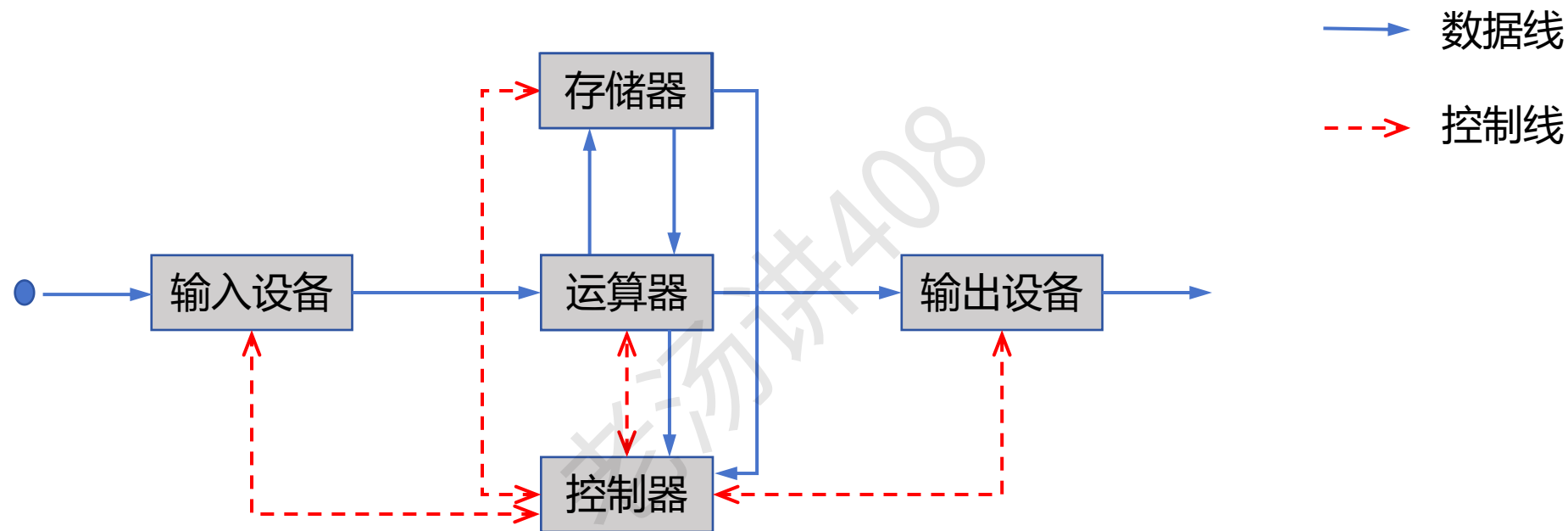
①：计算机由运算器、控制器、存储器、输入设备和输出设备 5 个基本部件组成



以**运算器**为中心的

冯·诺依曼计算机结构的 5 个特点

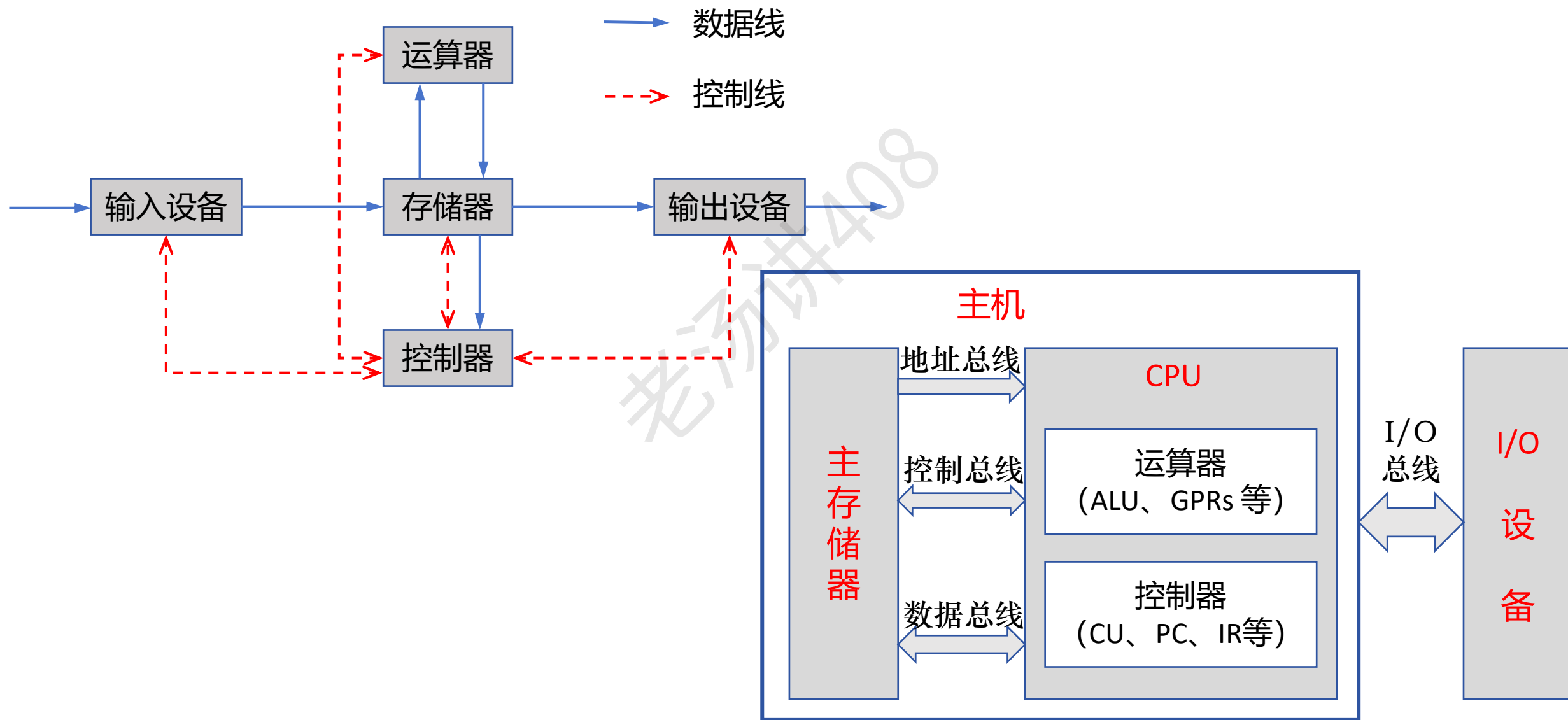
①：计算机由运算器、控制器、存储器、输入设备和输出设备 5 个基本部件组成



以**存储器**为中心的

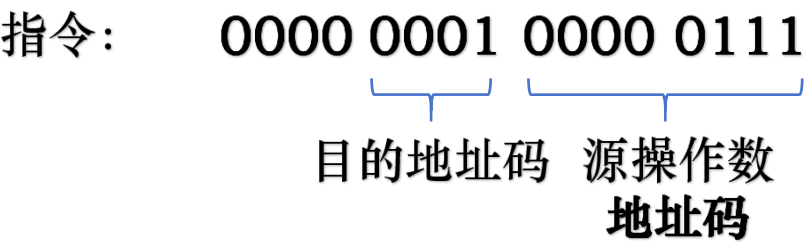
冯·诺依曼计算机结构的 5 个特点

①：计算机由运算器、控制器、存储器、输入设备和输出设备 5 个基本部件组成



冯·诺依曼计算机结构的 5 个特点

- ②：指令和数据以同等地位存储在存储器中，形式上没有区别
- ③：指令和数据均用二进制代码表示
- ④：指令由操作码和地址码组成两部分组成，地址码可以是多个



主存储器（主存、内存）					
0000	0000	0001	0010	0000	0000
0000	0001	0001	0011	0000	0010
0011	0000	0000	0001	0000	0100
0000	0001	0001	0100	0000	0110
0010	0000	0000	0001	0000	1000
0001	0001	0101	0000	0000	1010
				0000	1100
				0000	1110
				0001	0000
0000	0101	0000	0110	0001	0010
0000	1100			0001	0100

冯·诺依曼计算机结构的 5 个特点

⑤：采用 “存储程序” 工作方式

“存储程序” 的基本思想是：将事先编制好的程序和原始数据送入主存储器后才能执行，一旦程序被启动执行，就无须操作人员的干预，计算机会自动逐条执行指令，直至程序执行结束

机器语言、汇编语言和高级语言

① 什么是机器语言、汇编语言、高级语言？

② 什么是汇编器、解释器以及编译器？

什么是机器语言、汇编语言和高级语言？

表达式: $ax + b$

0000 0000 0001 0010
0000 0001 0001 0011
0011 0000 0000 0001
0000 0001 0001 0100
0010 0000 0000 0001
0001 0001 0101 0000



LOAD R0 12H
LOAD R1 13H
MUL R0 R0 R1
LOAD R1 14H
ADD R0 R0 R1
STORE 15H R0

机器语言

汇编语言

汇编器



什么是机器语言、汇编语言和高级语言？

表达式: $ax + b$

<u>0000 0000 0001 0010</u>	↔	LOAD R0 12H
<u>0000 0001 0001 0011</u>	↔	LOAD R1 13H
<u>0011 0000 0000 0001</u>	↔	MUL R0 R0 R1
<u>0000 0001 0001 0100</u>	↔	LOAD R1 14H
<u>0010 0000 0000 0001</u>	↔	ADD R0 R0 R1
<u>0001 0001 0101 0000</u>	↔	STORE 15H R0

机器语言

汇编语言

汇编器

① 可移植性差

② 编写难度高

什么是机器语言、汇编语言和高级语言？

表达式: $ax + b$

```
0000 0000 0001 0010
0000 0001 0001 0011
0011 0000 0000 0001
0000 0001 0001 0100
0010 0000 0000 0001
0001 0001 0101 0000
```



```
LOAD R0 12H
LOAD R1 13H
MUL R0 R0 R1
LOAD R1 14H
ADD R0 R0 R1
STORE 15H R0
```



```
int main() {
    short a = 5;
    short x = 6;
    short b = 12;
    short res = a * x + b;
    printf("res is %d\n", res);
}
```

机器语言

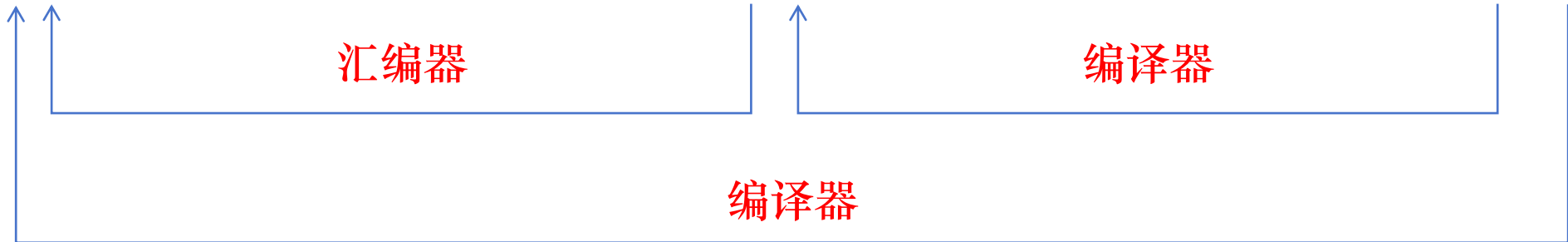
汇编语言

高级语言 (C 语言等)

汇编器

编译器

编译器



什么是机器语言、汇编语言和高级语言？

表达式: $ax + b$

```
int main() {  
    short a = 5;  
    short x = 6;  
    short b = 12;  
    short res = a * x + b;  
    printf("res is %d\n", res);  
}
```

编译器 A

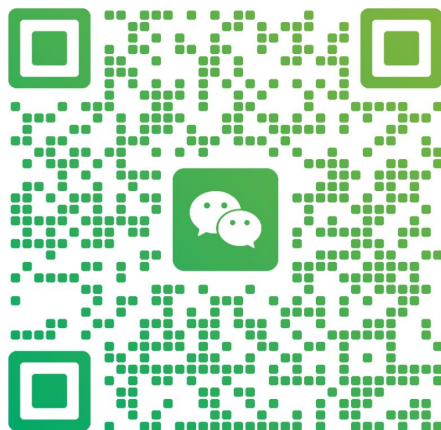
编译器 B



高级语言程序翻译成机器语言程序

加老汤微信，帮你 408 冲 120+

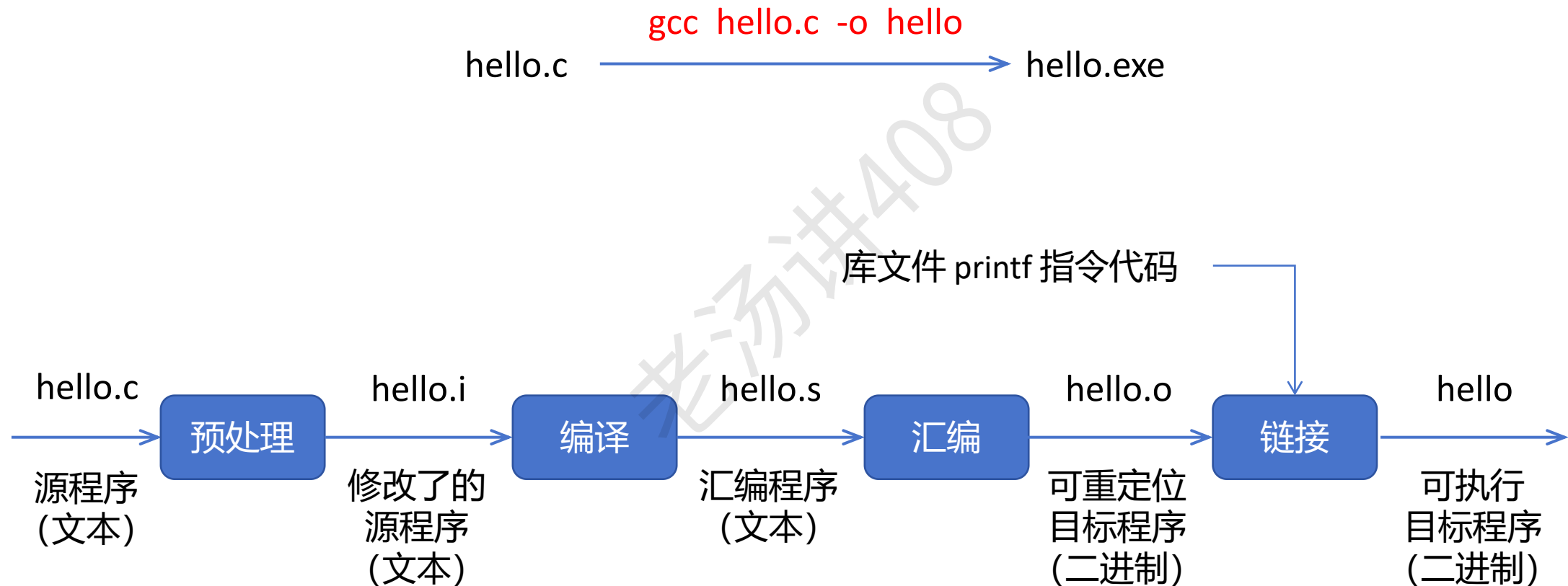
① 编译器



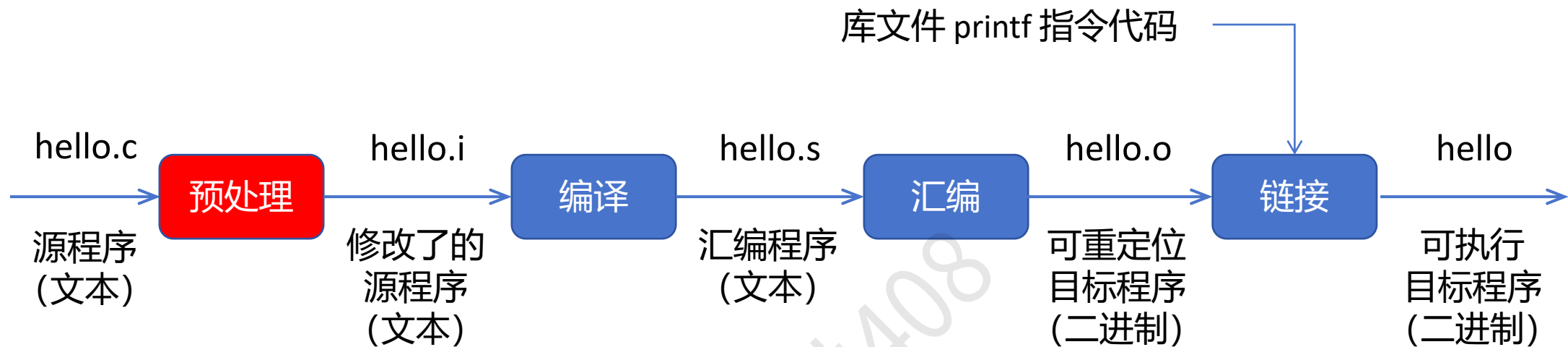
② 解释器

扫一扫上面的二维码图案，加我为朋友。

高级语言程序翻译成机器语言程序：编译器



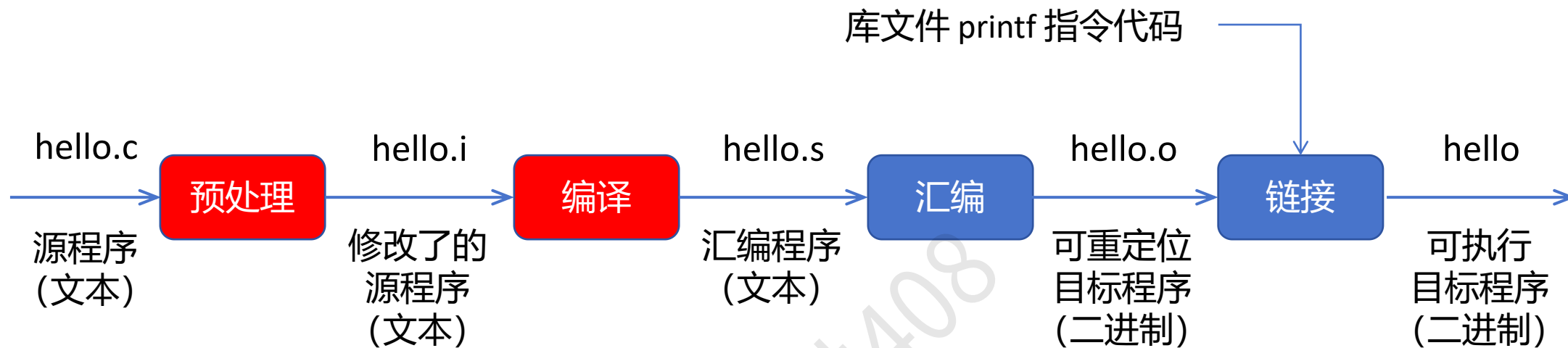
高级语言程序翻译成机器语言程序：编译器



```
#include <stdio.h>
```

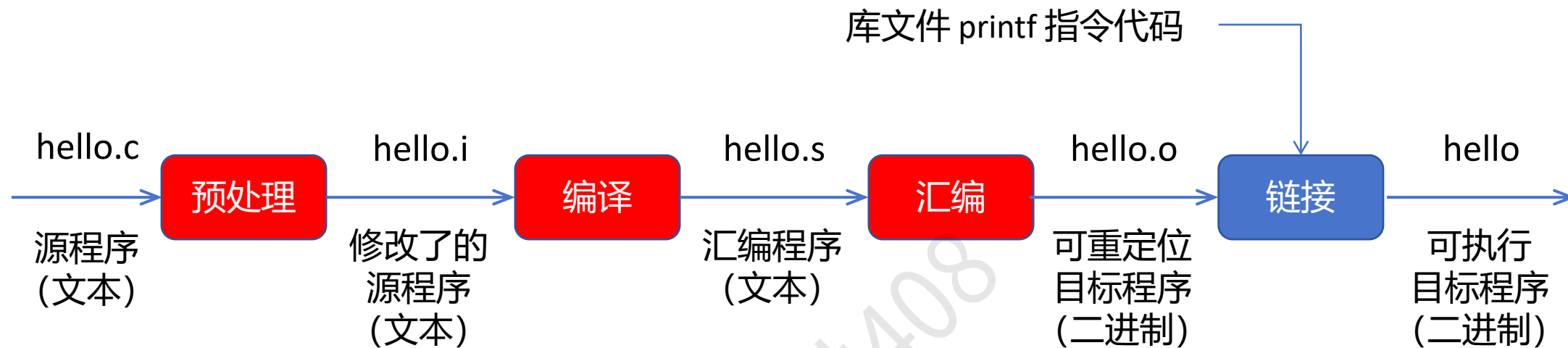
```
int main() {  
    printf("Hello World\n");  
    return 0;  
}
```

高级语言程序翻译成机器语言程序：编译器



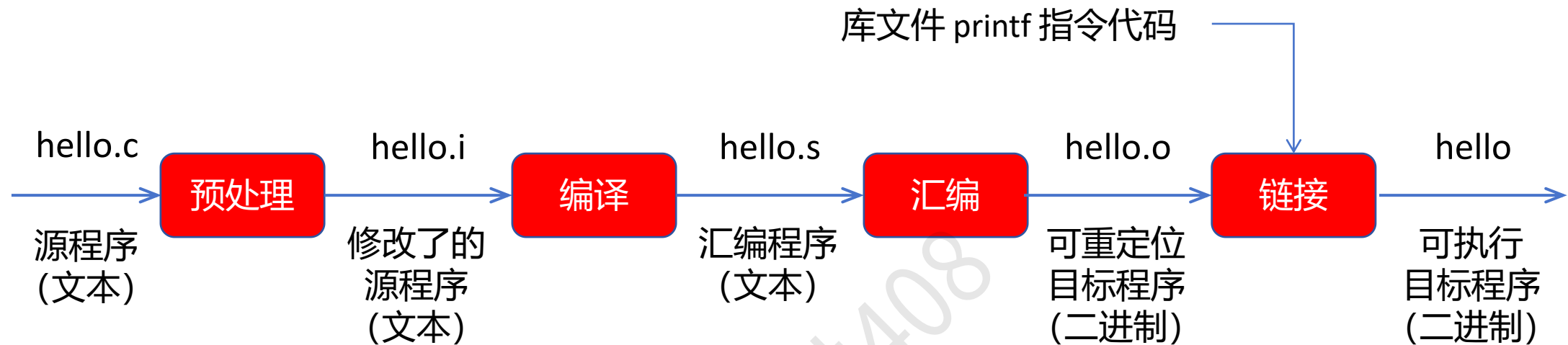
这个阶段将 `hello.i` 中的程序翻译成对应的汇编语言程序

高级语言程序翻译成机器语言程序：编译器



将 `hello.s` 中的汇编程序翻译成对应的机器语言程序

高级语言程序翻译成机器语言程序：编译器



```
#include <stdio.h>
```

```
int main() {  
    printf("Hello World\n");  
    return 0;  
}
```

高级语言程序翻译成机器语言程序：解释器

编译器：将高级语言程序一次性翻译成汇编语言或者机器语言目标程序

```
int main() {  
    short a = 5;  
    short x = 6;  
    short b = 12;  
    short res = a * x + b;  
    printf("res is %d\n", res);  
}
```

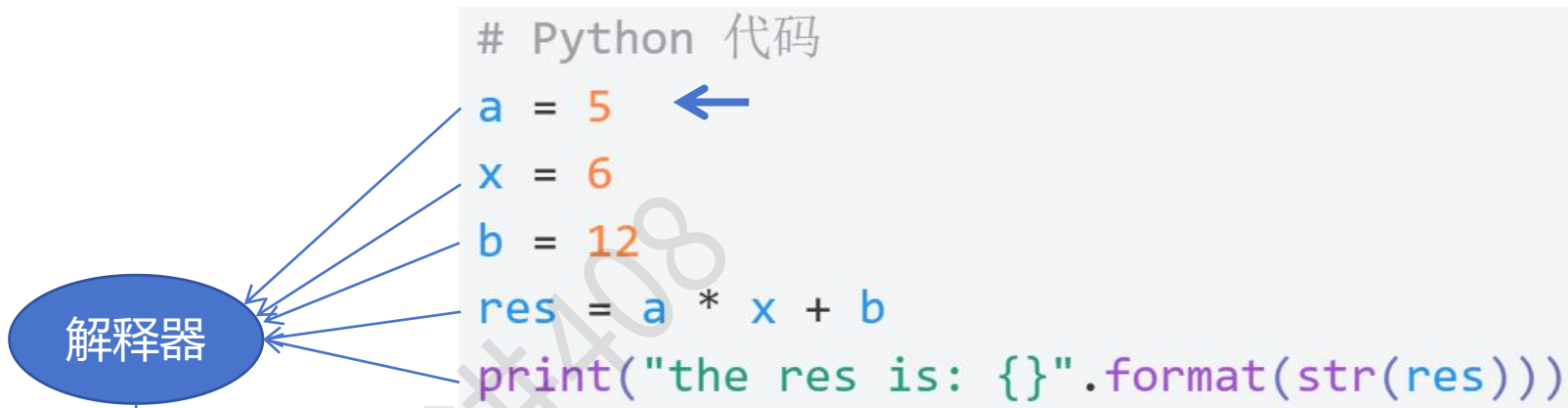


```
00000000 <_main>:  
 0: 55          push    %ebp  
 1: 89 e5       mov     %esp,%ebp  
 3: 83 e4 f0    and     $0xffffffff0,%esp  
 6: 83 ec 20    sub     $0x20,%esp  
 9: e8 00 00 00 00 call    e <_main+0xe>  
 e: 66 c7 44 24 1e 05 00 movw    $0x5,0x1e(%esp)  
15: 66 c7 44 24 1c 06 00 movw    $0x6,0x1c(%esp)  
1c: 66 c7 44 24 1a 0c 00 movw    $0xc,0x1a(%esp)  
23: 0f b7 54 24 1e    movzwl 0x1e(%esp),%edx  
28: 0f b7 44 24 1c    movzwl 0x1c(%esp),%eax  
2d: 0f af d0       imul    %eax,%edx  
30: 0f b7 44 24 1a    movzwl 0x1a(%esp),%eax  
35: 01 d0       add     %edx,%eax  
37: 66 89 44 24 18    mov     %ax,0x18(%esp)  
3c: 0f bf 44 24 18    movswl 0x18(%esp),%eax  
41: 89 44 24 04       mov     %eax,0x4(%esp)  
45: c7 04 24 00 00 00 00 movl    $0x0,(%esp)  
4c: e8 00 00 00 00    call    51 <_main+0x51>  
51: b8 00 00 00 00    mov     $0x0,%eax  
56: c9          leave  
57: c3          ret
```



高级语言程序翻译成机器语言程序：解释器

解释器：将源程序中的语句按其执行顺序逐条翻译成机器指令并立即执行

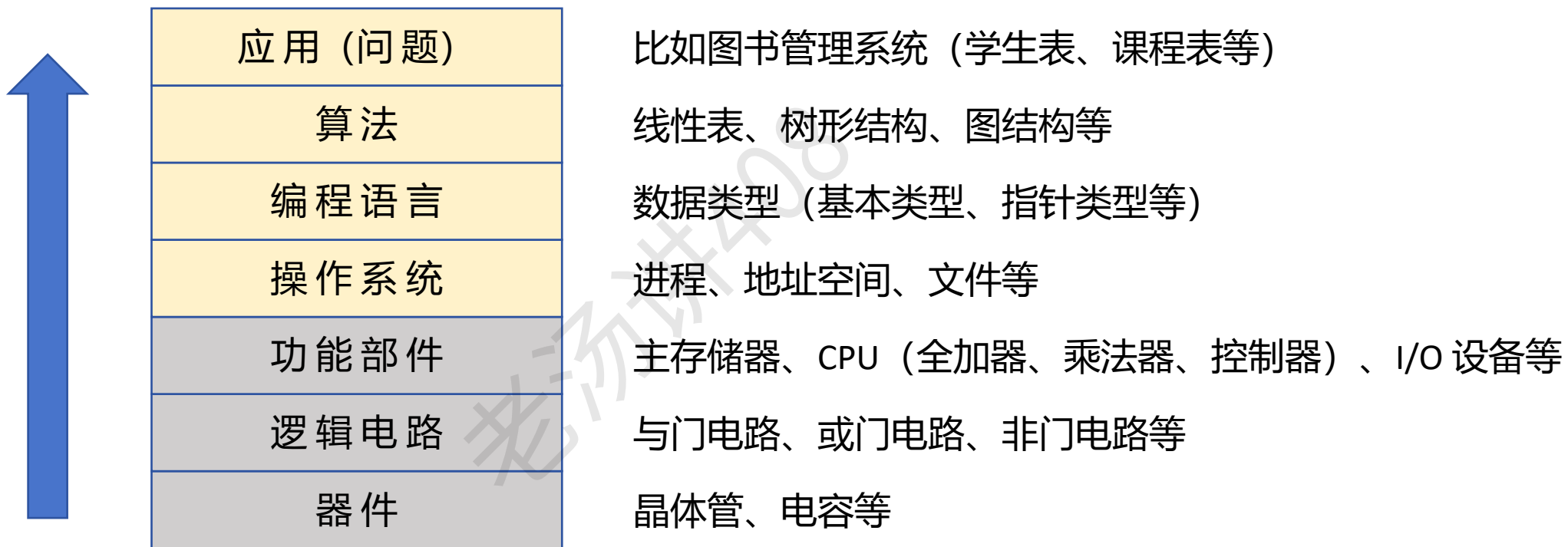


0f	bf	44	24	18		movswl	0x18(%esp),%eax
89	44	24	04			mov	%eax,0x4(%esp)
c7	04	24	00	00	00	movl	\$0x0,(%esp)
e8	00	00	00	00		call	51 <_main+0x51>
b8	00	00	00	00		mov	\$0x0,%eax
66	89	44	24	18		mov	%ax,0x18(%esp)

编译执行 vs 解释执行

	执行速度	开发周期	
编译执行	快	长	C、C++、Go 等
解释执行	慢	短	Python、JavaScript 等
			Java

程序员最重要的能力：抽象能力



计算机体系结构中的 7 个伟大思想：

① 使用抽象分层简化设计



② 加速经常性事件

③ 通过并行提高性能

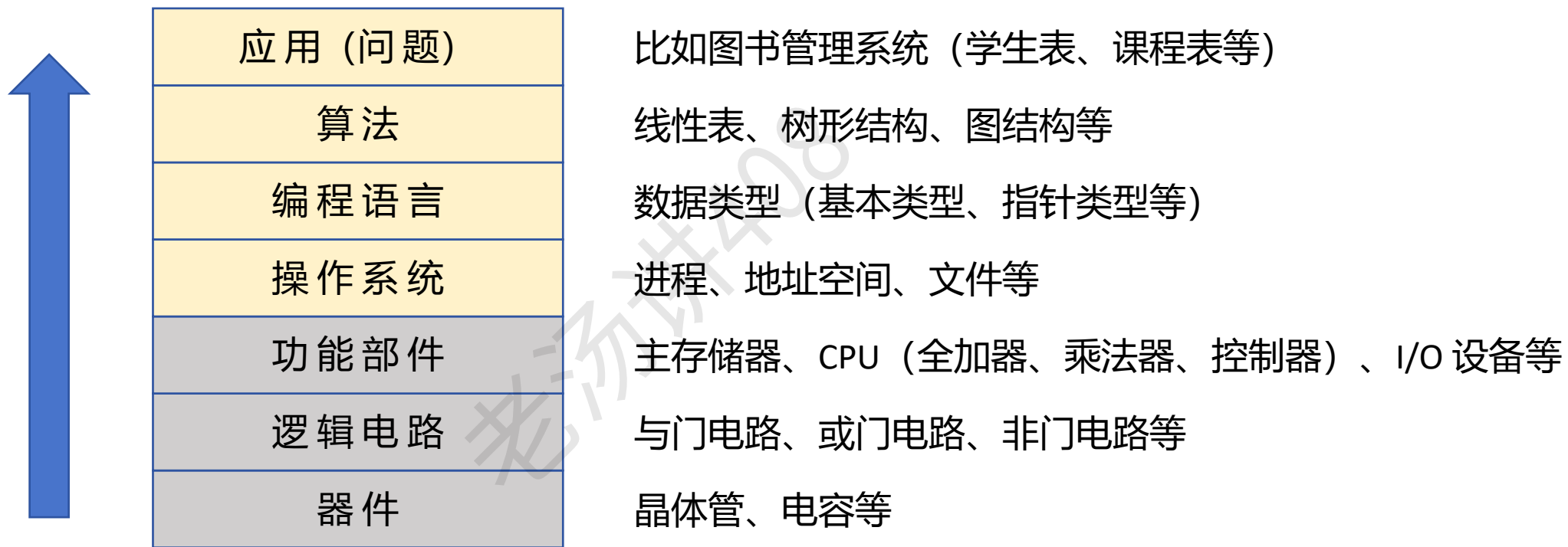
④ 通过流水线提高性能

⑤ 通过预测提高性能

⑥ 存储层次

⑦ 通过冗余提高可靠性

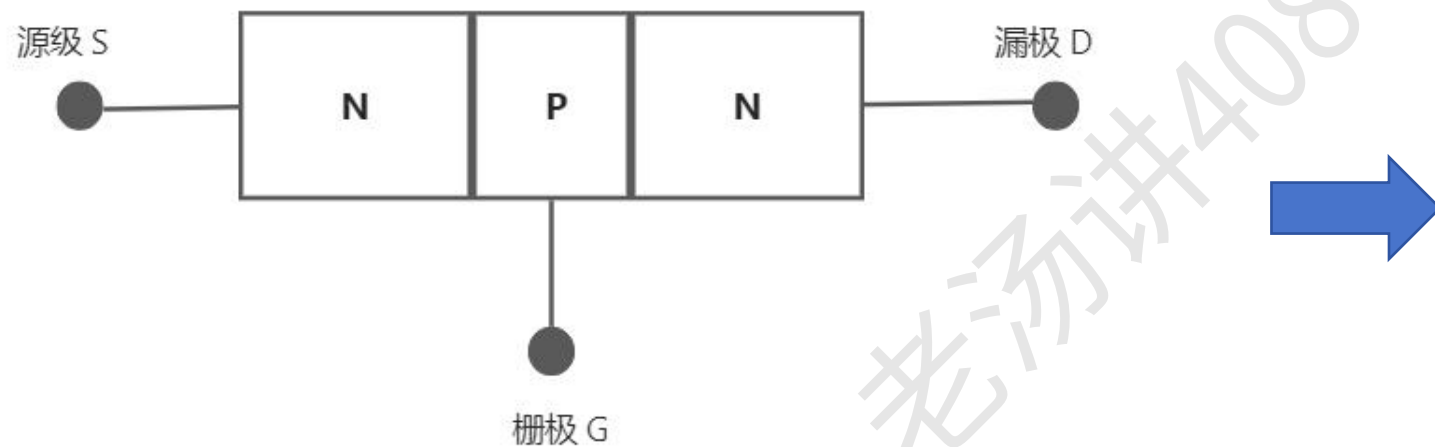
计算机系统的层次结构（抽象分层思想）



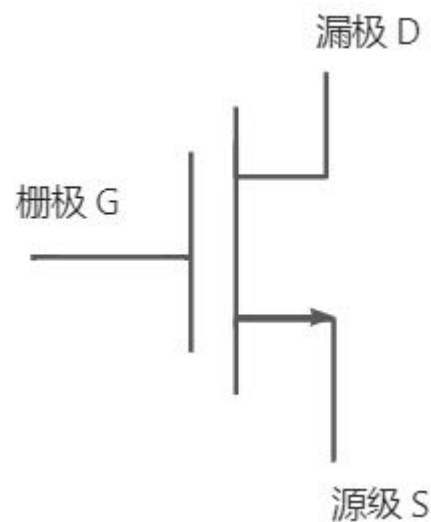
中间的每一层的功能都是利用下面一层提供的基本服务来完成了，每一层又为上一层提供更强大的服务

晶体管（场效应管 - MOS 管）

半导体：掺了硼元素、磷元素的硅



NPN 型的 MOS 管



NMOS

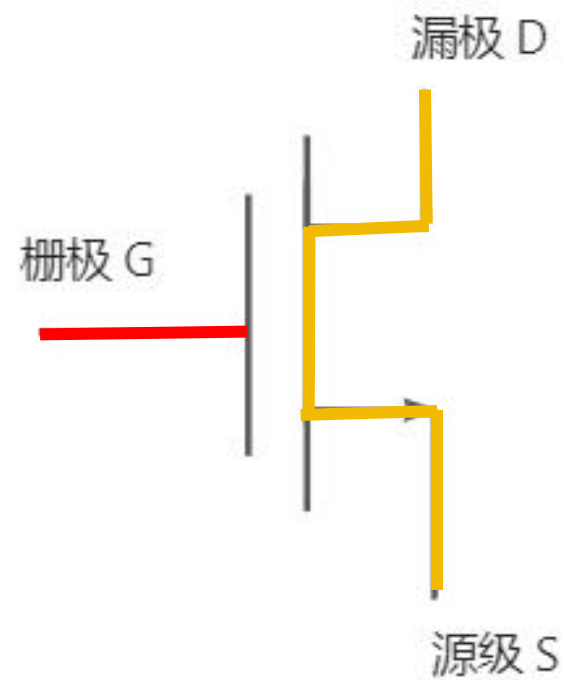
N 表示负极，英文 Negative 的首字母

P 表示正极，英文 Positive 的首字母

NMOS：电压控制的开关

当给栅极施加电压：

当电压高于某阈值电压时，这个开关可以导通



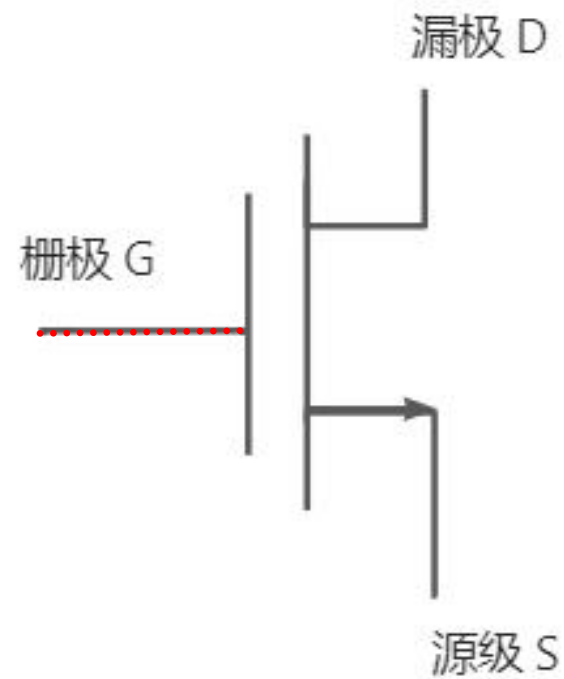
NMOS

NMOS：电压控制的开关

当给栅极施加电压：

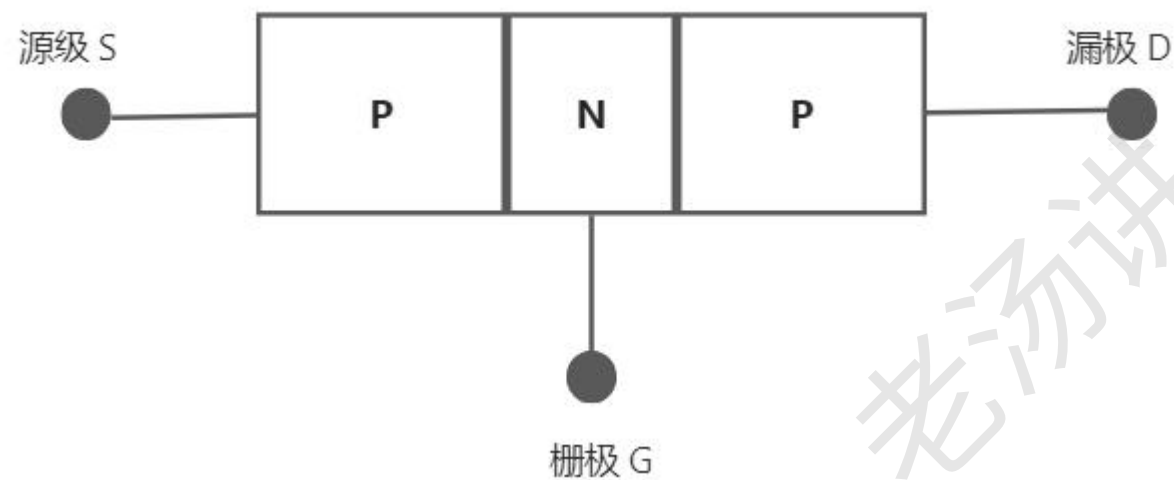
当电压**高于**某阈值电压时，这个开关**可以**导通

当电压**低于**某阈值电压时，这个开关**就不**导通

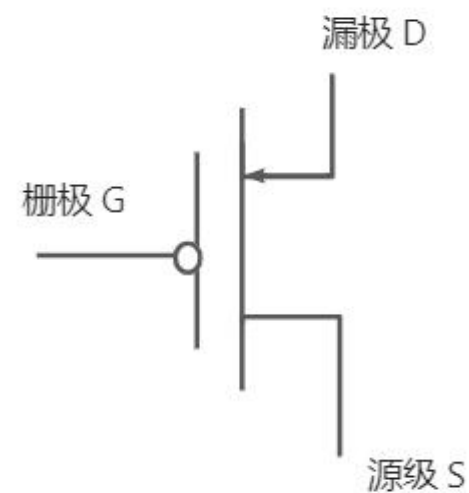


NMOS

晶体管（场效应管 - MOS 管）



PNP 型的 MOS 管



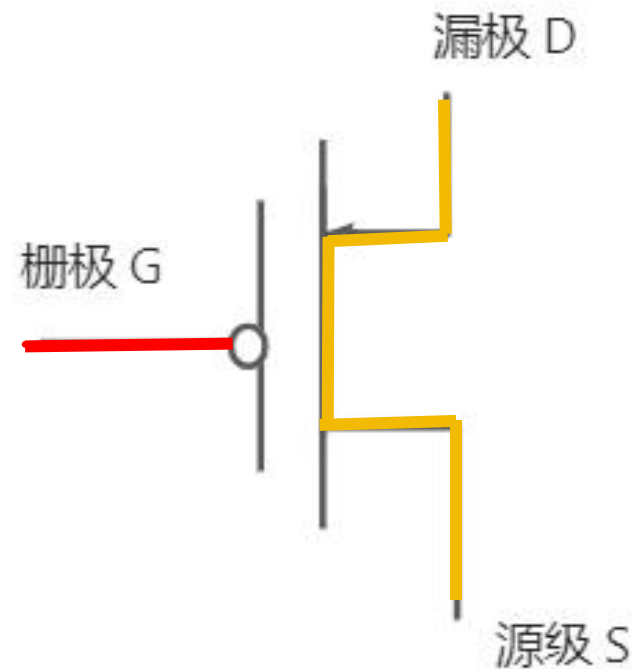
PMOS

PMOS: 电压控制的开关

当给栅极施加电压:

当电压**高于**某阈值电压时, 这个开关**就不**导通

当电压**低于**某阈值电压时, 这个开关**可以**导通



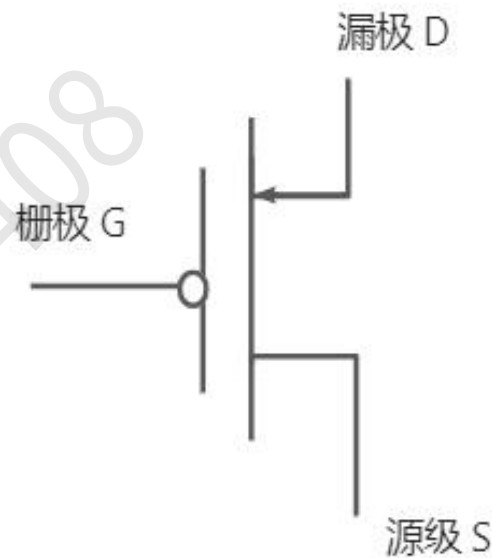
PMOS

NMOS vs PMOS

高于某阈值的电压，统称为高压；
低于某阈值的电压，统称为低压



NMOS

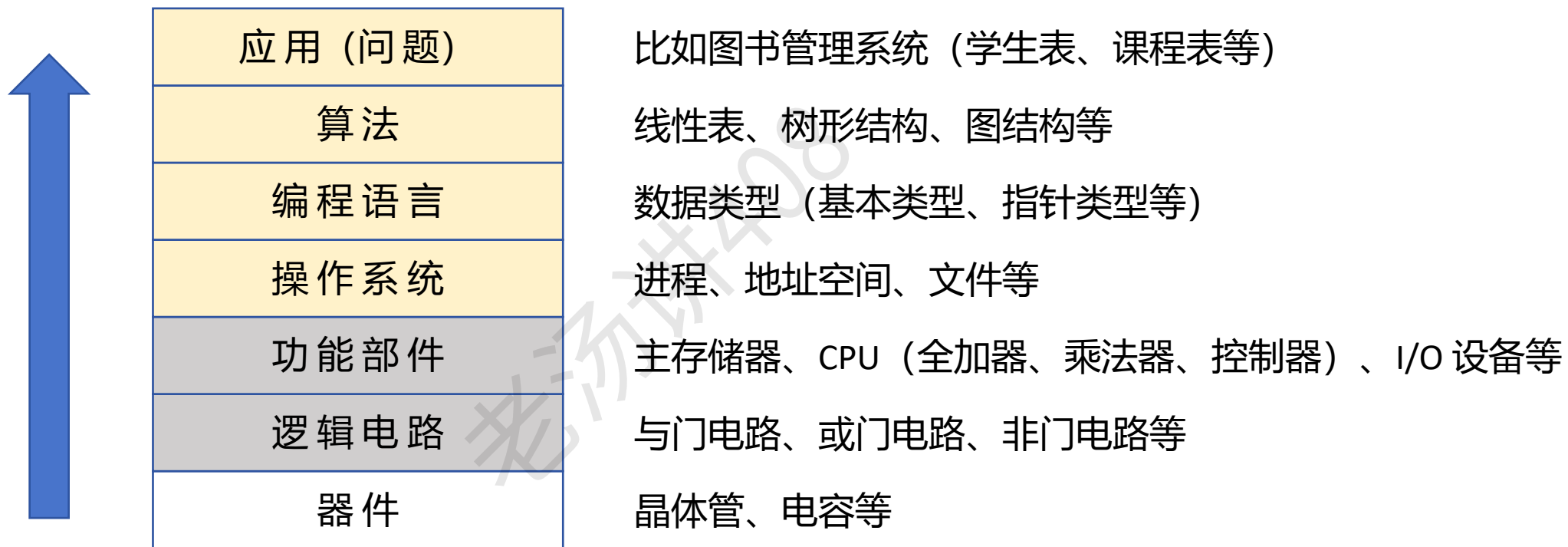


PMOS

NMOS: 高压导通，低压不导通

PMOS: 高压不导通，低压导通

计算机系统的层次结构（抽象分层思想）



中间的每一层的功能都是利用下面一层提供的基本服务来完成了，每一层又为上一层提供更强大的服务

逻辑计算：非、或、与

低压



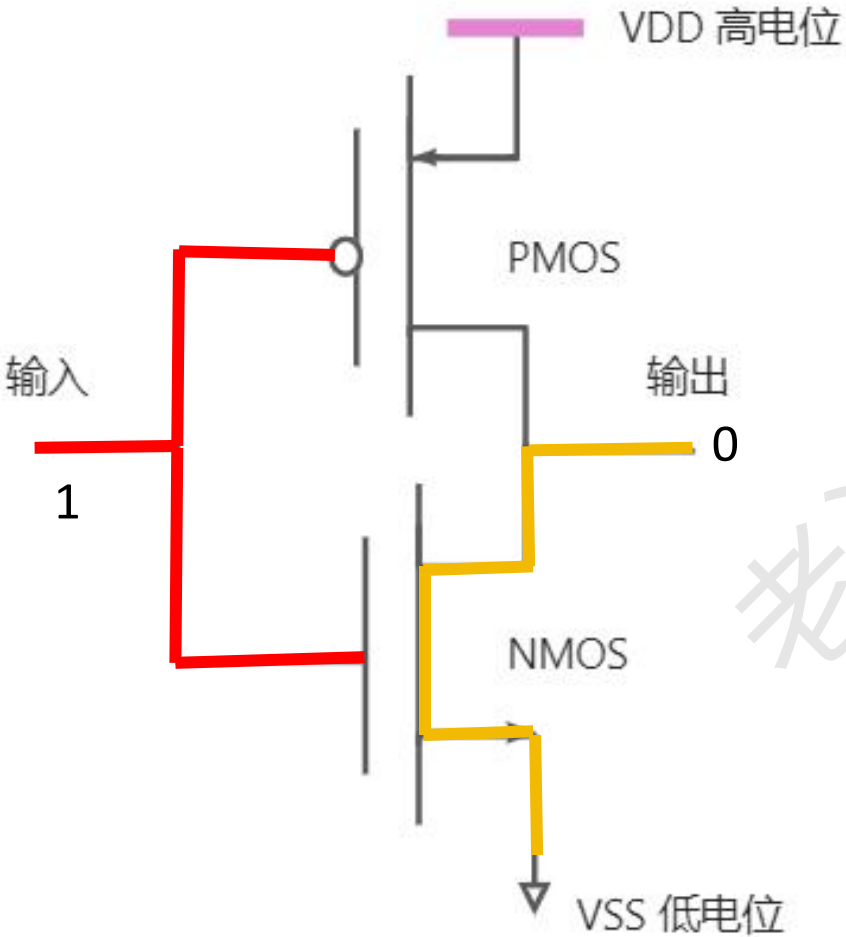
0

高压



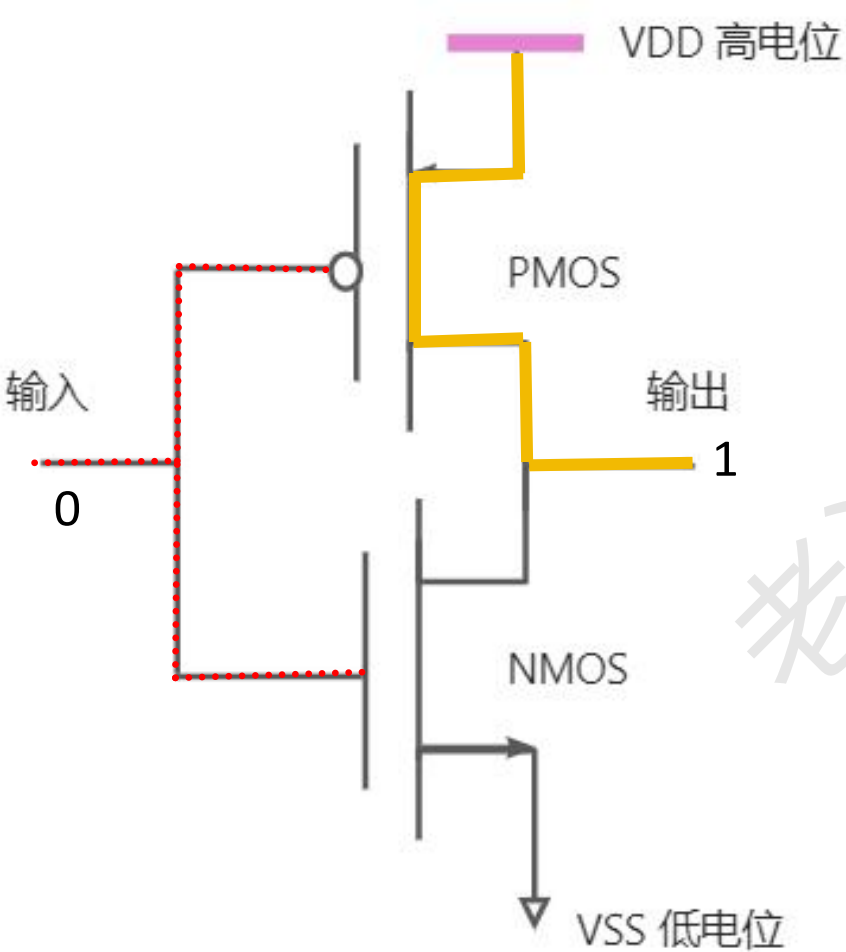
1

电路实现逻辑非



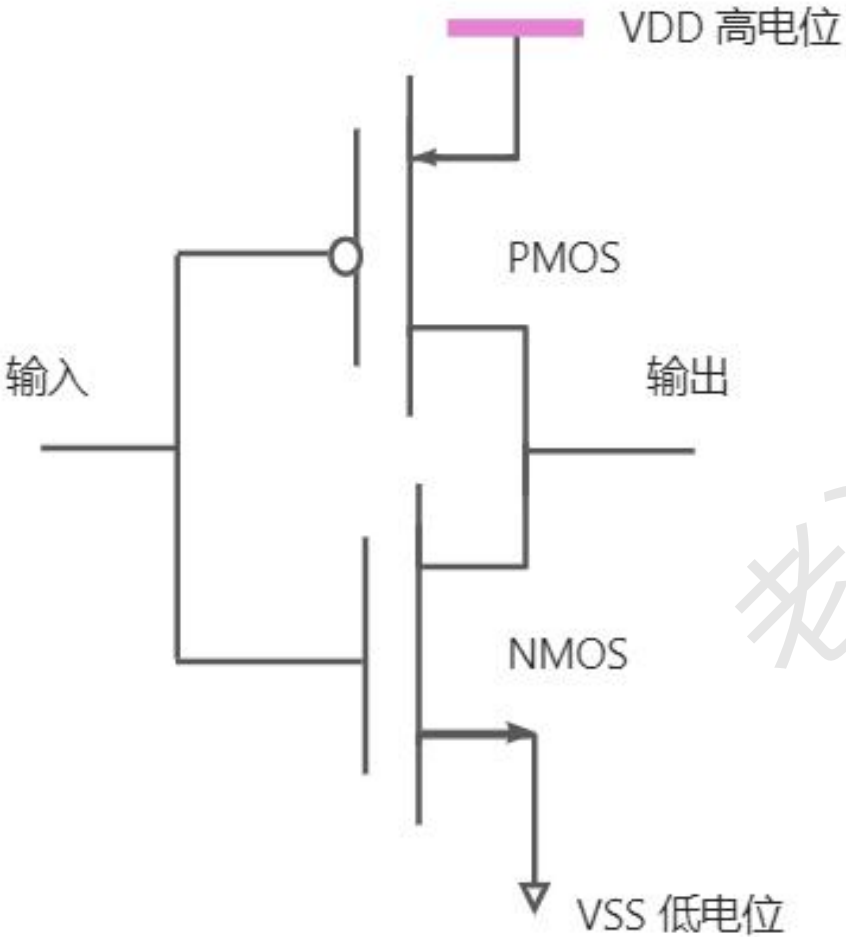
输入	输出
1	0

电路实现逻辑非



输入	输出
1	0
0	1

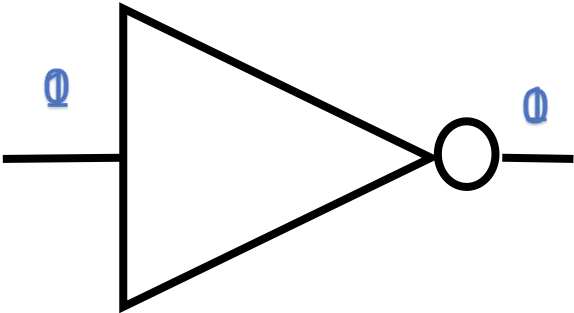
电路实现逻辑非



抽象

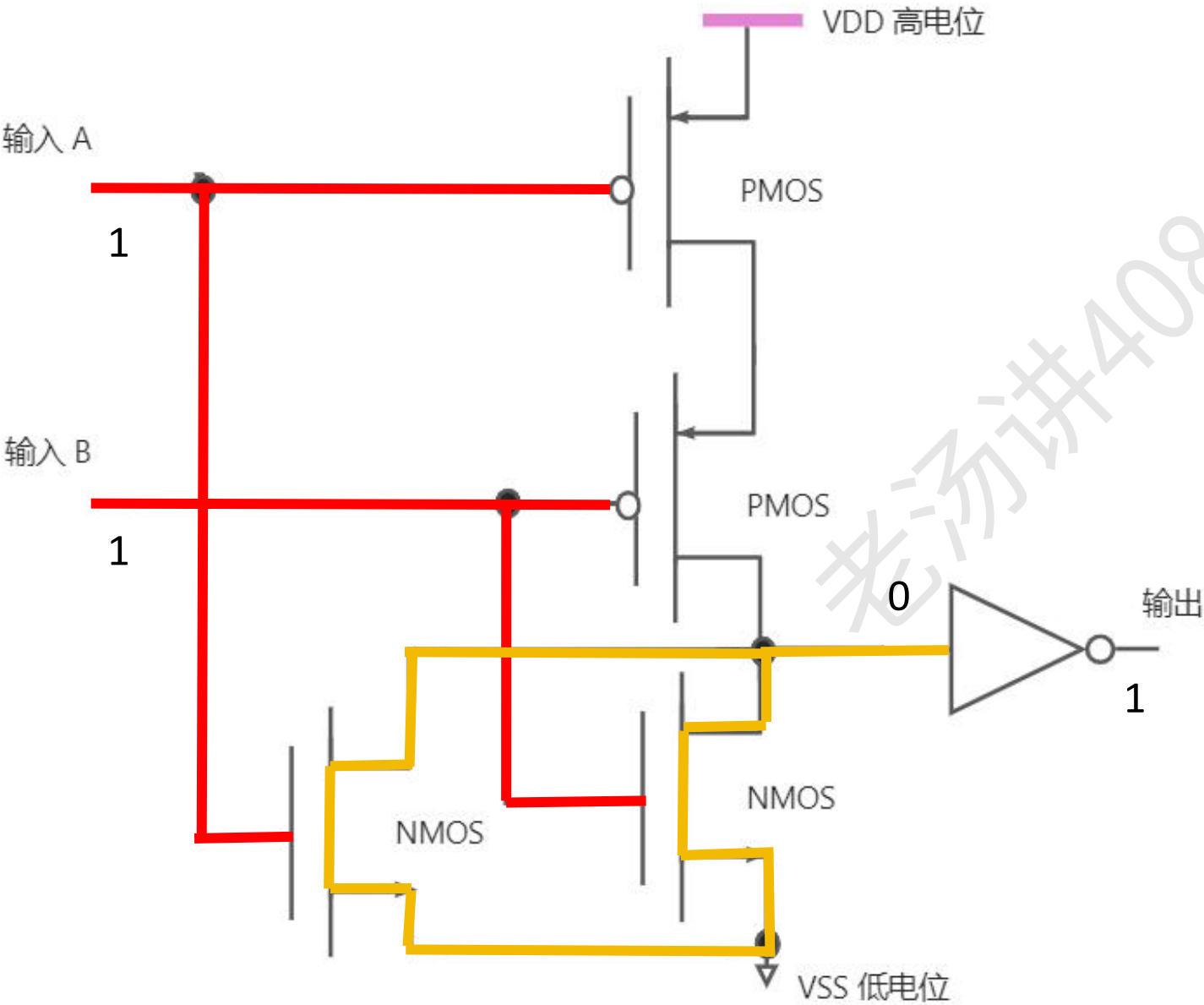


输入	输出
1	0
0	1



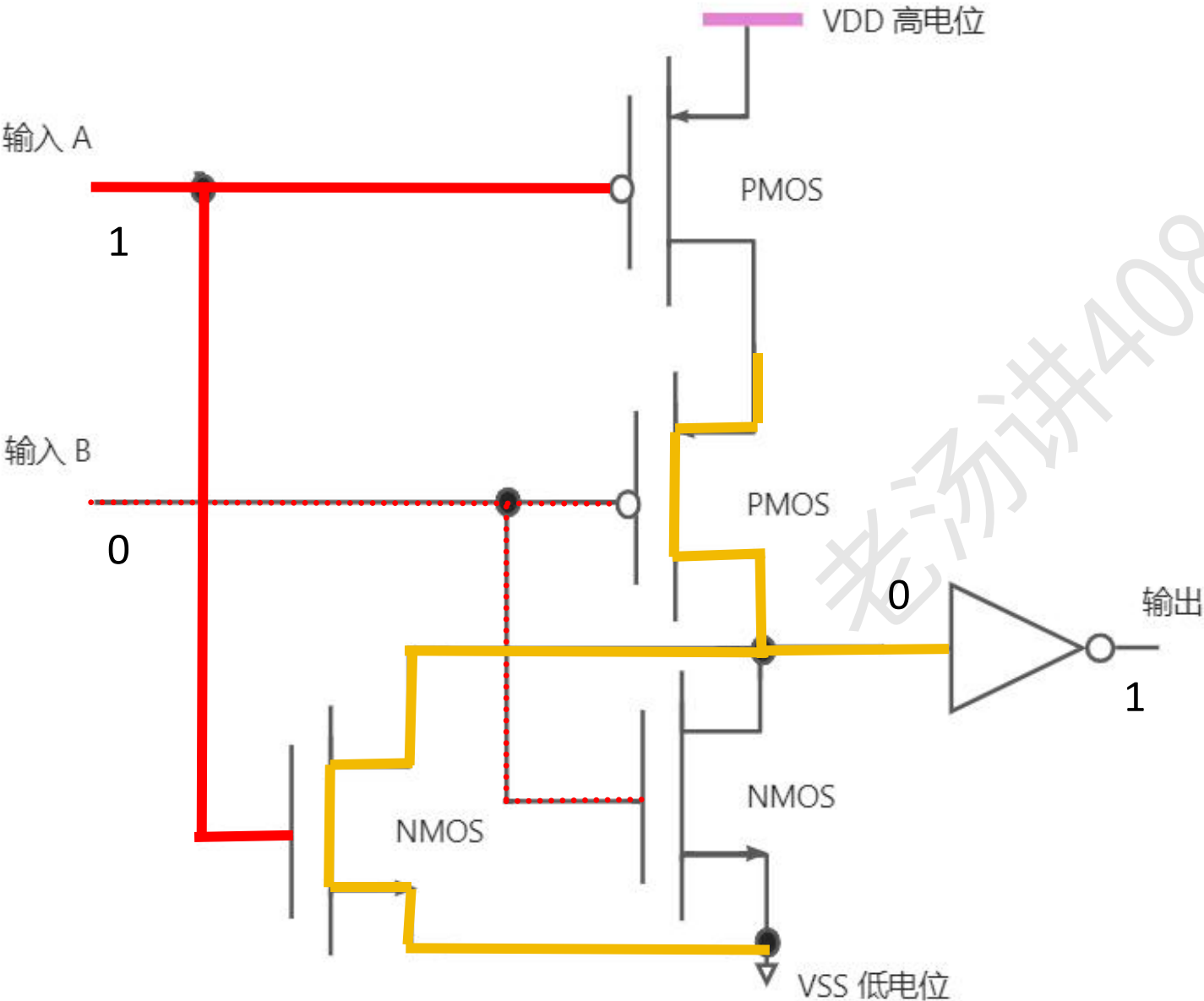
逻辑非门

电路实现逻辑或



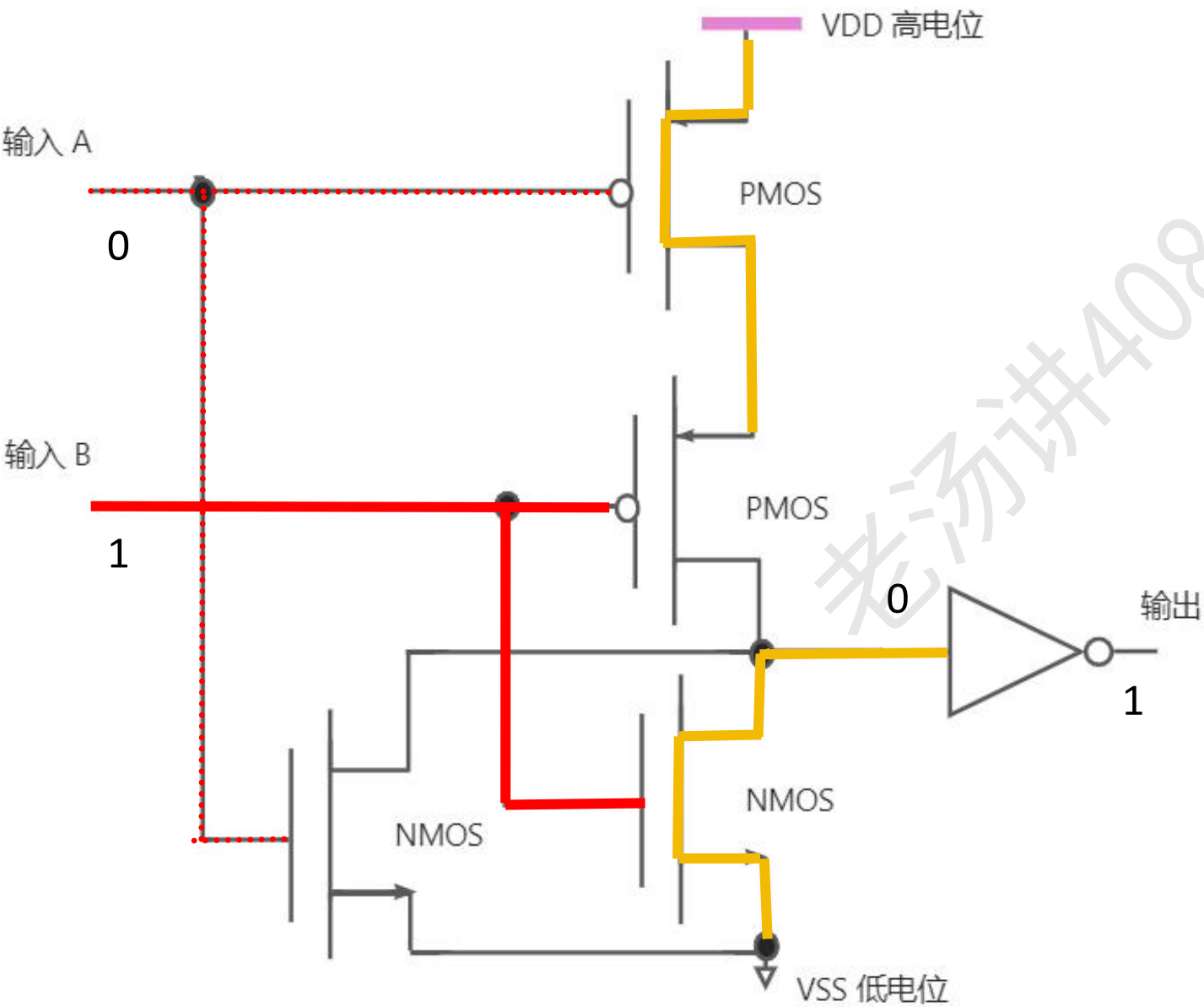
输入 A	输入 B	输出
1	1	1

电路实现逻辑或



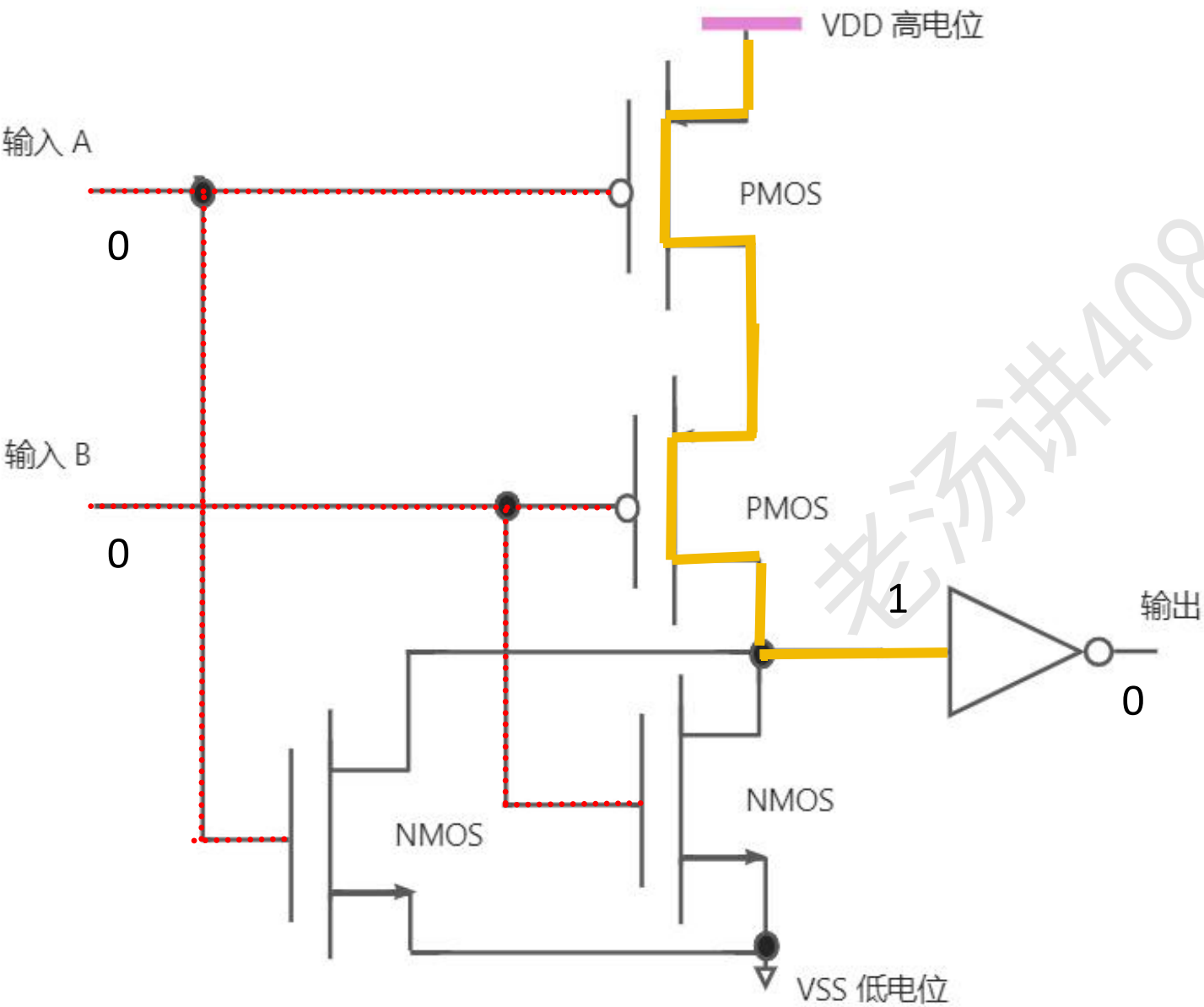
输入 A	输入 B	输出
1	1	1
1	0	1

电路实现逻辑或



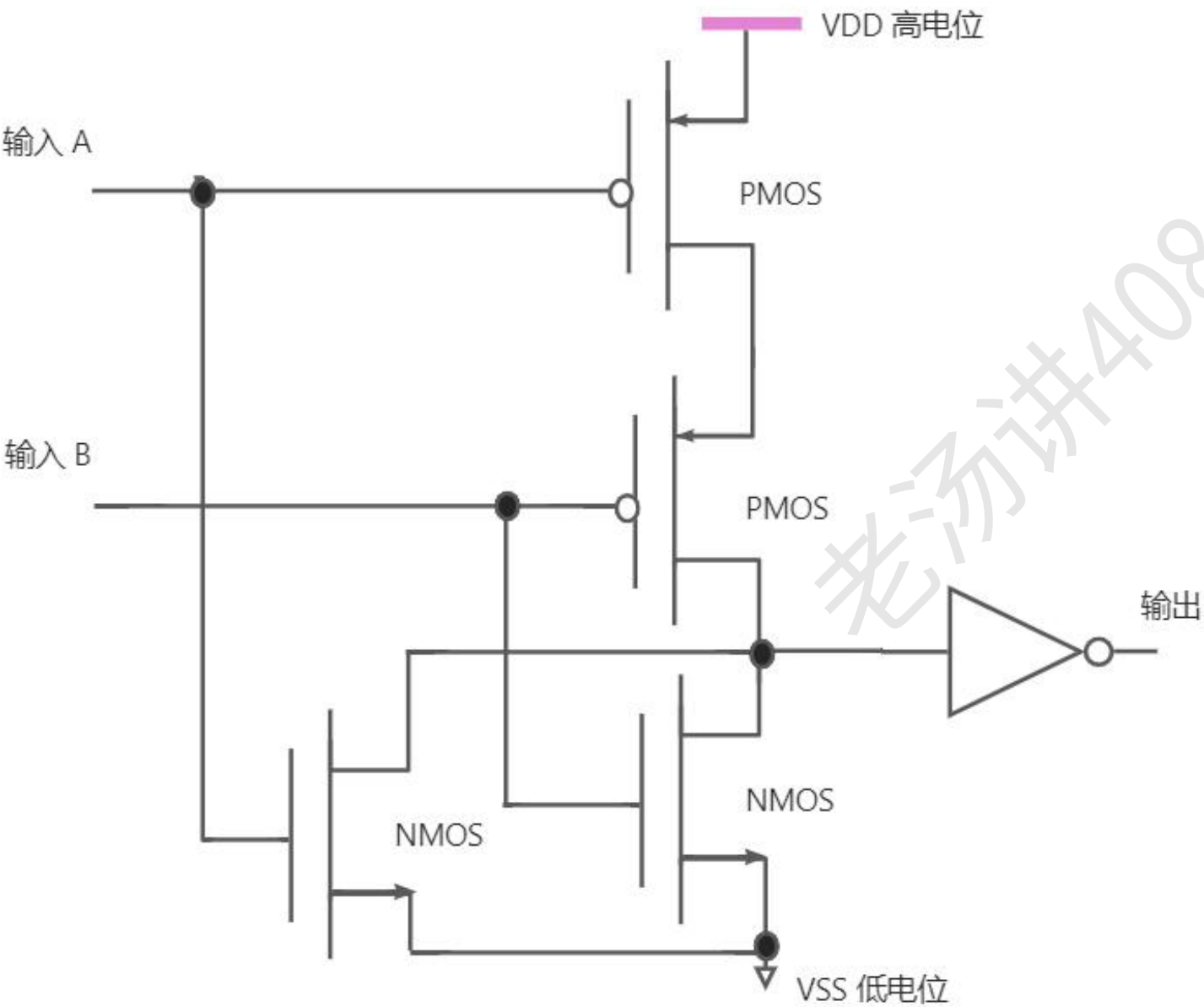
输入 A	输入 B	输出
1	1	1
1	0	1
0	1	1

电路实现逻辑或



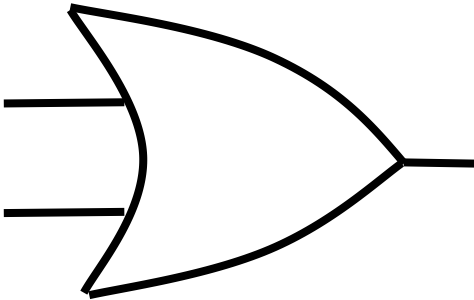
输入 A	输入 B	输出
1	1	1
1	0	1
0	1	1
0	0	0

电路实现逻辑或



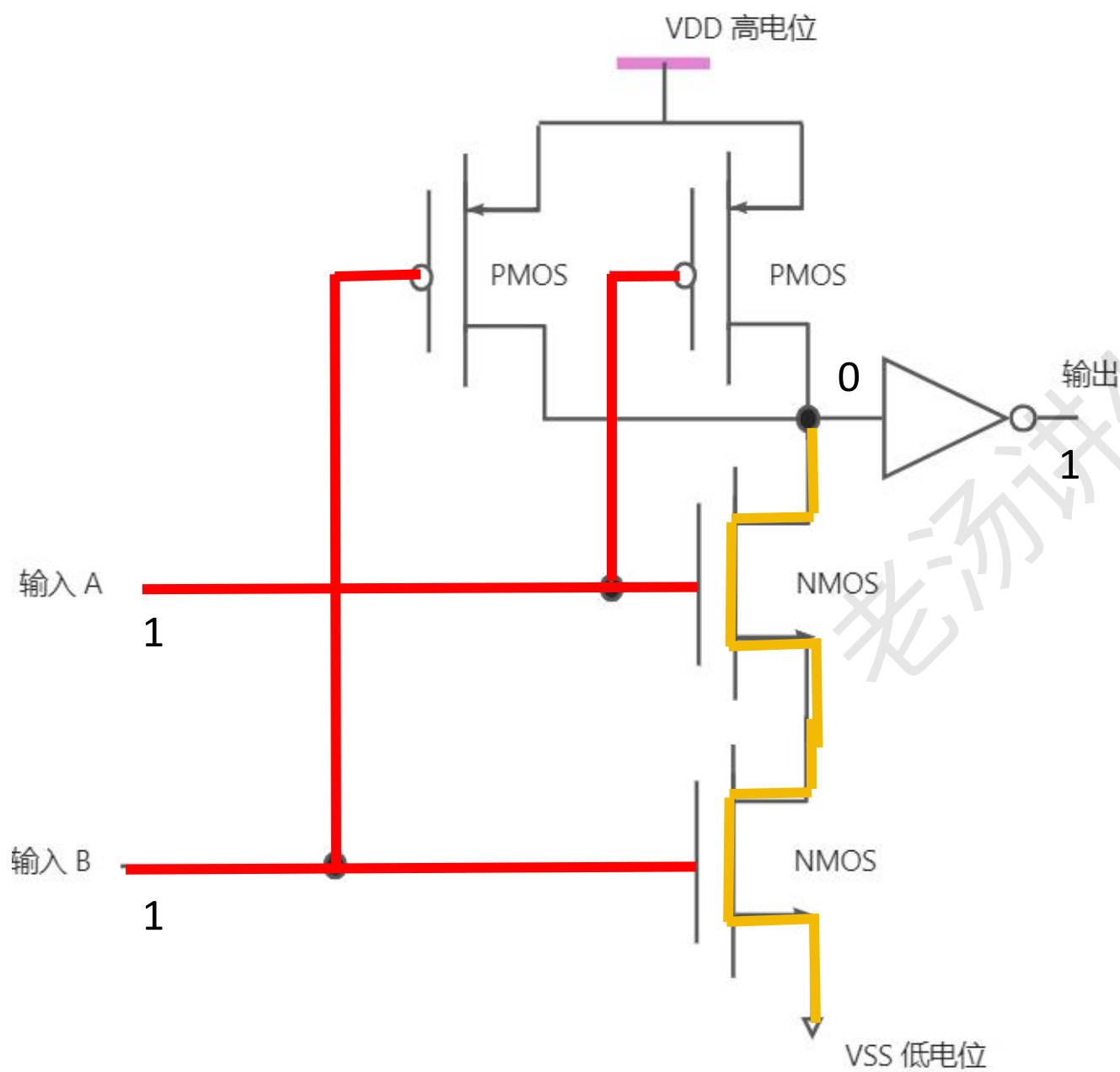
输入 A	输入 B	输出
1	1	1
1	0	1
0	1	1
0	0	0

抽象



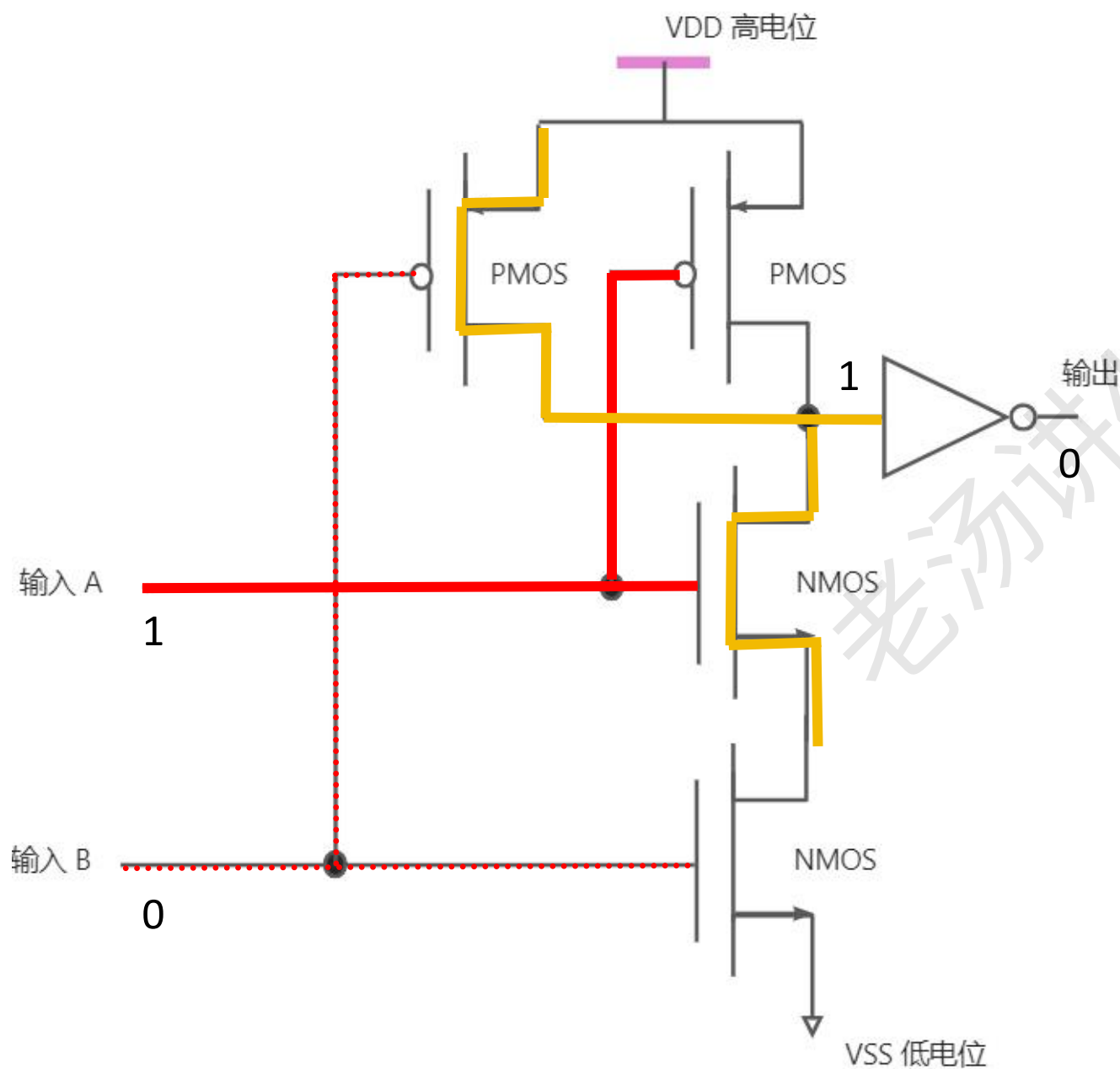
逻辑或门

电路实现逻辑与



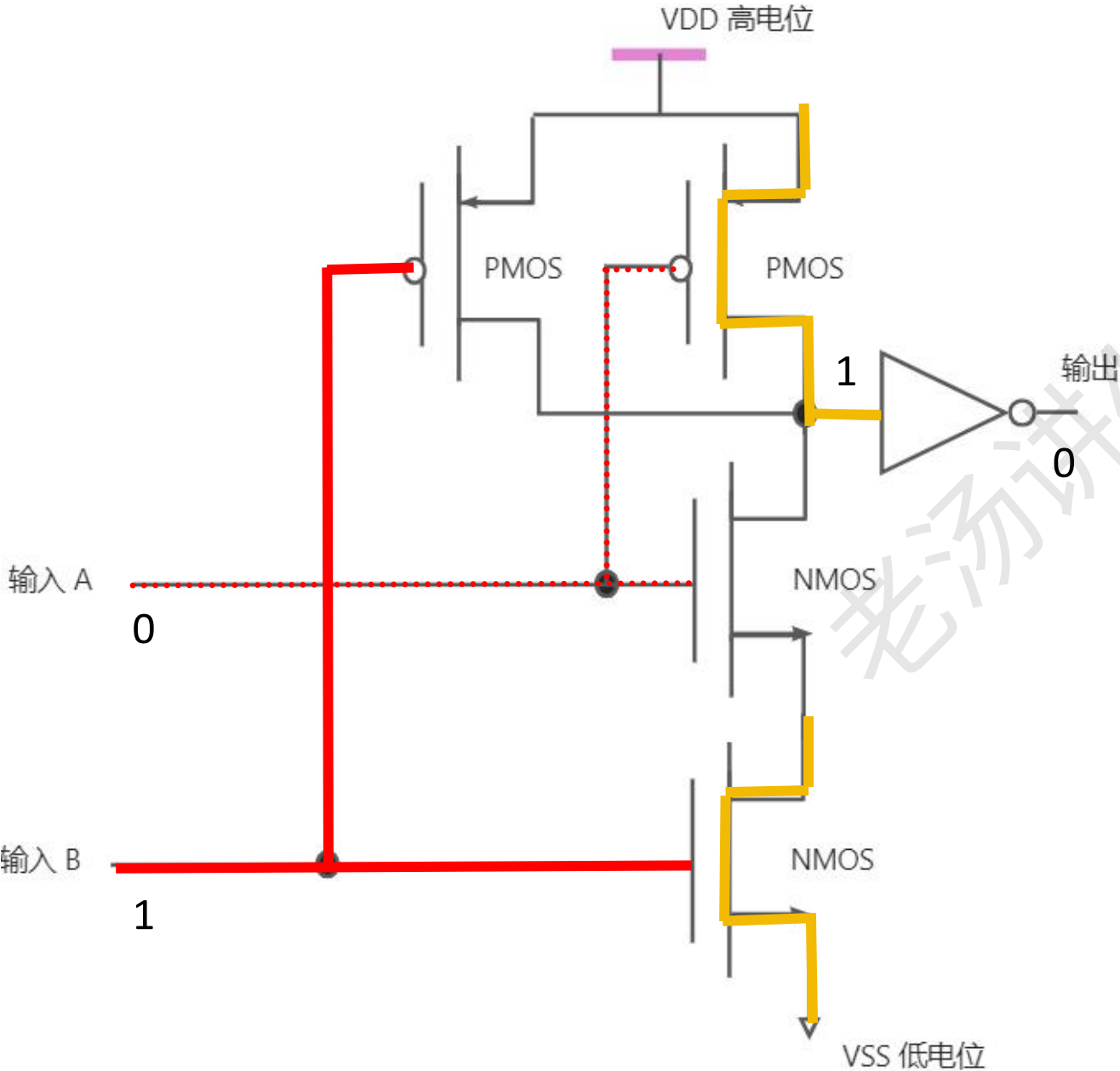
输入 A	输入 B	输出
1	1	1

电路实现逻辑与



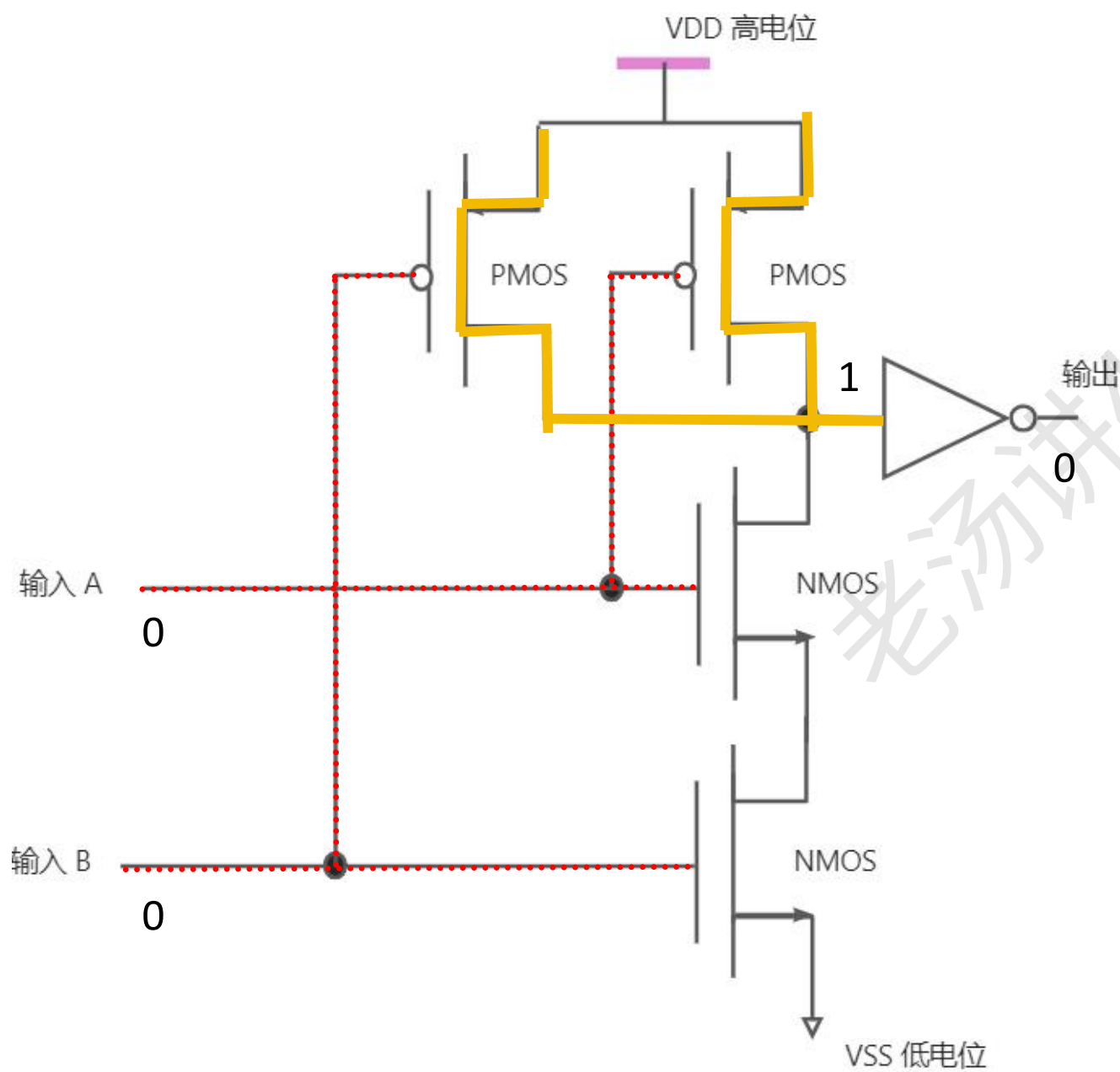
输入 A	输入 B	输出
1	1	1
1	0	0
0	1	0
0	0	0

电路实现逻辑与



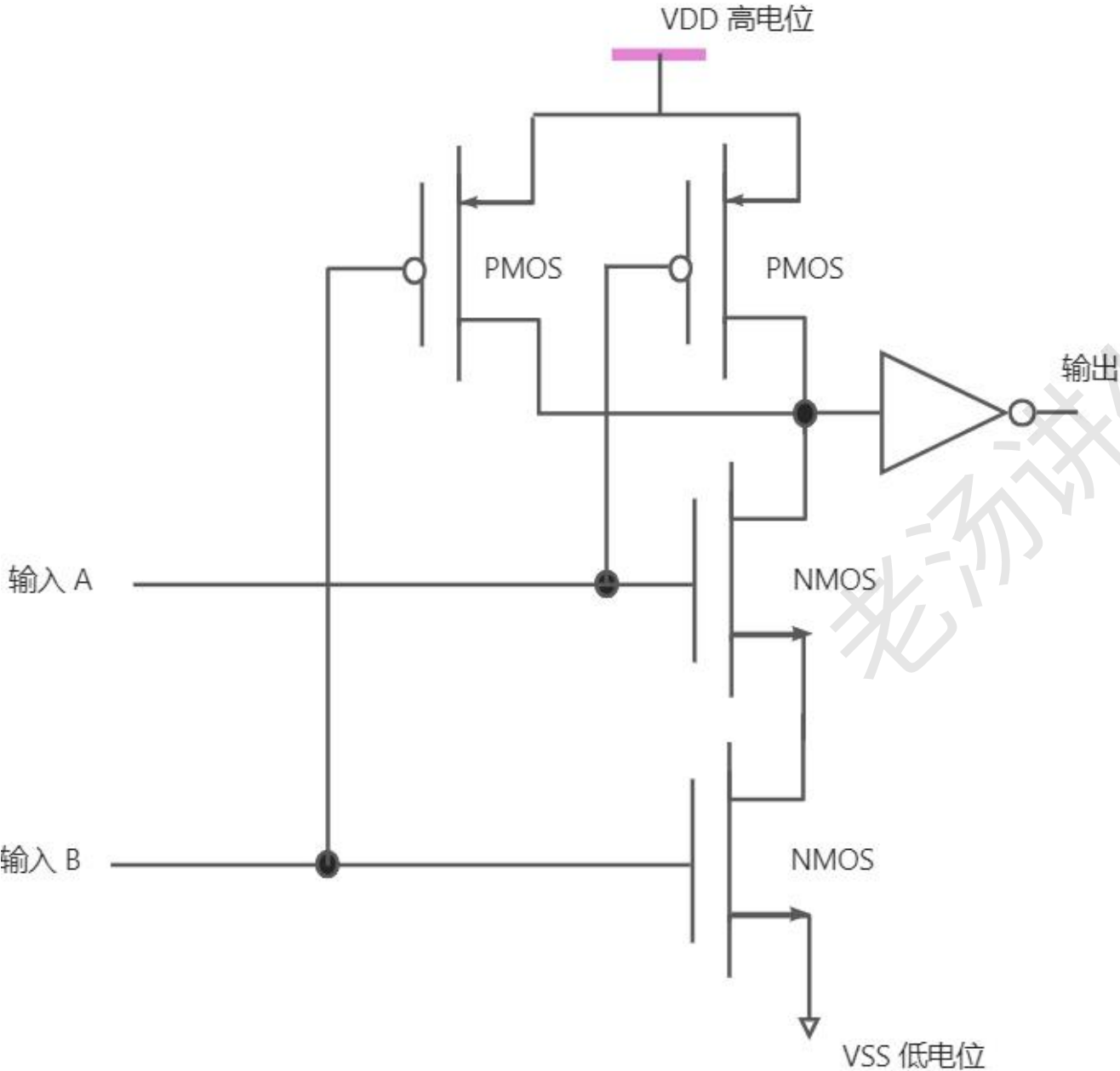
输入 A	输入 B	输出
1	1	1
1	0	0
0	1	0

电路实现逻辑与



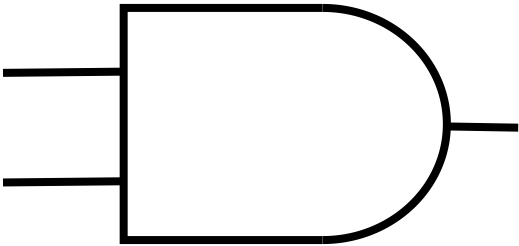
输入 A	输入 B	输出
1	1	1
1	0	0
0	1	0
0	0	0

电路实现逻辑与



输入 A	输入 B	输出
1	1	1
1	0	0
0	1	0
0	0	0

抽象



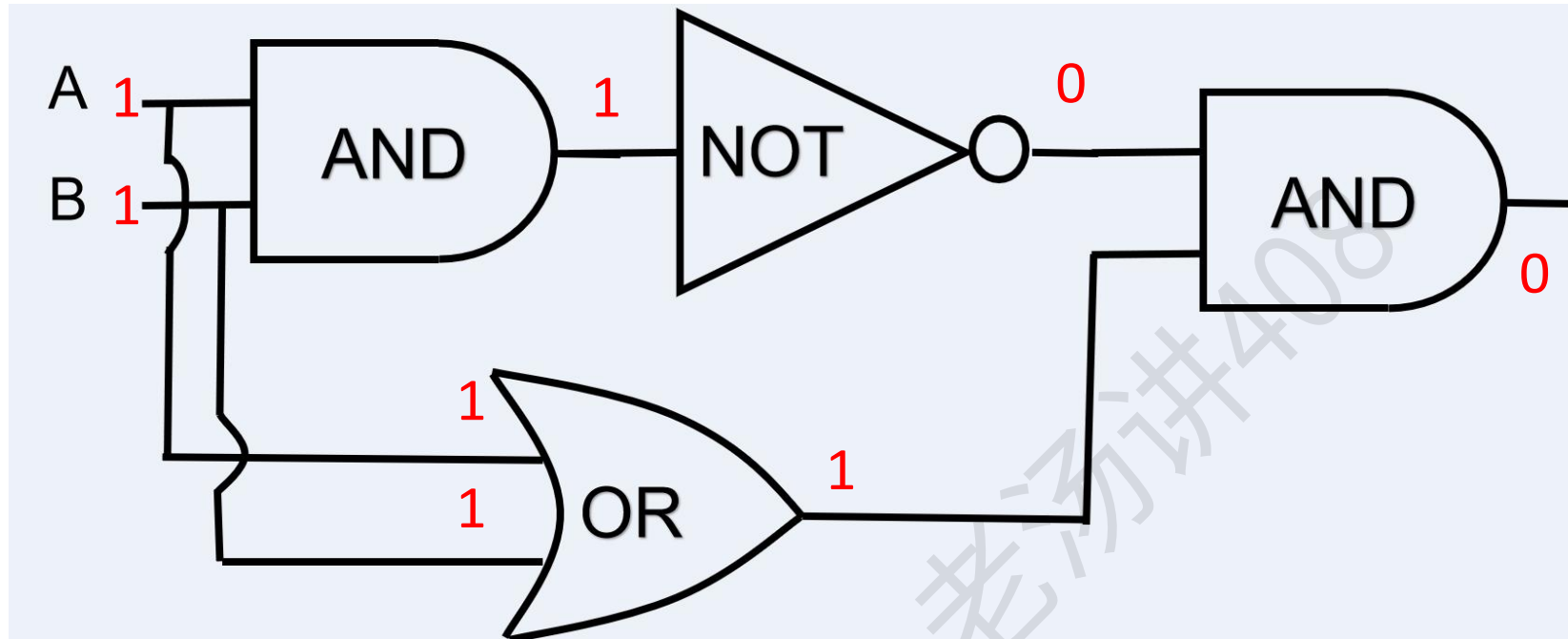
逻辑与门

电路实现：异或

输入 A	输入 B	输出
1	1	0
1	0	1
0	1	1
0	0	0

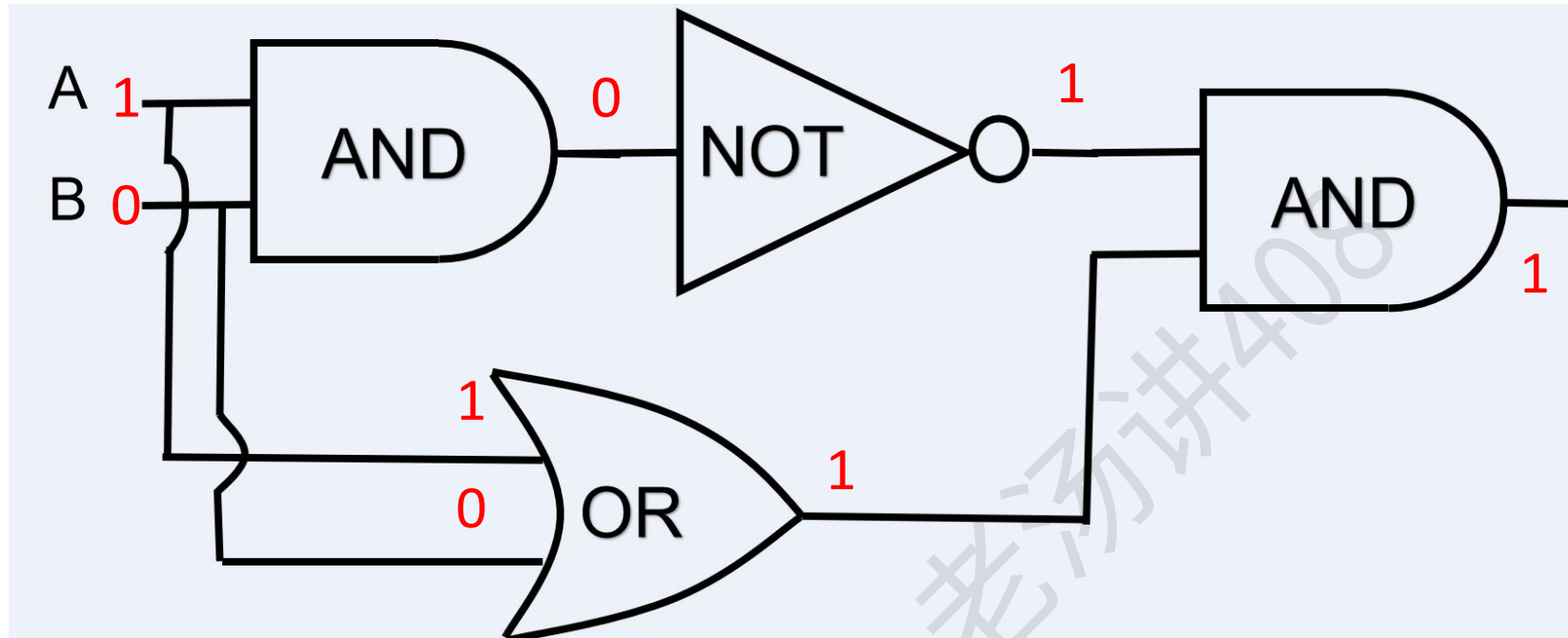
老汤讲408

电路实现：异或



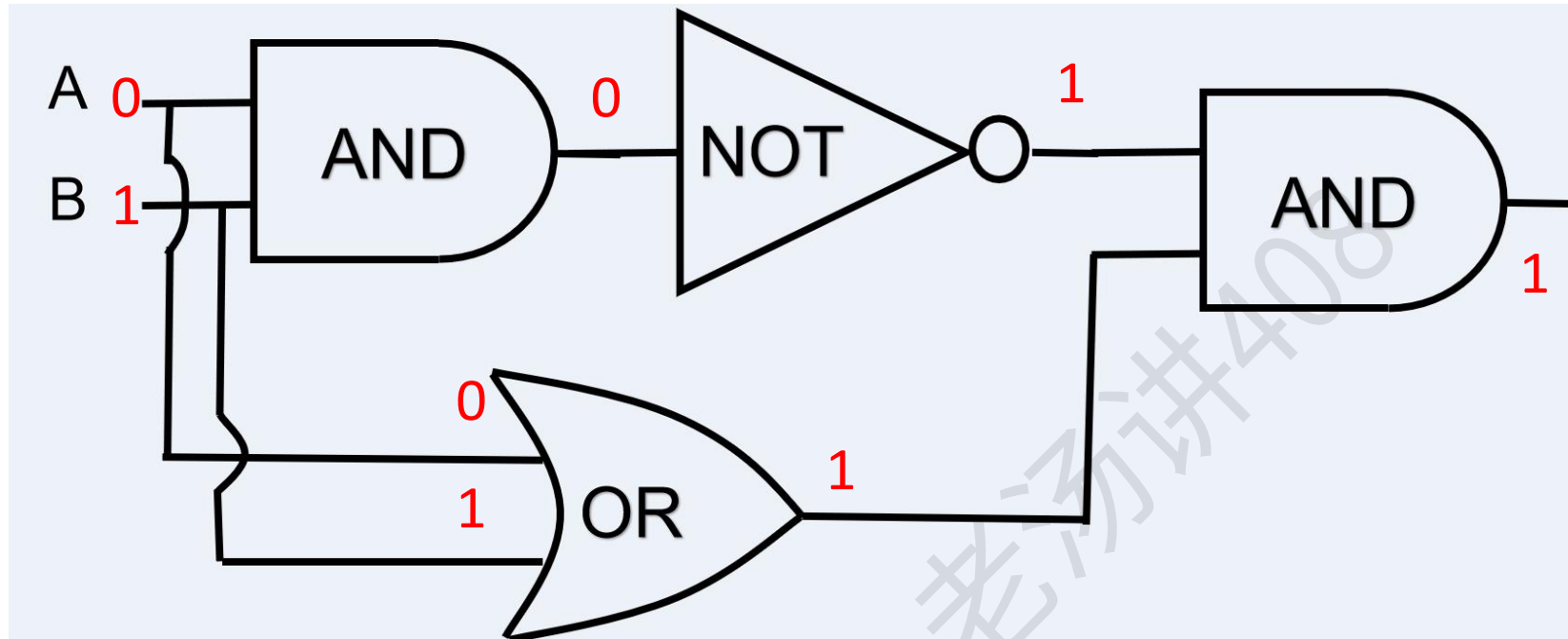
输入 A	输入 B	输出
1	1	0
1	0	1
0	1	1
0	0	0

电路实现：异或



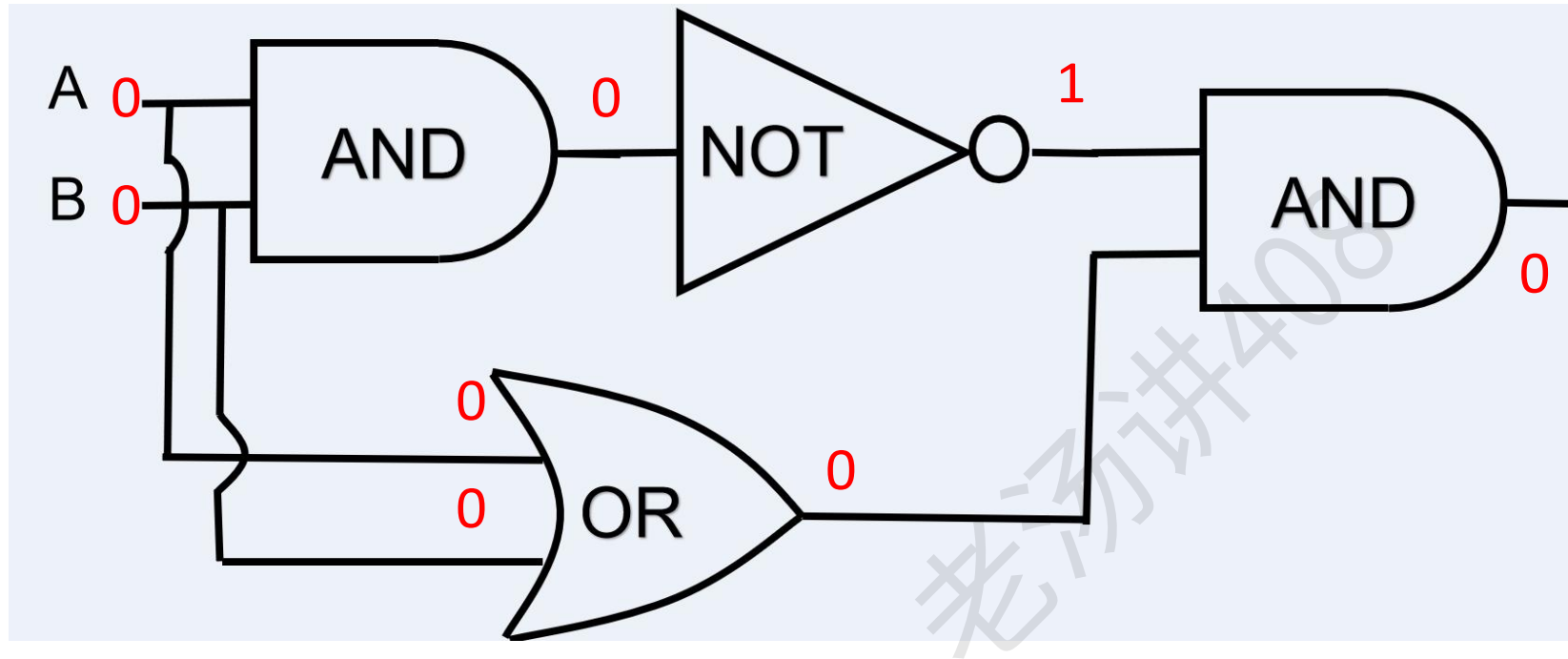
输入 A	输入 B	输出
1	1	0
1	0	1
0	1	1
0	0	0

电路实现：异或



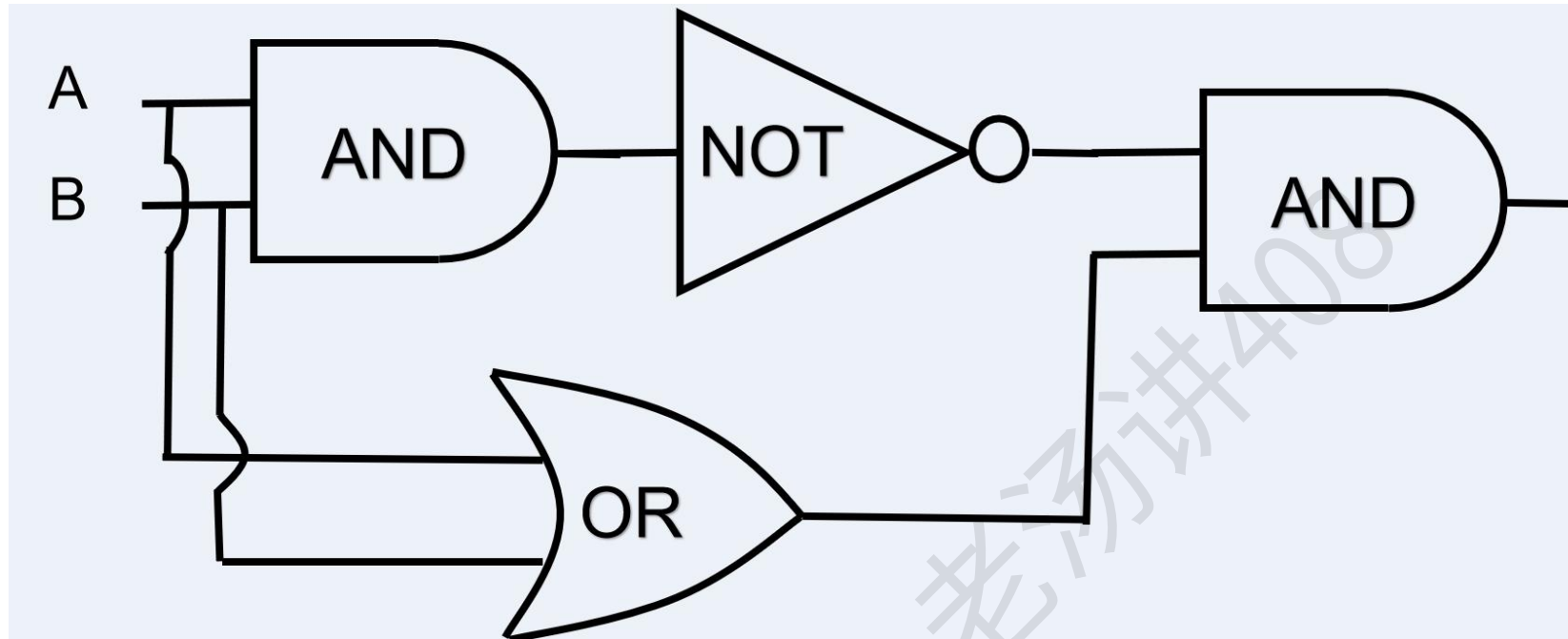
输入 A	输入 B	输出
1	1	0
1	0	1
0	1	1
0	0	0

电路实现：异或

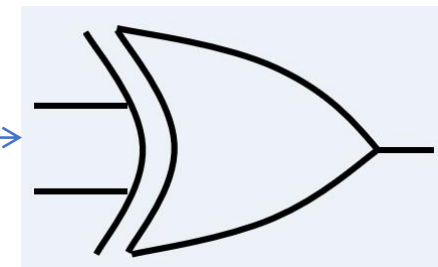


输入 A	输入 B	输出
1	1	0
1	0	1
0	1	1
0	0	0

电路实现：异或

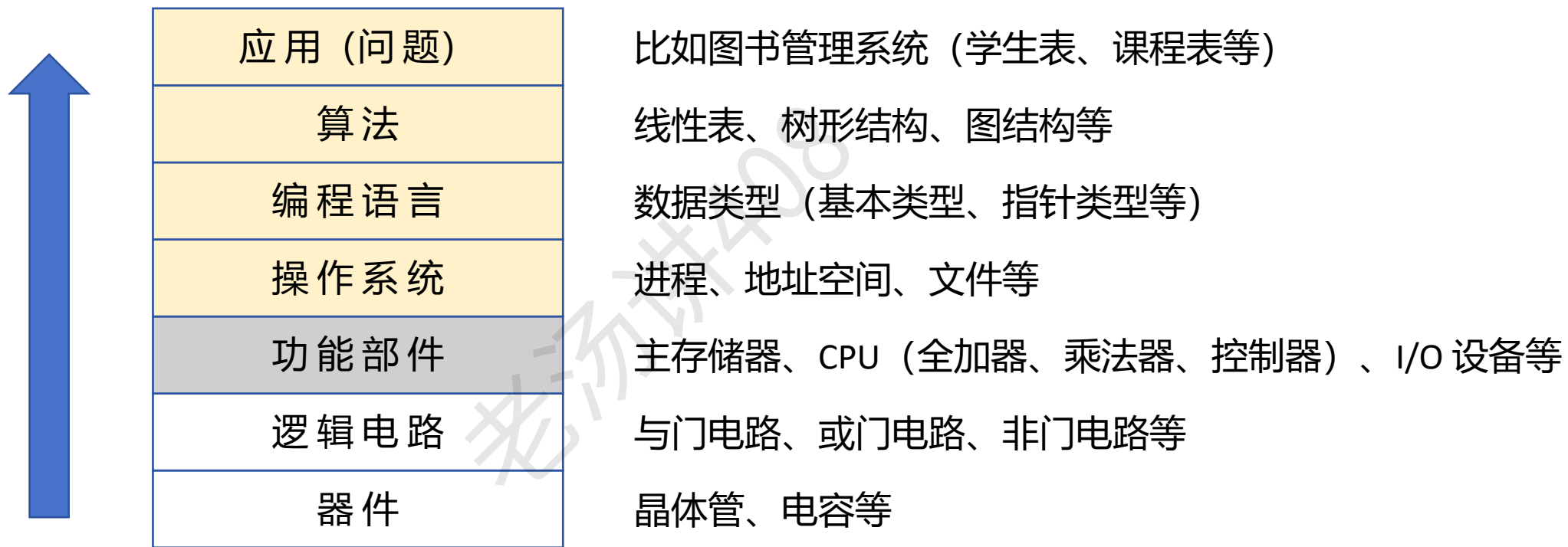


输入 A	输入 B	输出
1	1	0
1	0	1
0	1	1
0	0	0



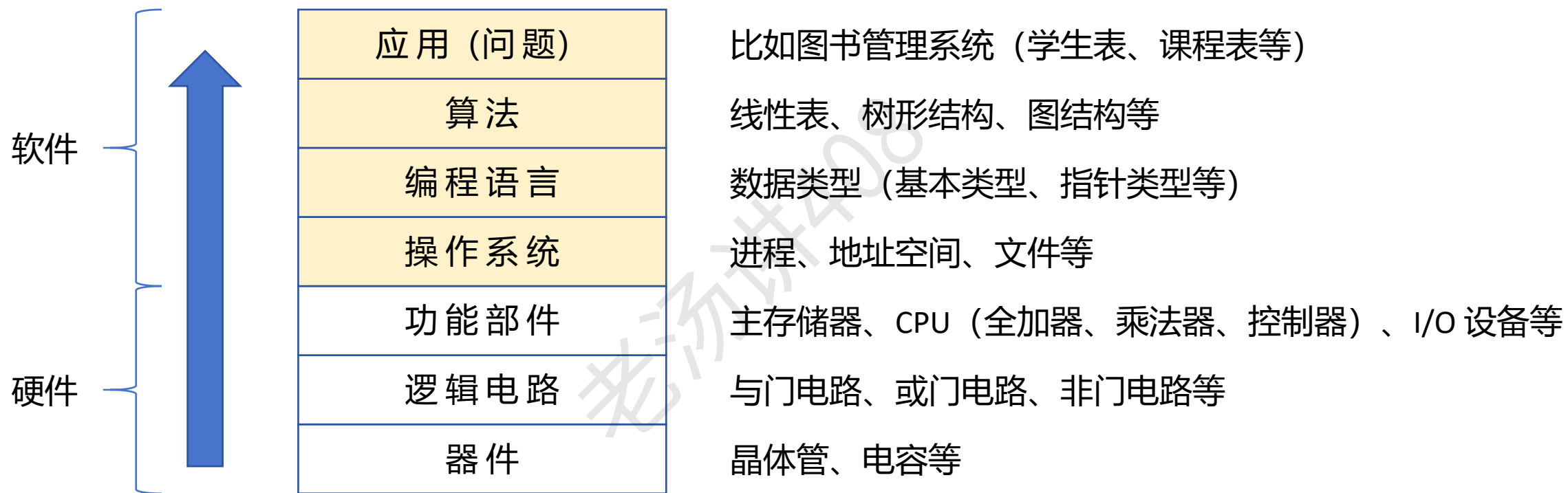
逻辑异或门

计算机系统的层次结构（抽象分层思想）

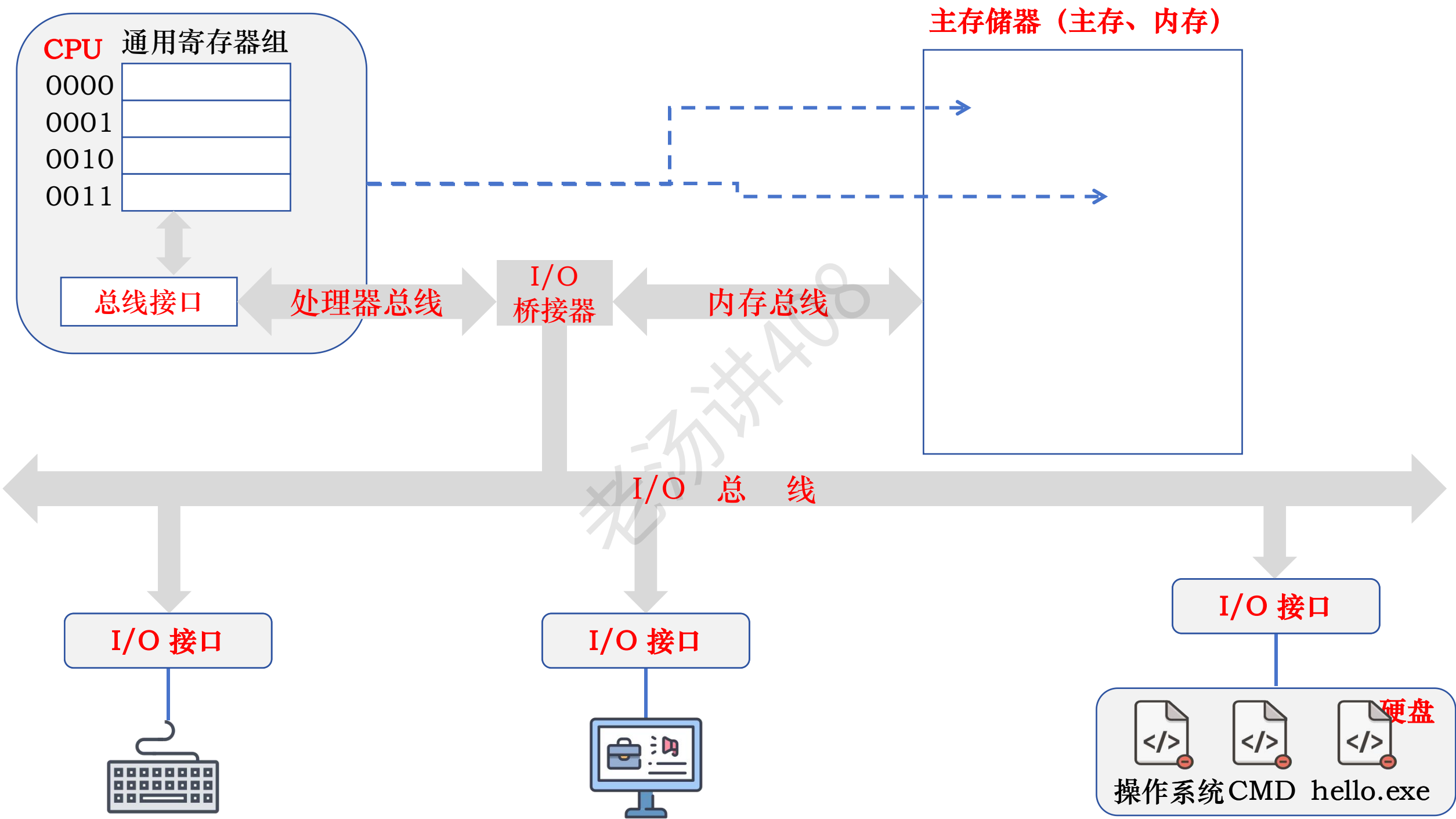


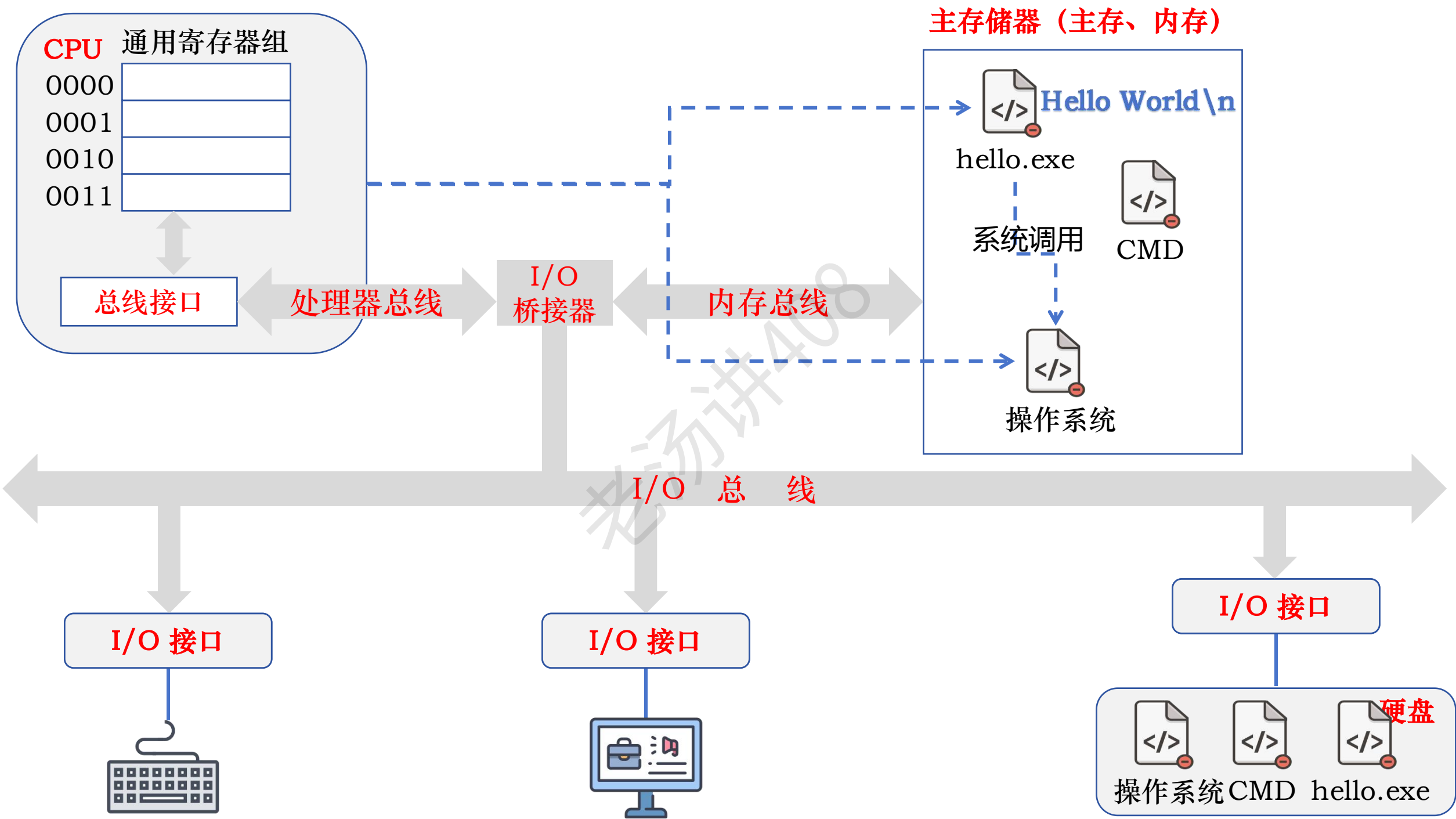
中间的每一层的功能都是利用下面一层提供的基本服务来完成了，每一层又为上一层提供更强大的服务

计算机系统的层次结构（抽象分层思想）

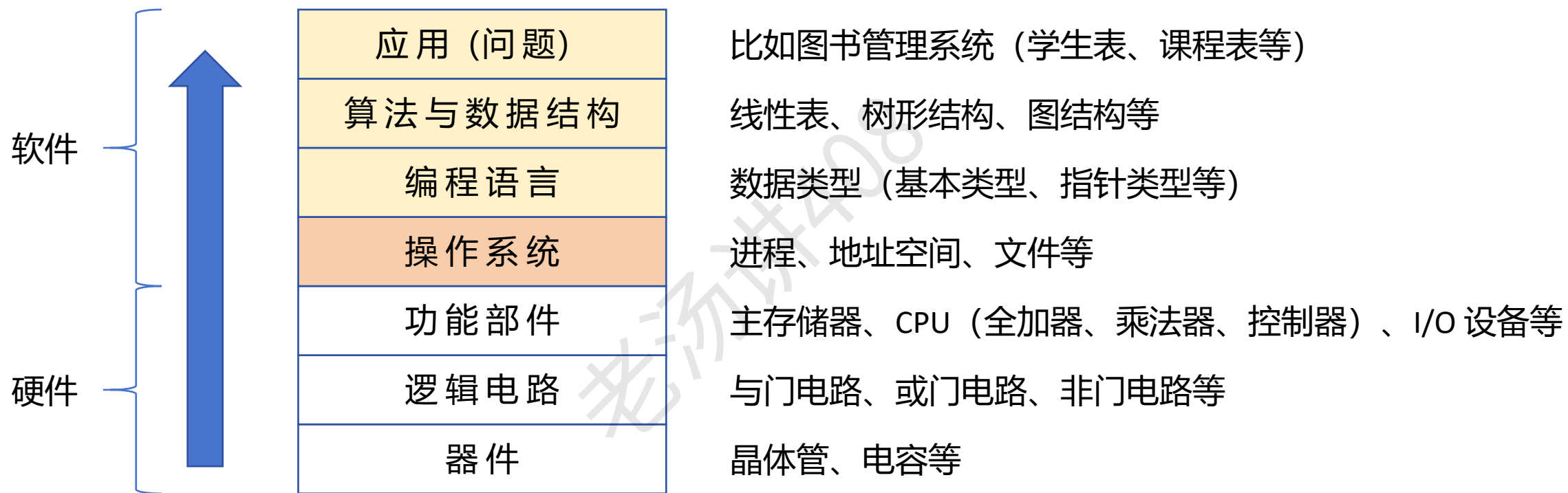


中间的每一层的功能都是利用下面一层提供的基本服务来完成了，每一层又为上一层提供更强大的服务



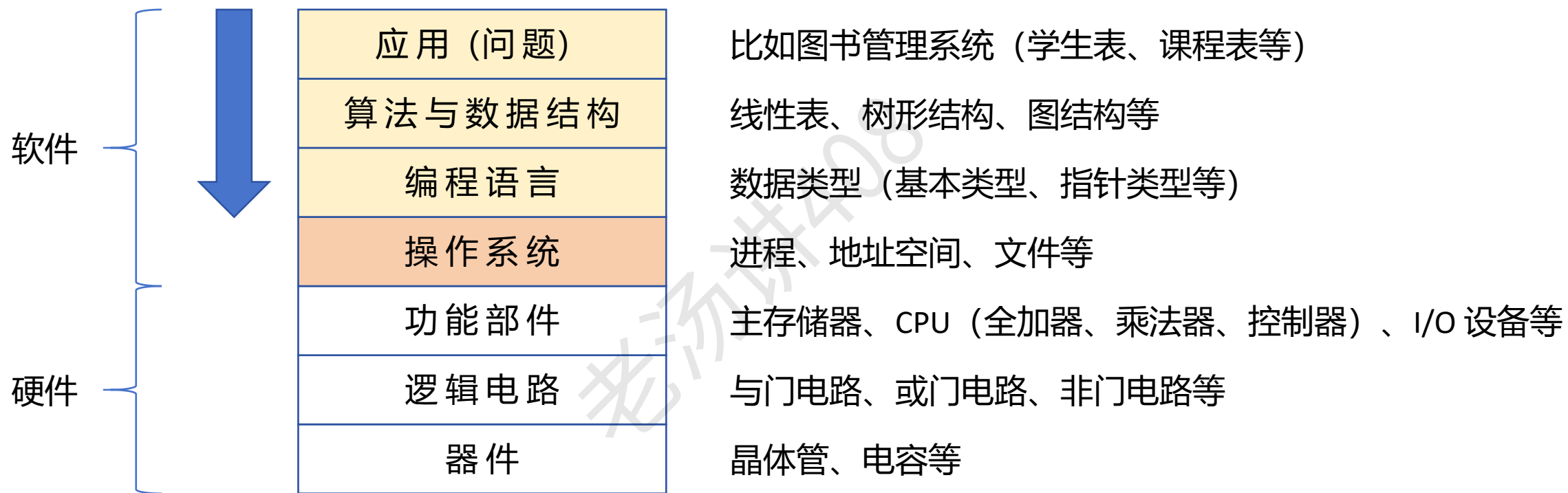


计算机系统的层次结构（抽象分层思想）



中间的每一层的功能都是利用下面一层提供的基本服务来完成了，每一层又为上一层提供更强大的服务

计算机系统的层次结构（抽象分层思想）



中间的每一层的功能都是利用下面一层提供的基本服务来完成了，每一层又为上一层提供更强大的服务

计算机系统的层次结构 - 两个体系结构

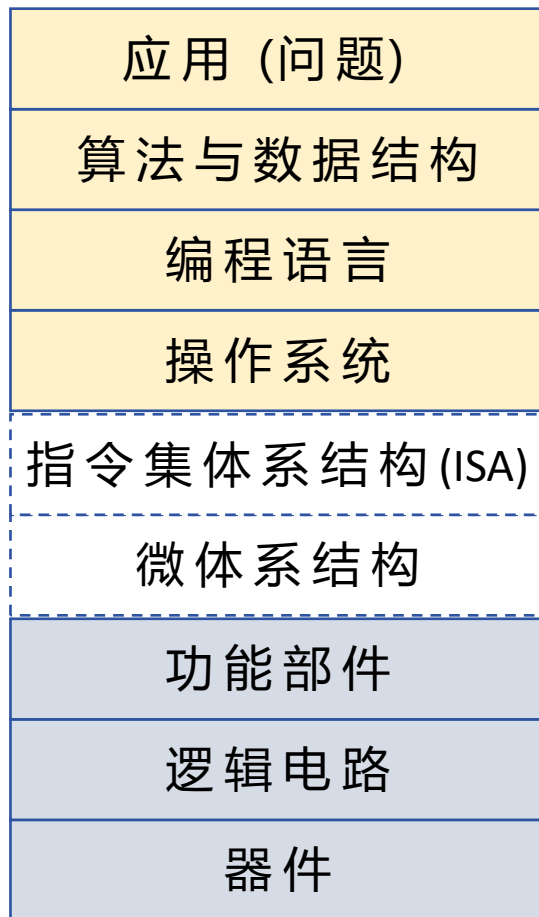
应用 (问题)
算法与数据结构
编程语言
指令集体系结构 (ISA)
微体系结构
逻辑电路
器件

指令集体系结构 (Instruction Set Architecture, ISA)

是对指令系统的一种规定或结构规范，是软件和硬件之间的接口

- ① 定义了一台计算机可以执行的所有指令的集合
- ② 规定了每条指令对应计算机执行什么操作
- ③ 指令处理的操作数存放的位置以及操作数类型

计算机系统的层次结构 - 两个体系结构

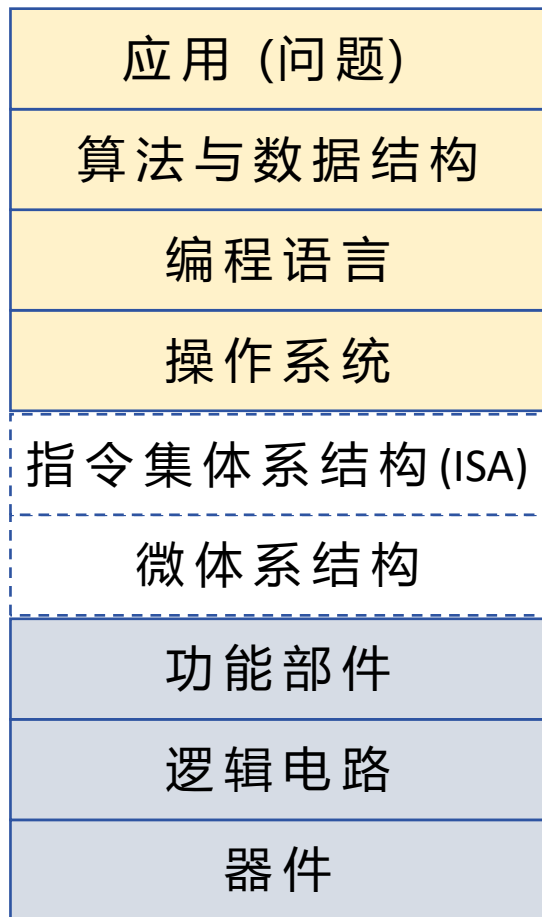


指令集体系结构 (Instruction Set Architecture, ISA)
是对指令系统的一种规定或结构规范

- ① 指令的流水线设计
- ② 加法器采用串行进位方式还是并行进位方式?
- ③ 存储器的结构 (如寄存器堆、Cache 的结构)
-

微体系结构(microarchitecture): ISA 的具体逻辑实现,
关注于处理器内部的硬件设计和实现细节

计算机系统的层次结构 - 两个体系结构

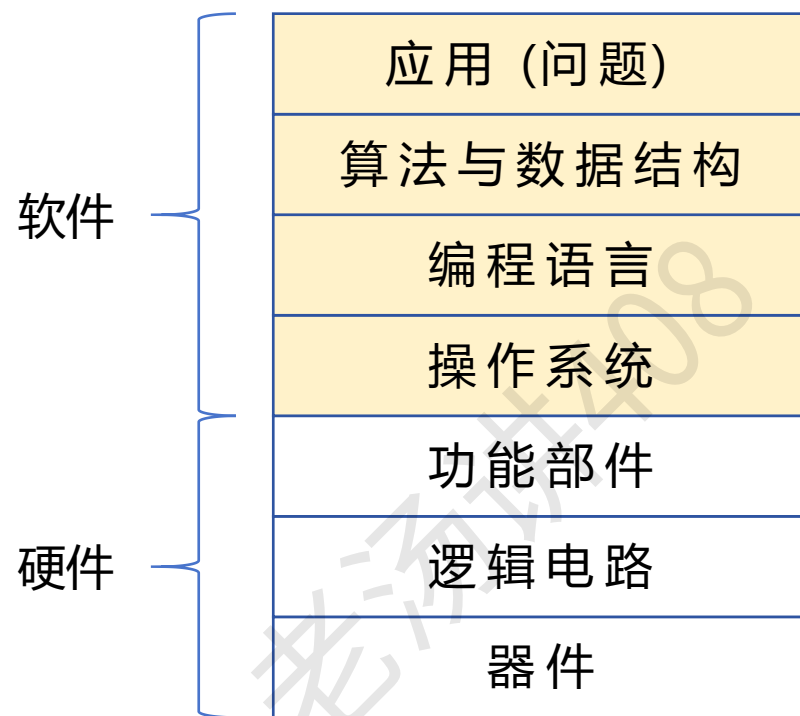


指令集体系结构 (Instruction Set Architecture, ISA)
是对指令系统的一种规定或结构规范



微体系结构(microarchitecture): ISA 的具体逻辑实现,
关注于处理器内部的硬件设计和实现细节

软件和硬件在逻辑功能上是等价的



软件能实现的功能，硬件也可以实现

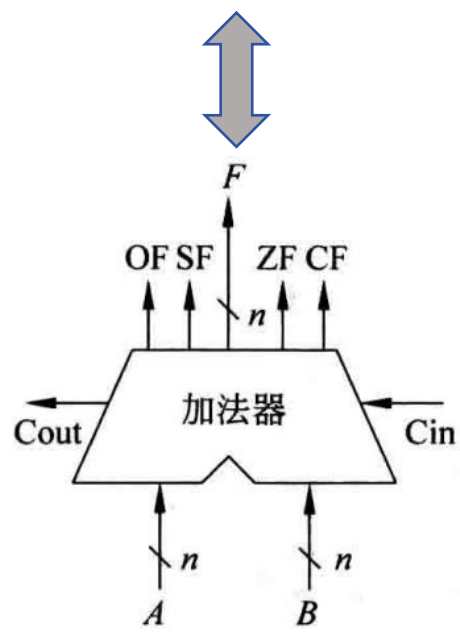
硬件能实现的功能，软件也可以实现

软件和硬件在逻辑功能上是等价的

加法指令

// 软件实现加法指令（基于逻辑异或实现）

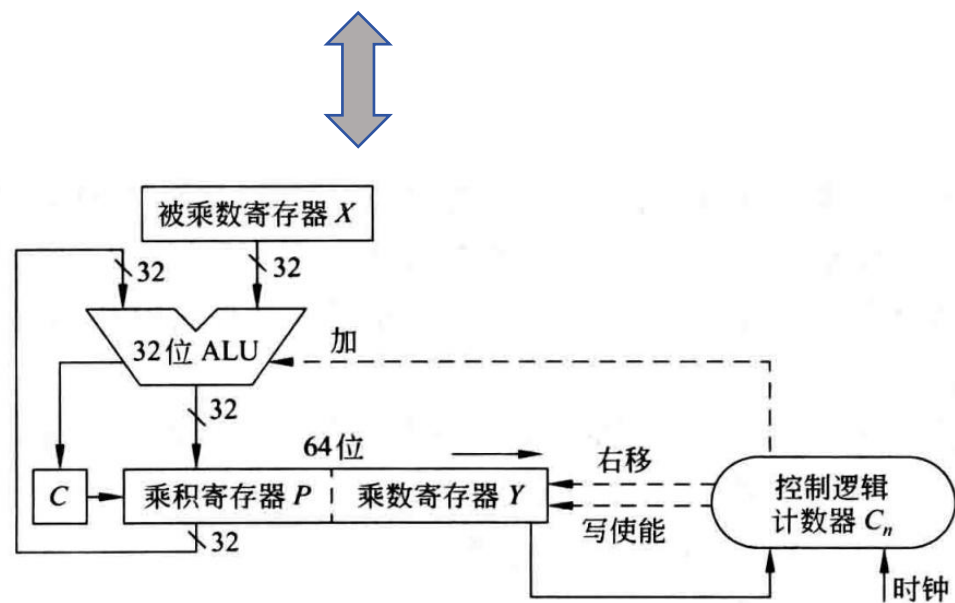
```
int add(int a, int b) {  
    int tmp;  
    while (b) {  
        tmp = a ^ b;  
        b = ((a & b) << 1);  
        a = tmp;  
    }  
    return a;  
}
```



乘法指令

// 软件实现乘法指令（基于移位 + 加法实现）

```
long mul(int a, int b) {  
    int ans = 0;  
    while (b) {  
        if (b & 1) ans = add(ans, a);  
        a = (a << 1);  
        b = (b >> 1);  
    }  
    return ans;  
}
```



选择软件实现？还是选择硬件实现？

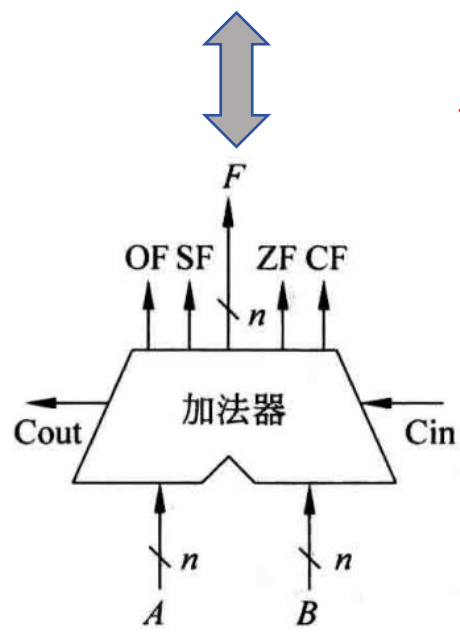
加法指令

// 软件实现加法指令（基于逻辑异或实现）

```
int add(int a, int b) {  
    int tmp;  
    while (b) {  
        tmp = a ^ b;  
        b = ((a & b) << 1);  
        a = tmp;  
    }  
    return a;  
}
```

软件实现：成本低，运行速度慢

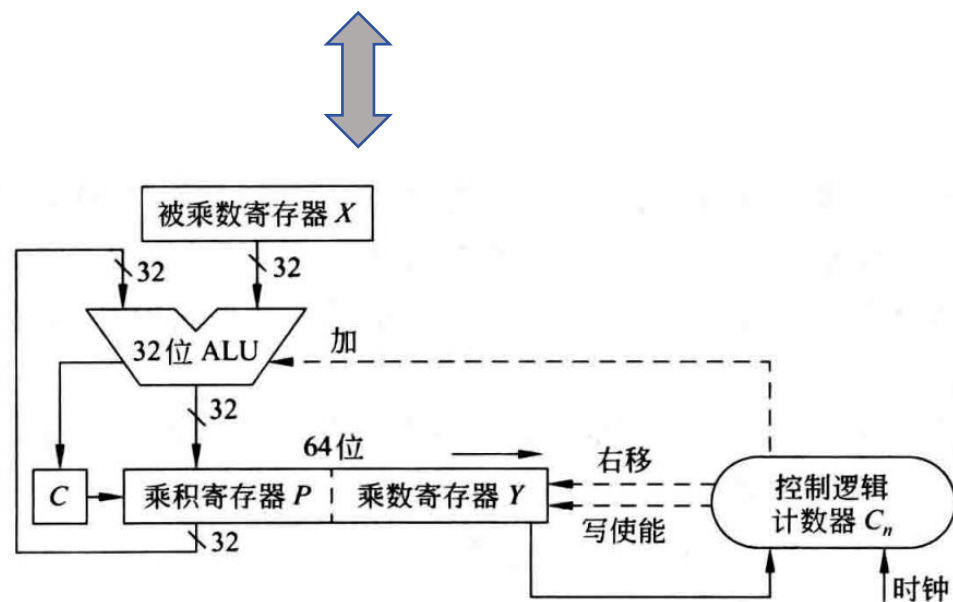
硬件实现：成本高，运行速度快



乘法指令

// 软件实现乘法指令（基于移位 + 加法实现）

```
long mul(int a, int b) {  
    int ans = 0;  
    while (b) {  
        if (b & 1) ans = add(ans, a);  
        a = (a << 1);  
        b = (b >> 1);  
    }  
    return ans;  
}
```



计算机体系结构中的 7 个伟大思想：

- ① 使用抽象分层简化设计
- ② 加速经常性事件 
- ③ 通过并行提高性能
- ④ 通过流水线提高性能
- ⑤ 通过预测提高性能
- ⑥ 存储层次
- ⑦ 通过冗余提高可靠性

加老汤微信，帮你 408 冲 120+



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

选择软件实现？ 还是选择硬件实现？

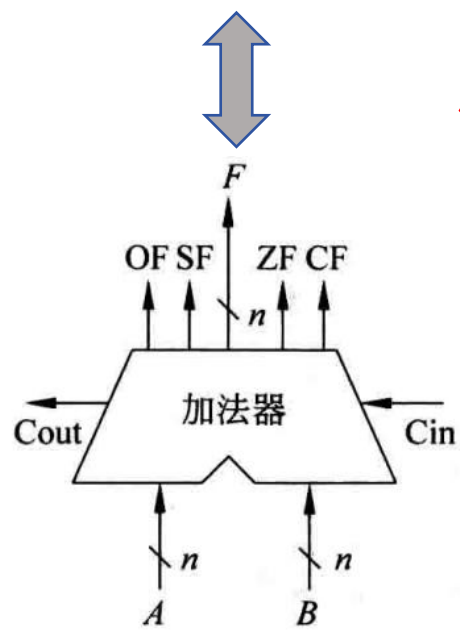
加法指令

// 软件实现加法指令（基于逻辑异或实现）

```
int add(int a, int b) {  
    int tmp;  
    while (b) {  
        tmp = a ^ b;  
        b = ((a & b) << 1);  
        a = tmp;  
    }  
    return a;  
}
```

软件实现：成本低，运行速度慢

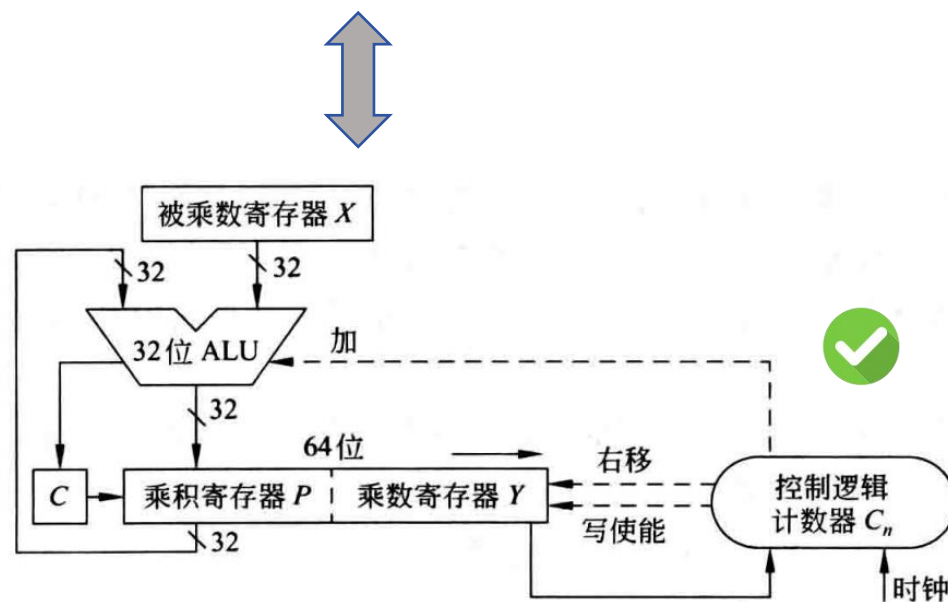
硬件实现：成本高，运行速度快



乘法指令

// 软件实现乘法指令（基于移位 + 加法实现）

```
long mul(int a, int b) {  
    int ans = 0;  
    while (b) {  
        if (b & 1) ans = add(ans, a);  
        a = (a << 1);  
        b = (b >> 1);  
    }  
    return ans;  
}
```



应用软件和系统软件



应用软件和系统软件

专门为数据处理、科学计算、事务管理、多媒体处理、工程设计以及过程控制等应用所编写的各类程序都称为**应用软件**

电子邮件

播放软件

游戏软件

炒股软件

文字处理软件

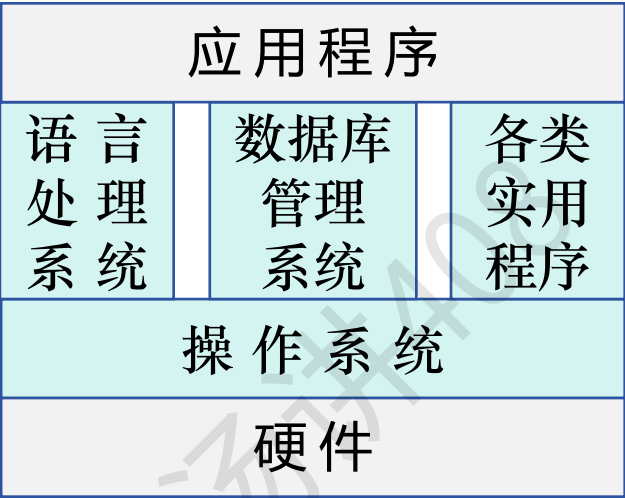
电子表格软件

演示文稿制作软件

各种 APP、网站等

应用软件和系统软件

系统软件包括为有效、安全地使用和管理计算机以及为开发和运行应用软件而提供的各种软件



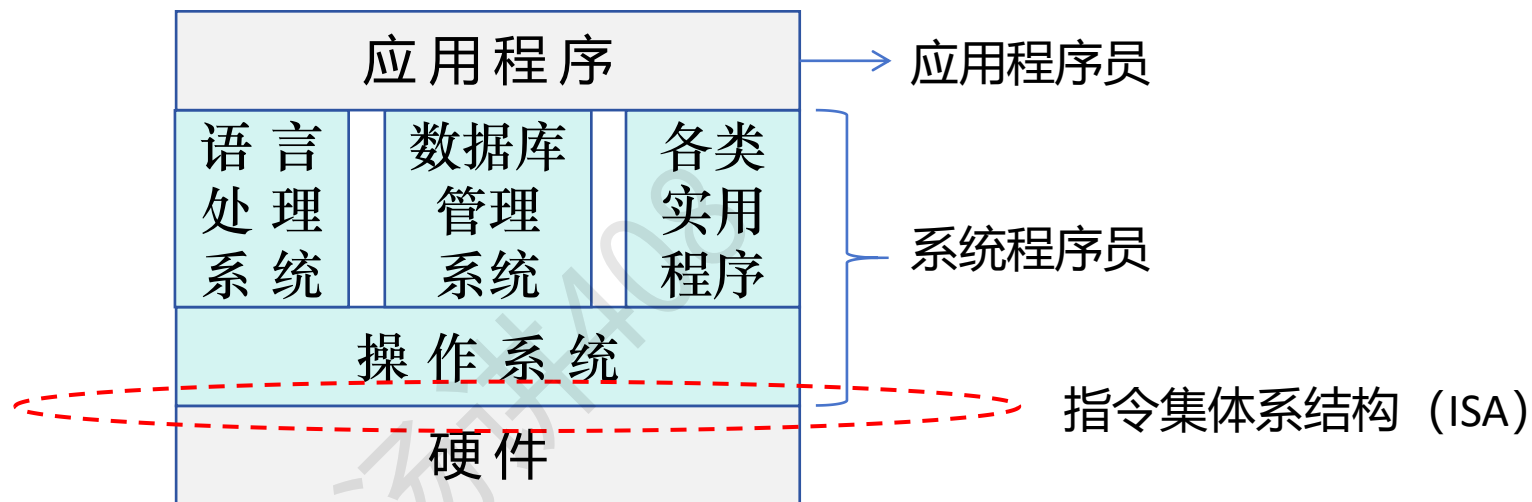
如磁盘碎片整理程序、备份程序等

操作系统 (如 Windows) 主要用来管理整个计算机系统的各种资源，包括对它们进行调度、管理、监视和服务等，操作系统还提供计算机用户和硬件之间的人机交互界面，并提供对应用软件的支持

语言处理系统 (如 C 语言编译器) 主要用于提供一个用高级语言编程的环境，包括源程序编辑、翻译、调试、链接、装入运行等功能

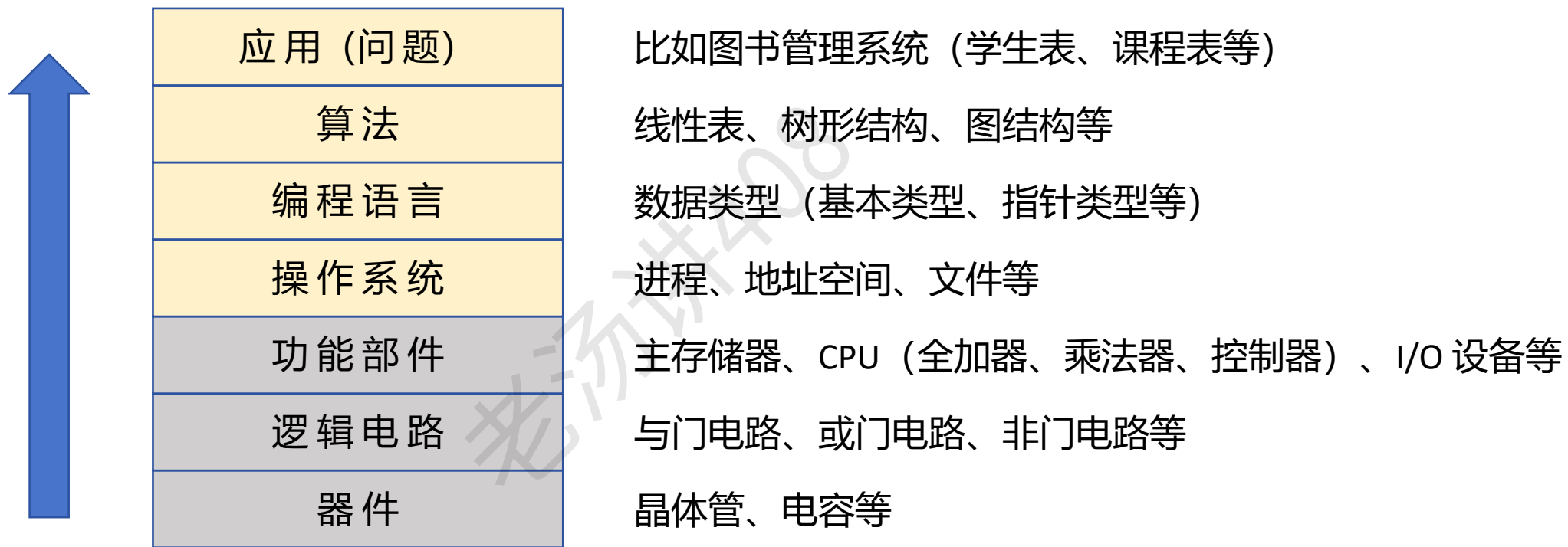
数据库管理系统 (如 Oracle) 是一种用于建立、使用和维护数据库的软件系统

应用程序员和系统程序员



系统软件包括为有效、安全地使用和管理计算机以及为开发和运行应用软件而提供的各种软件

计算机系统的层次结构（抽象分层思想）



中间的每一层的功能都是利用下面一层提供的基本服务来完成了，每一层又为上一层提供更强大的服务

计算机性能评估



时钟周期 (clock cycle)



时钟频率 (主频)



CPI (cycles per instruction)

三个性能指标



三种性能评估方式

① 指令执行速度

② 基准程序

③ 阿姆达尔 (Amdahl) 定律

计算机性能评估：CPU 时钟周期

时钟



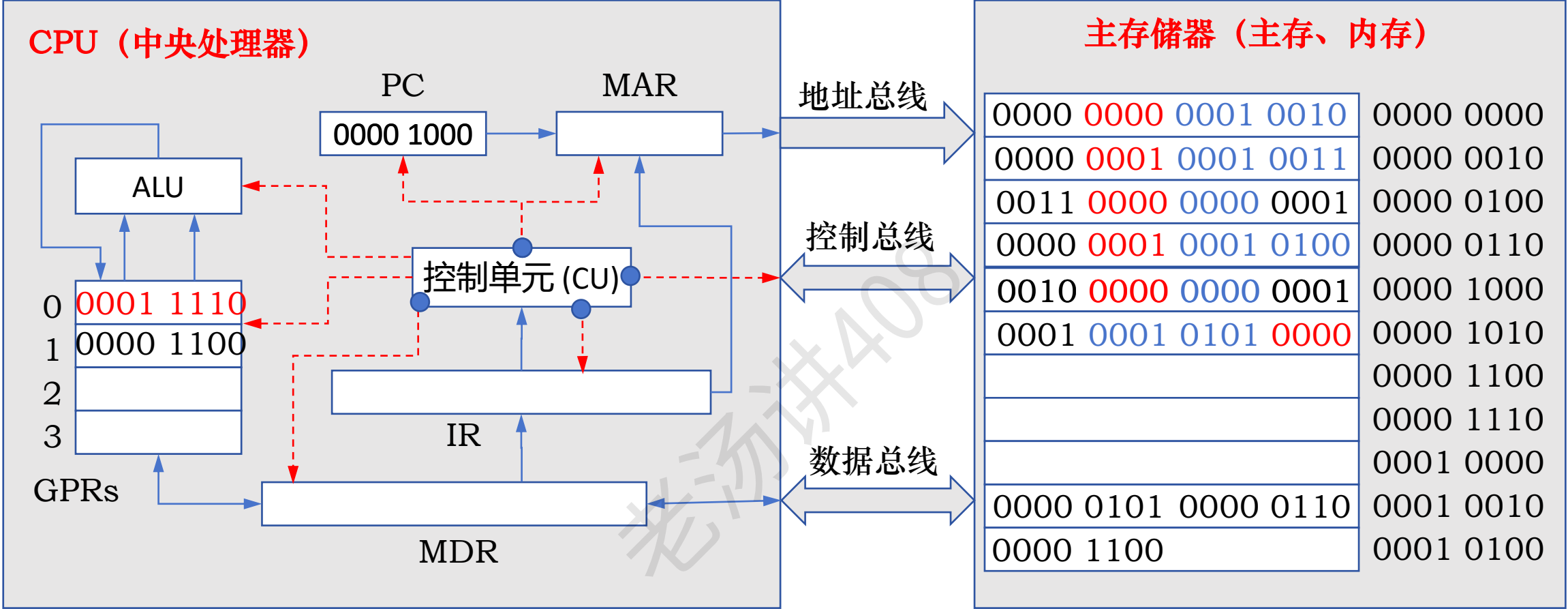
主脉冲信号



时钟周期

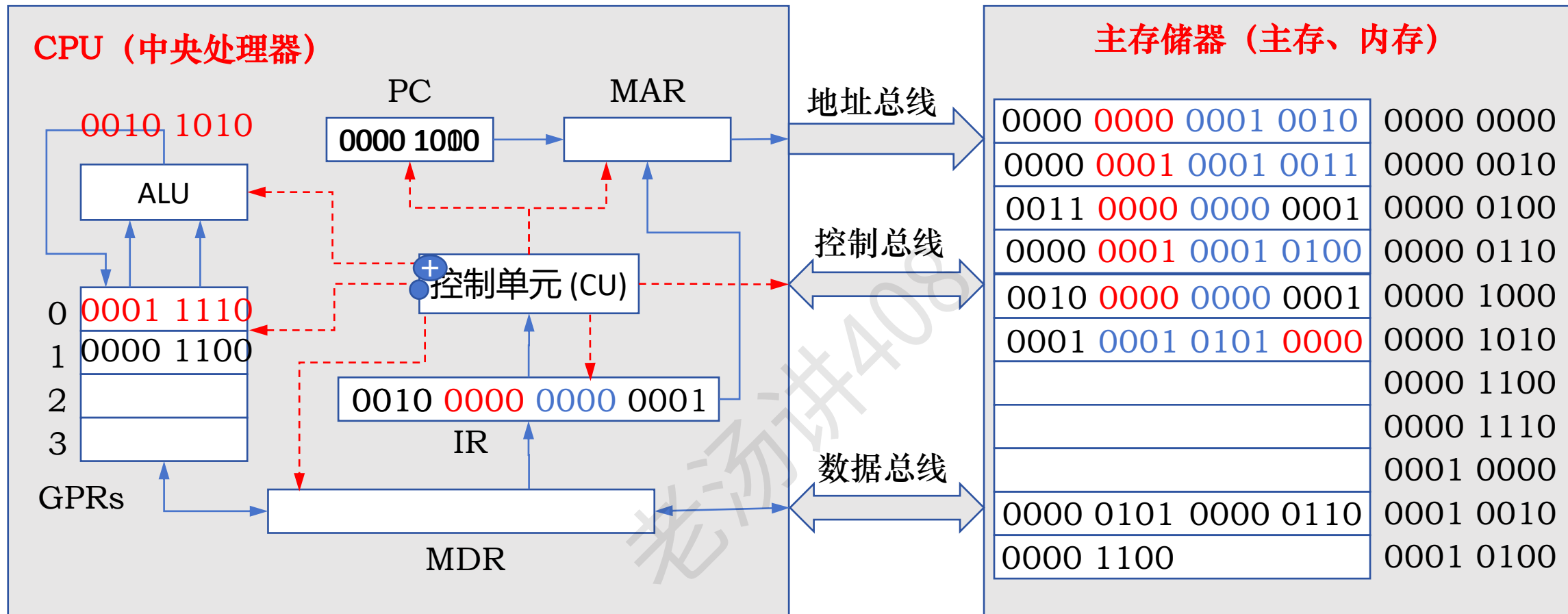
10ns、20ns 等等

计算机性能评估：CPU 时钟周期



取指阶段

计算机性能评估：CPU 时钟周期



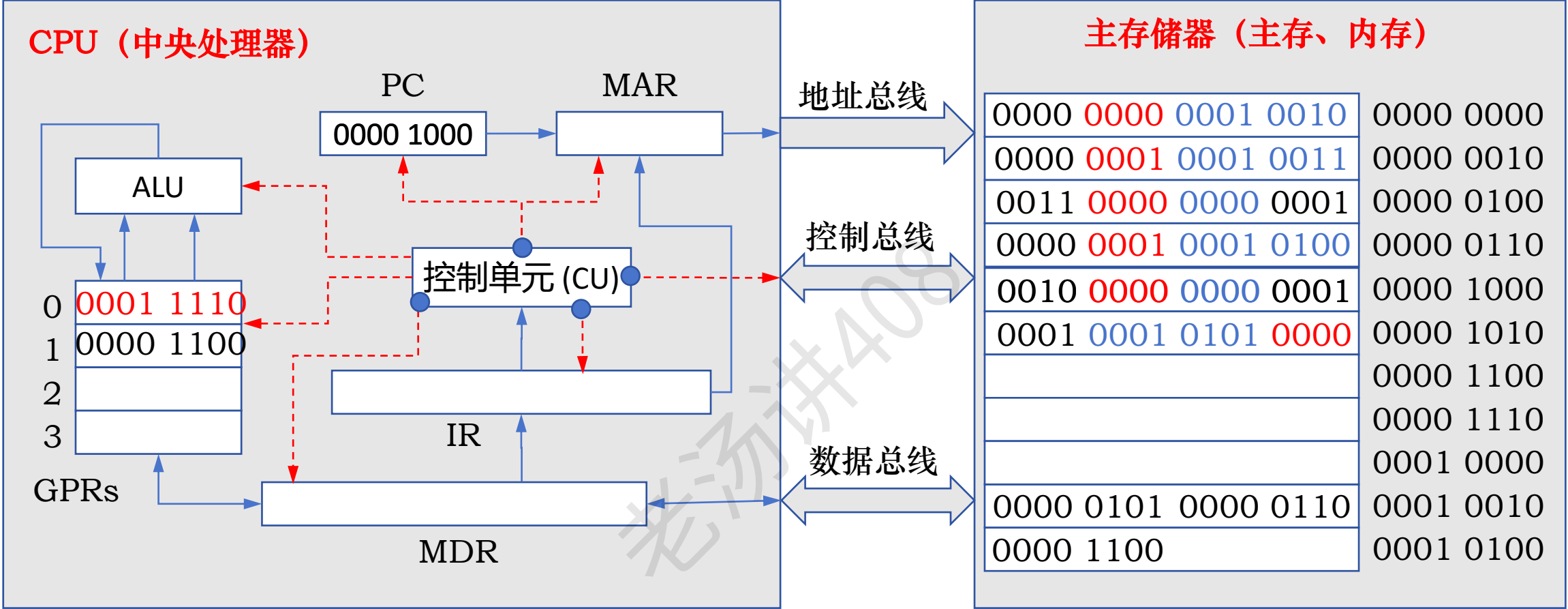
取指阶段

指令执行阶段

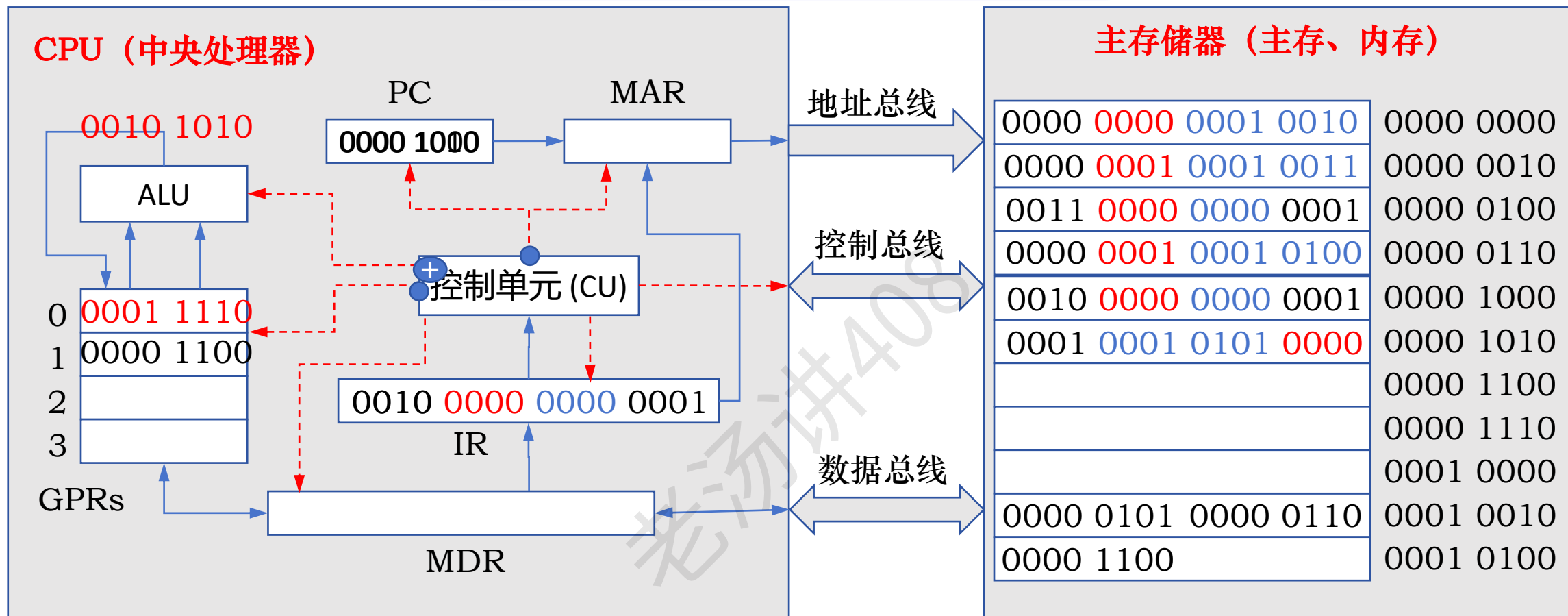
这些控制信号什么时候发出呢？

要作用多长时间呢？

计算机性能评估：CPU 时钟周期



计算机性能评估：CPU 时钟周期



5 个时钟周期内，执行完一条指令

计算机性能评估：CPU 时钟周期

注意哈，至于每一个时钟周期内执行哪些操作，发送哪些控制信号，这就设计到了 CPU 的数据通路和控制器的设计问题了，这个我们在第 5 章【中央处理器】的数据通路和控制器章节会详细讲解了

第一章：计算机系统概述

第二章：总线系统

第三章：主存储器

第四章：数据的表示与运算

第五章：指令系统



第六章：中央处理器 (CPU)

第七章：输入输出 (I/O) 系统

计算机性能评估：CPU 时钟周期

一个指令：5 个时钟周期

1 个时钟周期是 20ns 执行一条指令需要 100ns

1 个时钟周期是 10ns 执行一条指令需要 50ns

计算机性能评估：CPU 时钟频率（主频）

单位：赫兹（Hz）

$$CPU\text{主频} = \frac{1}{CPU\text{时钟周期}}$$

$$\frac{1s}{10ns} = \frac{1s}{10 \times 10^{-9}} = 10^8$$



$$CPU\text{主频} = 10^8 \text{ Hz} = 10^5 \text{ kHz} = 10^2 \text{ MHz} = 0.1 \text{ GHz}$$

1 秒内有多少个时钟周期

$$1 \text{ GHz} = 10^3 \text{ MHz}$$

$$1 \text{ MHz} = 10^3 \text{ kHz}$$

$$1 \text{ kHz} = 10^3 \text{ Hz}$$

CPU 主频越大，那么 CPU 的执行速度就越快

计算机性能评估：CPI (Cycles Per Instruction)

CPI (Cycles Per Instruction) : 执行一条指令需要的时钟周期数

- 一条特定指令 ———> CPI 就是指执行该指令所需的时钟周期数
- 一个程序 ———> 该程序所有指令执行所需的平均时钟周期数
- 一台机器 ———> 该机器指令集中所有指令执行所需的平均时钟周期数

计算机性能评估：CPI (Cycles Per Instruction)

CPI (Cycles Per Instruction) : 执行一条指令需要的时钟周期数

一条特定指令 —————> CPI 就是指执行该指令所需的时钟周期数

一个程序 —————> 该程序所有指令执行所需的平均时钟周期数

一台机器 —————> 该机器指令集中所有指令执行所需的平均时钟周期数

指令类型	所占比例	CPI
A	50%	2
B	20%	3
C	10%	4
D	20%	5

$$CPI = 2 * 50\% + 3 * 20\% + 4 * 10\% + 5 * 20\% = 3$$

计算机性能评估：CPU 执行时间

CPI 增加 ← 减少时钟周期

$$\text{CPU 执行时间} = \text{程序总指令条数} \times \text{CPI} \times \text{时钟周期}$$

减少



指令集体系结构 (ISA)

增加



CPU 结构改变

指令条数、CPI 和时钟周期三个因素是相互制约

计算机性能评估：CPU 执行时间

假设某个频繁使用的程序 P 在机器 M1 上运行需要 10s，M1 的时钟频率为 2GHz。设计人员想开发一台与 M1 具有相同 ISA 的新机器 M2。采用新技术可使 M2 的时钟频率增加，但同时也会使 CPI 增加。假定程序 P 在 M2 上的时钟周期数是在 M1 上的 1.5 倍，则 M2 的时钟频率至少达到多少才能使程序 P 在 M2 上的运行时间缩短为 6s？

CPU 执行时间 = 程序总指令条数 × CPI × 时钟周期

M1 机器: 10s n m $\frac{1}{2GHz} s$

M2 机器: 6s n 1.5m $\frac{1}{x} s$

$$\frac{10s}{6s} = \frac{n * m * \frac{1}{2GHz} s}{n * 1.5m * \frac{1}{x} s} = \frac{x}{1.5 * 2GHz} \quad \Rightarrow \quad x = 5GHz$$

计算机性能评估



时钟周期 (clock cycle)



时钟频率 (主频)



CPI (cycles per instruction)

三个性能指标



三种性能评估方式

① 指令执行速度

② 基准程序

③ 阿姆达尔 (Amdahl) 定律

方式一：用指令执行速度进行性能评估

IPS (Instructions Per Second), 即平均每秒执行多少条指令

平均 CPI 是执行一条指令平均需要的时钟周期



一条指令执行平均时间 = 平均 CPI * 时钟周期



$$IPS = \frac{1}{\text{平均}CPI * \text{时钟周期}} = \frac{\text{主频}}{\text{平均}CPI}$$

MIPS (Million Instructions Per Second) : 平均每秒执行多少百万条指令

2 MIPS

$$MIPS = \frac{IPS}{10^6} = \frac{1}{\text{平均}CPI * \text{时钟周期} * 10^6} = \frac{\text{主频}}{\text{平均}CPI * 10^6}$$

方式一：用指令执行速度进行性能评估

假定某程序 P 编译后生成的目标代码由 A、B、C、D 四类指令组成，它们在程序中所占的比例分别为 43%、21%、12%、24%，已知它们的 CPI 分别为 1、2、2、2。现重新对程序 P 进行**编译优化**，**生成的新目标代码中 A 类指令条数减少了 50%**，其他类指令的条数没有变。请回答下列问题。

(1) 编译优化前后程序的 CPI 各是多少？

(2) 假定程序在一台主频为 50MHz 的计算机上运行，则优化前后的 MIPS 各是多少？

优化前：

指令类型	所占比例	CPI
A	43%	1
B	21%	2
C	12%	2
D	24%	2

优化后：

指令类型	所占比例	CPI
A	27%	1
B	27%	2
C	15%	2
D	31%	2

$$\text{A 类指令: } 21.5 / (21.5 + 21 + 12 + 24) = 27\%$$

$$\text{B 类指令: } 21 / (21.5 + 21 + 12 + 24) = 27\%$$

$$\text{C 类指令: } 12 / (21.5 + 21 + 12 + 24) = 15\%$$

$$\text{D 类指令: } 24 / (21.5 + 21 + 12 + 24) = 31\%$$

方式一：用指令执行速度进行性能评估

假定某程序 P 编译后生成的目标代码由 A、B、C、D 四类指令组成，它们在程序中所占的比例分别为 43%、21%、12%、24%，已知它们的 CPI 分别为 1、2、2、2。现重新对程序 P 进行**编译优化**，**生成的新目标代码中 A 类指令条数减少了 50%**，其他类指令的条数没有变。请回答下列问题。

(1) 编译优化前后程序的 CPI 各是多少？

(2) 假定程序在一台主频为 50MHz 的计算机上运行，则优化前后的 MIPS 各是多少？

优化前：

指令类型	所占比例	CPI
A	43%	1
B	21%	2
C	12%	2
D	24%	2

优化后：

指令类型	所占比例	CPI
A	27%	1
B	27%	2
C	15%	2
D	31%	2

$$\text{CPI} = 43\% * 1 + 21\% * 2 + 12\% * 2 + 24\% * 2 = 1.57$$

$$\frac{\text{主频}}{\text{平均CPI} * 10^6} = \frac{50\text{MHz}}{1.57 * 10^6} = \frac{50 * 10^6 \text{Hz}}{1.57 * 10^6} = 31.8 \text{MIPS}$$

$$\text{CPI} = 27\% * 1 + 27\% * 2 + 15\% * 2 + 31\% * 2 = 1.73$$

$$\frac{\text{主频}}{\text{平均CPI} * 10^6} = \frac{50\text{MHz}}{1.73 * 10^6} = 28.9 \text{MIPS}$$

FLOPS (FLOating-point operations Per Second)

IPS (Instructions Per Second) : 平均每秒执行多少条指令

FLOPS (FLOating-point operations Per Second) : 平均每秒执行多少次浮点运算操作

$$1.23 \times 10^2 + 4.56 \times 10^3$$

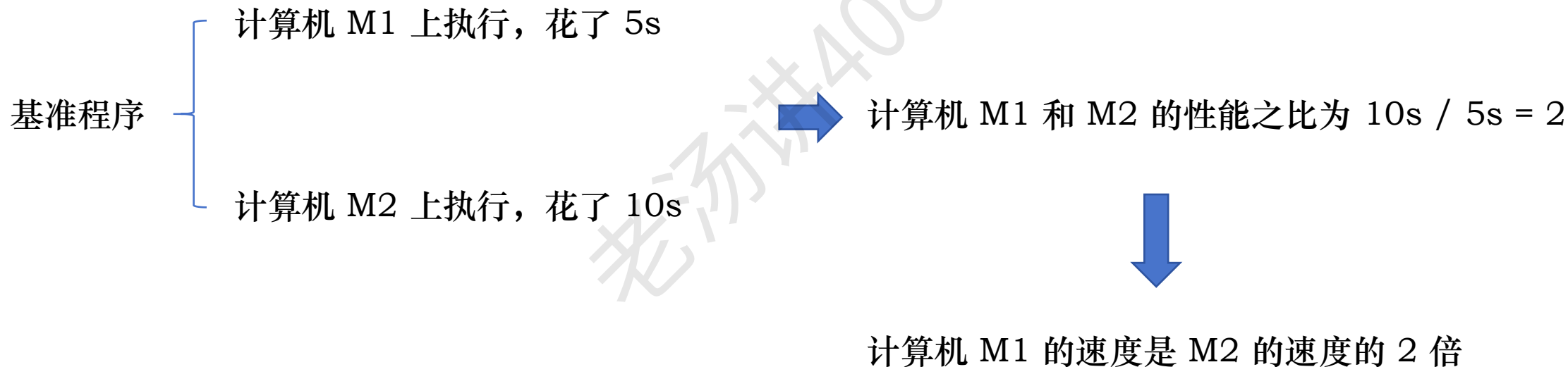
$$1.23 \times 10^2 \times 4.56 \times 10^3$$

- MFLOPS (Million FLOPS), 即每秒执行多少百万 (10^6) 次浮点运算
- GFLOPS (Giga FLOPS), 即每秒执行多少十亿 (10^9) 次浮点运算
- TFLOPS (Tera FLOPS), 即每秒执行多少万亿 (10^{12}) 次浮点运算
- PFLOPS (Peta FLOPS), 即每秒执行多少千万亿 (10^{15}) 次浮点运算
- EFLOPS (Exa FLOPS), 即每秒执行多少百京 (10^{18}) 次浮点运算(1京=1亿亿= 10^{16})
- ZFLOPS (Zetta FLOPS), 即每秒执行多少十万京 (10^{21}) 次浮点运算

$$1Z = 10^3E = 10^6P = 10^9T = 10^{12}G = 10^{15}M = 10^{21}$$

方式二：用基准程序进行性能评估

CPU 执行时间 = 程序总指令条数 × CPI × 时钟周期



方式二：用基准程序进行性能评估

基准程序是专门用来进行性能评价的**一组**程序

SPEC 测试程序集

整数测试程序集 SPECint

浮点测试程序集 SPECfp

CPU 性能测试集 (SPEC CPU2000)

Web 服务器性能测试集 (SPECweb99)

方式三：阿姆达尔 (Amdahl) 定律

基本思想：对系统中某个硬件部分，或者软件中的某部分进行更新所带来的系统性能的提高程度，取决于该硬件部分或软件部分被使用的频率或其执行时间占总执行时间的比例

$$\text{改进后的执行时间} = \frac{\text{改进部分执行时间}}{\text{改进部分的改进倍数}} + \text{未改进部分执行时间}$$

$$\text{整体改进倍数} = \frac{1}{\frac{\text{改进部分执行时间比例}}{\text{改进部分的改进倍数}} + \text{未改进部分执行时间比例}}$$

方式三：阿姆达尔 (Amdahl) 定律

例：假定计算机中的整数乘法器改进后可以加快 10 倍，若整数乘法指令在程序中占 40%，则整体性能能改进多少倍？如果整数乘法指令在程序中所占比例达 90%，则整体性能能改进多少倍？

$$\text{整体改进倍数} = \frac{1}{\frac{\text{改进部分执行时间比例}}{\text{改进部分的改进倍数}} + \text{未改进部分执行时间比例}}$$

$$\text{整体改进倍数} = \frac{1}{\frac{0.4}{10} + 0.6} = 1.56$$

$$\text{整体改进倍数} = \frac{1}{\frac{0.9}{10} + 0.1} = 5.26$$